pdf4tcllib

Reports, tables and forms with pdf4tcl

2026-10-26

Contents

1 What pdf4tcllib Adds **9**
1.1 Introduction 9
1.2 What the Examples Use 9
1.3 The Modules 9
1.4 A First Report 10
1.5 How to Read the Book 11
2 Setup and Fonts **12**
2.1 Introduction 12
2.2 Loading the Module 12
2.3 Scripts in UTF-8 12
2.4 fonts::init 13
2.4.1 Which Files It Looks For 13
2.4.2 Where It Looks 13
2.4.3 Where the Font Comes From 14
2.5 Checking What Is Loaded 14
2.6 Font Names Always Through the Accessor Procedures 15
2.7 256 Characters or CID 16
2.7.1 Choosing the Mode on the First Call 16
2.8 Summary 16
3 Safe Unicode Output **17**
3.1 Introduction 17
3.2 What sanitize Does 17
3.2.1 Monospaced Font: -mono 1 18
3.3 Who Calls sanitize 18
3.3.1 German Quotation Marks 20
3.4 The Font Used for Drawing Is Not the One sanitize Asks About 20
3.5 Emoji 21
3.6 Knowing What Was Missing 21
3.7 Summary 21
4 Units and Page Context **22**
4.1 Introduction 22
4.2 Converting Units 22
4.2.1 Keep pdf4tcl in Points 22
4.3 page::context 22
4.3.1 What the Dict Contains 23
4.3.2 top and bottom Depend on the Direction 25
4.3.3 Line by Line 25
4.3.4 The First Line 26
4.4 Header, Footer and Page Number 26
4.5 Guide Lines During Development 26
4.6 Summary 27
5 Setting Text **27**
5.1 Introduction 27
5.2 Measuring Width 27
5.3 Wrapping 28

5.3.1 A Word That Does Not Fit 29
5.4 Truncating or Shrinking 29
5.5 A Paragraph in One Go 30
5.6 Text Across Pages 30
5.7 Summary 31
6 Headers, Footers, Page Numbers 32
6.1 Introduction 32
6.2 One Place for the Page Change 32
6.3 Page 2 of 3 32
6.4 Page Number Without a Word 34
6.5 Summary 34
7 Column Flow and Watermarks 35
7.1 Introduction 35
7.2 Text in Columns 35
7.2.1 The Page Change Belongs to the Caller 37
7.2.2 The First Baseline 37
7.2.3 What Does Not Work 37
7.3 Watermarks 37
7.4 Summary 38
8 Tables with Page Breaks 38
8.1 Introduction 38
8.2 Columns and Rows 39
8.3 A List Across Two Pages 39
8.3.1 What the Page Break Needs 40
8.4 Appearance 41
8.4.1 Individual Rows and Cells 41
8.5 One Row, One Value: Truncation 41
8.6 Merged Cells 42
8.7 How Wide Will the Table Be? 42
8.8 Worth Knowing 43
8.9 Summary 43
9 Group Totals and Crosstabs 43
9.1 Introduction 43
9.2 The Example 43
9.3 table::group 45
9.3.1 The Data Must Be Sorted 46
9.3.2 Empty Fields Are Not Zero 46
9.3.3 The Label Spans Several Columns 46
9.3.4 Formatting Totals 46
9.4 table::pivot 47
9.5 Summary 47
10 Charts 47
10.1 Introduction 48
10.2 Data and Box 48
10.3 Three Charts on One Page 48
10.3.1 Bars 49
10.3.2 Lines 49

10.3.3 Pie 49
10.4 Options 50
10.4.1 The Scale 50
10.4.2 Numbers with a Decimal Comma 50
10.5 Accessibility 50
10.6 Summary 50
11 Table of Contents 51
11.1 Introduction 51
11.2 The Content as a Script 51
11.2.1 What the Script Must Not Do 53
11.2.2 Which Context Applies 53
11.3 Headings 53
11.4 Options of toc::document 53
11.5 A Cover Page in Front 54
11.6 Summary 55
12 Form Layout 55
12.1 Introduction 56
12.2 The Building Blocks 56
12.3 An Order Form 56
12.3.1 labelField 58
12.3.2 row 58
12.3.3 orderTable and sumLine 58
12.4 Fonts in Forms 58
12.5 Border and Background 59
12.6 Settings 59
12.7 Worth Knowing 60
12.8 Summary 60
13 Label Sheets 60
13.1 Introduction 60
13.2 The Sheets 61
13.3 Addresses on Two Sheets 61
13.3.1 render 62
13.3.2 Outlines for Checking 63
13.4 Text That Is Too Long 63
13.4.1 -clip as a Safeguard 64
13.5 Calibrating the Printer 64
13.6 Summary 64
14 Images 64
14.1 Introduction 64
14.2 Loading 65
14.3 An Image in a Frame 65
14.4 The Example 65
14.4.1 Resolution 67
14.5 Gallery 67
14.6 Images from Tk 68
14.7 Summary 68
15 Arrows and Callouts 68

15.1 Introduction 68
15.2 The Procedures 69
15.3 A Flow Diagram 69
15.3.1 Straight Arrows 71
15.3.2 Bent Arrows 71
15.4 Callouts 71
15.5 Rounded Corners 72
15.6 Accessibility 72
15.7 Summary 72
16 Formulas 72
16.1 Introduction 72
16.2 The Font 73
16.3 The LaTeX Subset 73
16.3.1 Spaces Do Not Count 73
16.3.2 The Minus Sign 74
16.4 Inline and Displayed 74
16.4.1 Inline 75
16.4.2 Displayed 75
16.5 The eqn Notation 75
16.6 Summary 76
17 Treeview 76
17.1 Introduction 76
17.2 Table or Outline 76
17.3 A Stock List 77
17.3.1 The Script Needs an exit 78
17.4 render 79
17.4.1 Page Breaks 79
17.5 outline 79
17.5.1 Bookmarks 80
17.6 Both in One Document 80
17.7 Summary 80
18 Accessible PDFs 81
18.1 Introduction 81
18.2 What a Document Needs for PDF/UA 81
18.3 A Report with a Structure Tree 81
18.4 What the Building Blocks Do Themselves 83
18.4.1 Headings 84
18.4.2 Charts 84
18.4.3 Tables 84
18.5 Classifying Your Own Drawing 84
18.5.1 getUntaggedCount 85
18.6 Checking 85
18.7 Summary 85
19 Saving the Graphics State 85
19.1 Introduction 85
19.2 What Gets Lost 86
19.3 state::with 87

19.3.1 By Hand 87
19.4 Why Not gsave and grestore 87
19.5 Summary 87
20 Measuring and Stacking 87
20.1 Introduction 88
20.2 Measuring by Drawing 88
20.2.1 pages and overflow 88
20.3 The Example 88
20.4 layout::stack 90
20.4.1 What keep Does and What It Does Not 91
20.4.2 The Header and the Top Margin 91
20.5 Summary 92
21 Case Study: Warehouse Report 92
21.1 Introduction 92
21.2 The Script 92
21.3 1. The Data First 95
21.4 2. The Page Break 96
21.5 3. The Content 96
21.6 4. The Document 96
21.7 What Is Deliberately Missing 97
21.8 Summary 97
22 Pitfalls 97
22.1 Setup 97
22.2 Text and Characters 97
22.3 Setting Text 98
22.4 Pages and Columns 98
22.5 Tables and Charts 98
22.6 Documents 98
22.7 Accessibility and Structure 99
22.8 Page 99
23 API per Module 100
23.1 Packages 100
23.2 Fonts: fonts 100
23.3 Characters: unicode 101
23.4 Setting Text: text 101
23.5 Formulas: math 102
23.6 Page: page, units 102
23.7 Structure Tree: tag 102
23.8 State, Measuring, Stacking: state, measure, layout 102
23.9 Tables: table 103
23.10 Drawing: drawing 103
23.11 Images: image 104
23.12 Form Layout: form 104
23.13 Charts: chart (pdf4tclchart) 105
23.14 Column Flow: flow (pdf4tclflow) 105
23.15 Table of Contents: toc (pdf4tcltoc) 105
23.16 Labels: labels (pdf4tcllabels) 106

23.17 Fillable Forms: forms (pdf4tclforms) 106
23.18 Tk Widgets: canvas, tablelist, textwidget, treeview 106
23.19 Miscellaneous 107
24 Glossary 107
25 Error Messages 108
25.1 For All Building Blocks 108
25.1.1 ...: unknown option -... (allowed: ...) / (known: ...) 108
25.1.2 ...: expected an even number of option/value words 108
25.1.3 ...: expected option/value pairs 108
25.2 Page and Fonts 108
25.2.1 unknown paper size: ... (known: ...) 108
25.2.2 pdf4tcllib::page::footer: unknown option ... 109
25.2.3 pdf4tcllib::fonts: No TTF fonts found, falling back to Helvetica 109
25.2.4 pdf4tcllib::fonts: TTF error: ... / Fallback on Helvetica 109
25.2.5 pdf4tcllib::unicode: sanitize: at least one character is missing from the font and was replaced with '?' (first: U+...). ... 109
25.2.6 pdf4tcllib::math: unknown symbol '\...' rendered as literal text 109
25.3 Text, Measuring, Stacking 109
25.3.1 text::fitSize: unknown option ... 109
25.3.2 layout::stack: an entry needs a script (or must be {gap }) 109
25.3.3 measure::block: unknown option ... 109
25.4 Tables 109
25.4.1 table::draw: -orient 0 is not supported. ... 109
25.4.2 table::draw: rows ... to ... are joined by rowspan and need ... pt, more than a page holds (... pt) 109
25.4.3 table::group: -by ... is outside the row (0.....) 110
25.4.4 table::group: the rows are not sorted by column ...; a group falls into several blocks and each gets its own subtotal 110
25.4.5 table::pivot: -agg must be sum, count, avg, min or max, not "..." 110
25.4.6 orderTable: column widths total ... pt, the text width is only ... pt -- the table would be drawn past the edge of the page 110
25.4.7 orderTable: ... header(s) but only ... column width(s) 110
25.4.8 tablelist::render: ... could not be reproduced (...). ... 110
25.5 Charts 110
25.5.1 chart::...: no data 110
25.5.2 chart::...: data must be label/value pairs or a list of {label value}, got ... element(s) 110
25.5.3 chart::bar: value for "..." is not a number: "..." 110
25.5.4 chart::bar: box too small -- ... x ... pt leaves no room for axis and labels 110
25.5.5 chart::pie: negative value for "..." -- a slice of a whole cannot be negative 110
25.5.6 chart::pie: the values add up to 0 110
25.6 Column Flow and Table of Contents 110
25.6.1 flow::...: -columns must be at least 1 110
25.6.2 flow::boxes: ... columns with a gap of ... pt leave no width in ... pt 111
25.6.3 toc::heading: level must be 1..6, got "..." 111
25.6.4 toc::document: the contents was calculated as ... page(s) and written as ... -- every page number in it would be off by ... 111
25.7 Images and Drawing 111

25.7.1 image::place: -fit must be contain, cover or stretch, not "..." 111
25.7.2 image::place: width and height must be positive 111
25.7.3 image::place: image ... has no size 111
25.7.4 image::place: ... at ... dpi (...x... pixels on ...x... points), below ... 111
25.7.5 image::gallery: -cols and -rows must be at least 1 111
25.7.6 image::gallery: the frame is too small for ...x... cells 111
25.7.7 pdf4tcllib::image needs Tk: ... 111
25.7.8 drawing::arrow: -at must be end, start or both, not "..." 111
25.7.9 drawing::callout: width and height must be positive 111
25.7.10 drawing::watermark: no text 111
25.7.11 canvas::place: no such widget: ... 111
25.7.12 canvas::place: no room left for the drawing after the caption 111
25.7.13 canvas::fitPage: the context leaves no height 111
25.8 Labels 111
25.8.1 unknown label sheet "..."; known: ... 112
25.8.2 label format ...: missing "..." 112
25.8.3 label format ...: paper "roll" means one label per page, so cols and rows must both be 1 112
25.8.4 argSpec must name five variables: {x y w h record} 112
25.8.5 -start must be between 0 and ... for sheet ... 112
25.8.6 -only: "..." is not a position on sheet ... (0.....) 112
25.8.7 -only names ... position(s) for ... record(s) 112
25.8.8 -frame: 0, 1 or layer, not "..." 112
25.8.9 -frame layer needs pdf4tcl 0.9.4.70 or newer: ... ignores -print 0, the grid would print 112
25.8.10 -frame layer under -pdfa ...: PDF/A forbids /AS, so -print 0 has no effect and the grid would print; use -frame 1 on a separate proof sheet 112
25.9 Fillable Forms 112
25.9.1 forms::read: cannot read "..." / forms::write: cannot read "..." 112
25.9.2 forms::write: ... has no field(s): ... 112
25.9.3 forms::write: nothing to fill in 112
25.9.4 pdf4tclforms::renderSchema: spec needs 'sections' key 112
25.9.5 pdf4tclforms::renderSchema: 'sections' must be a dict (name -> section), not a list of sections 112
25.9.6 pdf4tclforms: field entry must be a dict, got: ... 112
25.9.7 pdf4tclforms::template: unknown template '...' (callnote inventory checklist order) 113
25.10 Tk Widgets 113
25.10.1 treeview::render: no such widget: ... / treeview::outline: no such widget: ... 113
25.11 From pdf4tcl 113
25.11.1 tagged: the ... element is empty -- no content, no child element and no annotation. ... 113
25.11.2 ... must contain at least one ... 113
26 License 113

1 What pdf4tcllib Adds

1.1 Introduction

pdf4tcl draws: text at a position, a line from here to there, an image into a rectangle. Anyone who builds a report with it soon writes the same helper procedures as everyone else: computing page margins, wrapping text, carrying a table across the page boundary, setting header and footer, catching special characters.

pdf4tcllib is this layer as a library. It does not draw itself; it calls the same methods of the pdf4tcl object that your own code uses. The two can therefore be mixed: a table from pdf4tcllib, below it a line drawn directly with pdf4tcl.

This book is a workshop book. Each chapter solves a task that comes up again and again in reports, forms and lists, with an example that runs exactly as printed. How pdf4tcl itself works (coordinates, fonts, forms, encryption) is explained in the book *pdf4tcl*; here it appears only where the task needs it.

1.2 What the Examples Use

Part	Version
pdf4tcllib	0.6.7
pdf4tcl	0.9.4.73 (pdf4tcllib requires at least 0.9.4.23)
Tcl	8.6 and 9.0
Font	DejaVu Sans Condensed (TrueType)

All examples are available as scripts in the folder `beispiele/` next to the book, with the chapter number in the name. The output shown in the text comes from these scripts.

1.3 The Modules

pdf4tcllib is a single file, `pdf4tcllib-0.6.7.tm`, with several namespaces. Next to it are packages that build on it and are loaded individually.

Namespace	Task	Chapter
fonts	find and load a TrueType font, return font names	Fonts
unicode	replace characters the font does not have	Unicode
units, page	units, type area, header and footer	Page context
text	wrapping, measuring width, truncating, superscript and subscript	Setting text

Namespace	Task	Chapter
table	tables with page breaks, groups, crosstabs	Tables
drawing, image	shapes, gradients, arrows, images	Images, Arrows
form	labeled form fields in rows	Form layout
math	formulas in text	Formulas
tag, state, measure, layout	accessibility, graphics state, measuring height in advance, stacking blocks	Accessible, State, Stacking

Package	Task	Chapter
pdf4tcltoc	table of contents with real page numbers	Table of contents
pdf4tclchart	bar, line and pie charts	Charts
pdf4tclflow	text across columns and pages	Column flow
pdf4tcllabels	label sheets	Label sheets
pdf4tcltreeview	a ttk::treeview as a table or outline	Treeview
pdf4tclforms	fillable forms from a description	only Appendix B
pdf4tcltable	a tablelist widget as PDF	only Appendix B
pdf4tcltext	a Tk text widget as PDF	only Appendix B
pdf4tclcanvas	a Tk canvas drawing as PDF	only Appendix B

The last four have no chapter of their own in the book; their calls are in Appendix B, their messages in Appendix D.

1.4 A First Report

The smallest useful example: header, heading, a wrapped paragraph, footer with page number.

```
# 010-erstes-dokument.tcl -- ein Bericht in wenigen Zeilen
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init
set ctx [pdf4tcllib::page::context a4 -margin 20]

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
pdf4tcllib::page::header $pdf $ctx "Monatsbericht"

set x [dict get $ctx left]
```

```
set y [expr {[dict get $ctx top] + 20}]
set w [dict get $ctx text_w]

$pdf setFont 16 [pdf4tcllib::fonts::fontSansBold]
pdf4tcllib::unicode::safeText $pdf "Umsatz im Oktober" -x $x -y $y
set y [expr {$y + 28}]

set absatz "Der Umsatz stieg gegenüber dem Vormonat um 4 %. Am\
    stärksten wuchs der Bereich Wartung, der jetzt ein Drittel des\
    Umsatzes ausmacht. Die Zahlen sind vorläufig; die Schlussrechnung\
    folgt mit dem Quartal."
set y [pdf4tcllib::text::writeParagraph $pdf $absatz $x $y $w 11]
puts "nächste Zeile bei y = $y"

pdf4tcllib::page::footer $pdf $ctx "Vertraulich" 1
$pdf endPage
$pdf write -file 010-erstes-dokument.pdf
$pdf destroy
```

Output:

```
pdf4tcllib::fonts: TTF italic faces loaded
pdf4tcllib::fonts: TTF faces loaded from
    /usr/share/fonts/truetype/dejavu
nächste Zeile bei y = 152.69291338582678
```

The first two lines are written to stderr by `fonts::init`. They say that a TrueType font was found and where it came from.

What the lines do:

- `fonts::init` searches for a TrueType font once per process and registers it with pdf4tcl. After that, `fonts::fontSans`, `fonts::fontSansBold` etc. return the names under which it is registered.
- `page::context` computes the type area from paper and margin and returns a dict: `left`, `top`, `text_w` and further values, all in points.
- `page::header` and `page::footer` set header and footer with a thin line, both inside the margin.
- `unicode::safeText` writes text like `$pdf text`, but first replaces characters the font does not have.
- `text::writeParagraph` wraps the paragraph to the width `w` and returns the y position after the last line. That is where the next element goes.

`-orient true` on `pdf4tcl::new` is part of it: y grows downward, as in the Tk canvas. `page::context` assumes the same unless told otherwise. How the two must fit together is covered in the chapter Units and Page Context.

1.5 How to Read the Book

- **Part 1** (this chapter through chapter 4) sets up what all other chapters assume: font, Unicode,

page context.
- **Parts 2 to 4** are tasks: text, tables, charts, table of contents, labels, forms. They can be read individually.
- **Part 5** makes a document accessible and shows how to measure blocks in advance and stack them one below the other.
- The **case study** builds a report from the data to the table of contents.
- The **appendices** collect pitfalls, the API per module and the messages.

Tested with pdf4tcllib 0.6.7, pdf4tcl 0.9.4.73, Tcl 8.6.18 and 9.0.4, Debian with DejaVu fonts.

Next chapter: Setup and Fonts

2 Setup and Fonts

2.1 Introduction

pdf4tcllib needs two things: the package pdf4tcl and a TrueType font. Without TrueType it carries on with Helvetica, but then all characters outside Latin-1 are missing, and the PDF cannot be archived as PDF/A. This chapter sets up both and shows how to check what `fonts::init` has found.

2.2 Loading the Module

pdf4tcllib is a Tcl module: a file `pdf4tcllib-0.6.7.tm` in the folder `lib/`, next to it the add-on packages (`pdf4tcltoc-0.1.tm` etc.). Tcl finds them through the module path:

```
tcl::tm::path add /pfad/zu/pdf4tcllib/lib
package require pdf4tcllib
```

Or, without changing the script, through the environment, depending on the Tcl version:

```
export TCL8_6_TM_PATH=/pfad/zu/pdf4tcllib/lib
export TCL9_0_TM_PATH=/pfad/zu/pdf4tcllib/lib
```

pdf4tcl itself is a classic package with `pkgIndex.tcl`; its folder belongs in `auto_path` or in `TCLLIBPATH`. `package require pdf4tcllib` does not load pdf4tcl along with it. Both lines therefore appear at the start of every example:

```
package require pdf4tcl
package require pdf4tcllib
```

To ship pdf4tcllib with an application, copy the `.tm` files in use into a module folder of the application's own and add it with `tcl::tm::path add`. Nothing more is needed; apart from pdf4tcl, pdf4tcllib has no dependencies. Only the namespace `image` needs Tk.

2.3 Scripts in UTF-8

The examples contain umlauts in the source code. Tcl 9 reads scripts as UTF-8. Tcl 8.6 reads them in the system encoding, and without a locale set (LANG empty, for instance in a cron job or container) that is ISO 8859-1. Then "gegenüber" turns into "gegenÃ¼ber" in the PDF, without an error message.

The remedy, one of three:

- set the locale (`LANG=de_DE.UTF-8` or `C.UTF-8`),
- load the script with `source -encoding utf-8 skript.tcl`,
- write characters outside ASCII as a `\u` sequence (`gegen\u00fcber`).

The encoding can be checked with `encoding system`.

2.4 fonts::init

```
pdf4tcllib::fonts::init ?-fontdir dir? ?-family name? \
    ?-cid 0|1? ?-force 0|1?
```

`fonts::init` searches for a TrueType family, registers its faces with pdf4tcl and remembers the names. It takes effect once per process; every further call returns immediately.

Option	Default	Effect
-family	DejaVuSansCondensed	name of the family as it appears in the file names
-fontdir	–	folder searched before all others
-cid	0	1: full font instead of 256 characters (section 256 Characters or CID)
-force	0	1: search again, even if already loaded

2.4.1 Which Files It Looks For

A family counts as found when **both** files are in one folder:

```
<familie>.ttf
<familie>-Bold.ttf
```

From the same folder `fonts::init` also takes, if present:

- `<familie>-Oblique.ttf` and `<familie>-BoldOblique.ttf` for italic,
- `<familie>Mono.ttf`, otherwise `DejaVuSansMono.ttf`, otherwise `LiberationMono-Regular.ttf` for the monospaced font.

The names follow the DejaVu scheme. Liberation uses `LiberationSans-Regular.ttf` and `LiberationSans-Italic.ttf`; that does not match, not even with `-family LiberationSans`.

2.4.2 Where It Looks

In this order; the first folder with both files wins:

1. the folder from `-fontdir`,
2. `fonts/` next to `lib/` and `lib/fonts/`, i.e. with the module itself,
3. on Linux `/usr/share/fonts/truetype/dejavu`, `/usr/share/fonts/dejavu-sans-fonts` (Fedora), `/usr/share/fonts/TTF` (Arch), `/usr/share/fonts/truetype`, `$HOME/.fonts`, `$HOME/.local/share/fonts`,
4. on Windows `%SystemRoot%\Fonts` and `%LOCALAPPDATA%\Microsoft\Windows\Fonts`,
5. on macOS `/Library/Fonts`, `/System/Library/Fonts`, `$HOME/Library/Fonts`.

It does not search subfolders.

2.4.3 Where the Font Comes From

System	What to do
Debian, Ubuntu	apt install fonts-dejavu-core fonts-dejavu-extra fonts-dejavu-mono
Fedora	dnf install dejavu-sans-fonts dejavu-sans-mono-fonts
Windows, macOS	DejaVu is not included: put the .ttf files in fonts/next to lib/or pass -fontdir

On Debian the faces are split across three packages. `fonts-dejavu-core` contains `DejaVuSans.ttf` and `DejaVuSans-Bold.ttf`, but not the narrow family `DejaVuSansCondensed` that `fonts::init` looks for without `-family`. That one is in `fonts-dejavu-extra`, together with all oblique faces. If only the core package is installed, `-family DejaVuSans` helps.

For an application meant to run on other people's machines, the folder `fonts/` next to `lib/` is the safe way: the font travels along, and the PDF looks the same everywhere.

2.5 Checking What Is Loaded

```
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init
set kursiv [pdf4tcllib::fonts::hasTtfItalic]
puts "TrueType:  [pdf4tcllib::fonts::hasTtf]  kursiv: $kursiv"
puts "CID-Modus: [pdf4tcllib::fonts::isCidMode]"
foreach p {fontSans fontSansBold fontSansItalic fontSansBoldItalic
           fontMono} {
    puts [format "%-20s %s" $p [pdf4tcllib::fonts::$p]]
}
```

Output:

```
pdf4tcllib::fonts: TTF italic faces loaded
pdf4tcllib::fonts: TTF faces loaded from
    /usr/share/fonts/truetype/dejavu
TrueType:  1  kursiv: 1
CID-Modus: 0
fontSans             Pdf4tclSans
fontSansBold         Pdf4tclSansBold
fontSansItalic       Pdf4tclSansItalic
fontSansBoldItalic   Pdf4tclSansBoldItalic
fontMono             Pdf4tclMono
```

If `fonts::init` finds nothing, it reports on stderr where it looked, and falls back to the standard fonts:

```
pdf4tcllib::fonts: No TTF fonts found, falling back to Helvetica
pdf4tcllib::fonts: searched in:
pdf4tcllib::fonts:   [---] .../pdf4tcllib/lib/../fonts
pdf4tcllib::fonts:   [---] .../pdf4tcllib/lib/fonts
pdf4tcllib::fonts:   [exists] /usr/share/fonts/truetype/dejavu
...
TrueType: 0  Sans: Helvetica
```

`[exists]` only means that the folder exists, not that the family is in it. The program keeps running. Code that relies on TrueType therefore checks `fonts::hasTtf` and aborts itself:

```
pdf4tcllib::fonts::init
if {![pdf4tcllib::fonts::hasTtf]} {
    error "keine TrueType-Schrift gefunden, siehe stderr"
}
```

2.6 Font Names Always Through the Accessor Procedures

pdf4tcl knows the loaded font under a name that `fonts::init` assigns: `Pdf4tclSans` with TrueType, `Helvetica` without. The code queries it instead of hard-coding it:

Procedure	with TrueType	without
fonts::fontSans	Pdf4tclSans	Helvetica
fonts::fontSansBold	Pdf4tclSansBold	Helvetica-Bold
fonts::fontSansItalic	Pdf4tclSansItalic	Helvetica-Oblique
fonts::fontSansBoldItalic	Pdf4tclSansBoldItalic	Helvetica-BoldOblique
fonts::fontMono	Pdf4tclMono	Courier

This way the same code runs with and without TrueType. The italic faces fall back to Helvetica-Oblique individually if the oblique files are missing (`fonts::hasTtfItalic` is then 0).

`fonts::setFont` takes a style as a word:

```
pdf4tcllib::fonts::setFont $pdf 14 Sans Bold       ;# fontSansBold
pdf4tcllib::fonts::setFont $pdf 14 Sans Italic     ;# fontSansItalic
pdf4tcllib::fonts::setFont $pdf 14 Sans BoldItalic ;# ...BoldItalic
```

After `fonts::init` the family does not matter here: `Bold`, `Italic` and `BoldItalic` always lead to the loaded faces. Without a style, `fonts::setFont` sets the family unchanged, so `fonts::setFont $pdf 14 Helvetica` really sets Helvetica. For normal text, therefore, use `$pdf setFont 14 [pdf4tcllib::fonts::fontSans]`.

With TrueType all faces are embedded:

```
$ pdffonts 020-schriften.pdf     (Spalten gekürzt)
name                             type       emb
AAAAAA+_Pdf4tcl_Base_Italic      TrueType   yes
AAAAAA+_Pdf4tcl_Base_BoldItalic  TrueType   yes
AAAAAA+_Pdf4tcl_Base_Mono        TrueType   yes
AAAAAA+_Pdf4tcl_Base_Regular     TrueType   yes
AAAAAA+_Pdf4tcl_Base_Bold        TrueType   yes
```

2.7 256 Characters or CID

`fonts::init` registers the font in one of two modes:

	without -cid	**-cid 1**
Characters	256: ASCII, Latin-1 and 32 more characters	all that the font has
including	umlauts, €, –, —, …, •, arrows ←↑→↓, box-drawing lines, ✓ ✗ ☐ ☑, ≤ ≥	in addition Greek, mathematics, Cyrillic, typographic quotation marks
PDF with one line of text	22 KB	49–52 KB, depending on the text
Characters outside	become "?"	become "?" if the font does not have them

The sizes come from the example in the next chapter; the difference is the font, which is embedded more completely in CID mode.

For German texts `-cid 1` is the better choice. The 256 characters cover umlauts and ß, but not the German quotation marks " and ": they are missing from the selection. What that means for `safeText` is shown in the chapter Safe Unicode Output.

2.7.1 Choosing the Mode on the First Call

The mode applies to the whole process, and it is fixed on the **first** `fonts::init`. A later `fonts::init -cid 1` no longer changes the font, not even with `-force 1`: the font is already registered with pdf4tcl under its name and is not registered again. `fonts::isCidMode` nevertheless reports 1 afterwards. In one test "αβγ" thus came out as "???" in the PDF. The pdf4tcllib counter stayed 0, nothing appeared on stderr; only `$pdf getSubstCount` counted the three characters.

That is why `fonts::init` with all its options belongs in one place at the start of the program, before some library calls it with other options.

2.8 Summary

- `package require pdf4tcl` and `package require pdf4tcllib`, then `pdf4tcllib::fonts::init` once, with `-cid 1` for German texts.
- The font must be present as `<familie>.ttf` and `<familie>-Bold.ttf` in one of the search folders; on Debian install `fonts-dejavu-extra` or pass `-family DejaVuSans`.

- Check `fonts::hasTtf` when TrueType is mandatory.
- Font names only through `fonts::fontSans` and the other accessor procedures.
- Tcl 8.6 needs a UTF-8 locale for umlauts in the script.

Next chapter: Safe Unicode Output

3 Safe Unicode Output

3.1 Introduction

A character that the font does not have disappears in the PDF: it shows up as "?", as an empty box, or not at all. That it is missing is only noticed when reading. pdf4tcllib catches this in one place, in unicode::sanitize, and counts what it had to replace.

This chapter shows what `sanitize` does with a line, which procedures call it and which do not, and how to tell that characters have been lost.

3.2 What sanitize Does

```
pdf4tcllib::unicode::sanitize text ?-mono 0|1? ?-tabwidth n?
```

`sanitize` returns the text as the loaded font can show it. It works in two stages:

1. **Replace known characters.** A fixed table maps characters that often occur in texts to something printable: ✓ → ✓, (!) → (!), → → →, (TM) → (TM). Without TrueType it replaces more: → becomes `->`, € becomes `EUR`, ✓ becomes `(OK)`, box-drawing lines become `+`, `-` and `|`.
2. **Check every remaining character.** Whatever the font does not have in the current mode becomes "?". Invisible characters (zero-width spaces, variation selectors, BOM) are dropped entirely, tabs become spaces (`-tabwidth`, default 4).

Which characters the font "has" depends on the mode from `fonts::init` (chapter Setup and Fonts):

- without TrueType: printable ASCII and Latin-1,
- TrueType with 256 characters: the fixed selection from `fonts::subset`,
- TrueType in CID mode: every character for which the font has a glyph.

The same line in both TrueType modes:

```
# 030-unicode.tcl -- was sanitize aus einer Zeile macht
#   tclsh 030-unicode.tcl          256-Zeichen-Modus
#   tclsh 030-unicode.tcl -cid 1   CID-Modus (volle Schrift)
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init {*}$argv

set zeile "Pfeil \u2192, Euro \u20ac, Haken \u2705, alpha \u03b1,\
    Summe \u2211, \u201eZitat\u201c, Schneemann \u2603, Kanji \u6f22"
puts [pdf4tcllib::unicode::sanitize $zeile]
puts [pdf4tcllib::unicode::sanitize $zeile -mono 1]
puts [pdf4tcllib::unicode::substReport]
```

```
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
$pdf setFont 11 [pdf4tcllib::fonts::fontSans]
pdf4tcllib::unicode::safeText $pdf $zeile -x 50 -y 60
$pdf endPage
set cid [expr {[pdf4tcllib::fonts::isCidMode] ? "-cid" : ""}]
$pdf write -file 030-unicode$cid.pdf
$pdf destroy
```

```
$ tclsh 030-unicode.tcl
Pfeil →, Euro €, Haken ✓, alpha ?, Summe ?, "Zitat", Schneemann ?,
    Kanji ?
Pfeil ->, Euro EUR, Haken (OK), alpha ?, Summe ?, "Zitat", Schneemann
    ?, Kanji ?
count 8 chars {U+03B1 U+2211 U+2603 U+6F22}

$ tclsh 030-unicode.tcl -cid 1
Pfeil →, Euro €, Haken ✓, alpha α, Summe ∑, "Zitat", Schneemann ☃,
    Kanji ?
Pfeil →, Euro €, Haken ✓, alpha α, Summe ∑, "Zitat", Schneemann ☃,
    Kanji ?
count 2 chars U+6F22
```

The first line is normal text, the second `-mono 1`. In CID mode the Greek letter and the sum sign survive as well; only the kanji is missing, because DejaVu has no Chinese characters. The counter runs across both calls, which is why the 256-character mode shows 8: four characters per line.

3.2.1 Monospaced Font: -mono 1

`-mono 1` is meant for code and tables in a monospaced font. If `fonts::init` loads no TrueType monospaced font, that font is Courier, and Courier knows only Latin-1; then the large substitution table applies, as without TrueType (→ becomes `->`). In CID mode with DejaVu Sans Mono, Unicode is kept and checked against that font. In 256-character mode `-mono 1` always substitutes according to the large table, even when DejaVu Sans Mono is loaded.

3.3 Who Calls sanitize

Procedure	calls sanitize
unicode::safeText	yes
page::header, page::footer, page::number, page::centerText	yes, through safeText
text::writeParagraph	no, it calls $pdf drawTextBox directly
$pdf text, $pdf drawTextBox (pdf4tcl)	no

Whatever does not pass through `sanitize` is replaced by pdf4tcl itself when the font cannot

encode a character. In that case, too, "?" appears in the PDF. It is counted elsewhere: in `$pdf getSubstCount`.

```
# 030-drei-wege.tcl -- dieselbe Zeile auf drei Wegen
#   tclsh 030-drei-wege.tcl          256-Zeichen-Modus
#   tclsh 030-drei-wege.tcl -cid 1   CID-Modus
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init {*}$argv
set t "\u201eZitat\u201c, alpha \u03b1, Pfeil \u2192"

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
$pdf setFont 11 [pdf4tcllib::fonts::fontSans]
$pdf text "pdf4tcl direkt:  $t" -x 50 -y 60
pdf4tcllib::unicode::safeText $pdf "safeText:        $t" -x 50 -y 80
pdf4tcllib::text::writeParagraph $pdf "writeParagraph:  $t" \
    50 100 400 11
set bericht [pdf4tcllib::unicode::substReport]
puts "pdf4tcllib ersetzt: [dict get $bericht count]"
puts "pdf4tcl ersetzt:    [$pdf getSubstCount]"
set cid [expr {[pdf4tcllib::fonts::isCidMode] ? "-cid" : ""}]
$pdf write -file 030-drei-wege$cid.pdf
$pdf destroy
```

256-character mode:

```
pdf4tcllib ersetzt: 1
pdf4tcl ersetzt:    6

pdf4tcl direkt:  ?Zitat?, alpha ?, Pfeil →
safeText:        "Zitat", alpha ?, Pfeil →
writeParagraph:  ?Zitat?, alpha ?, Pfeil →
```

CID mode:

```
pdf4tcllib ersetzt: 0
pdf4tcl ersetzt:    0

pdf4tcl direkt:  "Zitat", alpha α, Pfeil →
safeText:        "Zitat", alpha α, Pfeil →
writeParagraph:  "Zitat", alpha α, Pfeil →
```

Two things stand out:

- In 256-character mode the characters are missing on all three paths. Only the counter differs: `safeText` counts with pdf4tcllib, the other two paths with pdf4tcl.
- `safeText` always turns "Zitat" into "Zitat", even in CID mode, where the font has " and ". The first stage of `sanitize` replaces the typographic quotation marks (" " " ' ' ') with straight ones in every mode.

3.3.1 German Quotation Marks

For German texts this means: CID mode, and text with "…" is not output through `safeText` but through `text::writeParagraph` or `$pdf text`. Both leave the characters alone. In 256-character mode there is no way to "…"; there only straight quotation marks or » « (which are in Latin-1) remain.

The headers and footers from `page::header` and `page::footer` go through `safeText`; a "Titel" (title) in them loses its quotation marks even in CID mode.

3.4 The Font Used for Drawing Is Not the One sanitize Asks About

`sanitize` goes by what `fonts::init` has loaded, not by the font that is currently set with `setFont`. Setting Helvetica for a paragraph after `fonts::init` lets characters through that Helvetica does not know:

```
# 030-falle-schrift.tcl -- sanitize folgt fonts::init, nicht setFont
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init
set t "Pfeil \u2192, Euro \u20ac, Haken \u2713"

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
$pdf setFont 12 [pdf4tcllib::fonts::fontSans]
pdf4tcllib::unicode::safeText $pdf "fontSans:  $t" -x 50 -y 60
$pdf setFont 12 Helvetica
pdf4tcllib::unicode::safeText $pdf "Helvetica: $t" -x 50 -y 80
set bericht [pdf4tcllib::unicode::substReport]
puts "pdf4tcllib ersetzt: [dict get $bericht count]"
puts "pdf4tcl ersetzt:    [$pdf getSubstCount]"
$pdf write -file 030-falle-schrift.pdf
$pdf destroy
```

```
pdf4tcllib ersetzt: 0
pdf4tcl ersetzt:    0

fontSans:  Pfeil →, Euro €, Haken ✓
Helvetica: Pfeil ?, Euro €, Haken ?
```

Under Tcl 8.6 both counters stay at 0, and still two characters are missing in the second line. Under Tcl 9 at least pdf4tcl reports them (`pdf4tcl ersetzt: 2`). After `fonts::init`, therefore, set only the fonts from `fonts::fontSans` and the other accessor procedures. Where Courier or Helvetica is really needed, output the text with `-mono 1`; then the large substitution table applies, and → becomes `->` instead of "?".

3.5 Emoji

Characters above U+FFFF, that is most emoji, are replaced by `sanitize` with an ASCII transcription: :-) becomes `:-)`, (+1) becomes `(+1)`, [L] becomes `[L]`. Unknown emoji become `(?)`. This transcription does not count as a loss and does not appear in `substReport`.

```
$ tclsh 030-emoji.tcl
Tcl 8.6, direkt gelesen: Gruss :-) und (+1)
Tcl 8.6, mit readFile:   Gruss :-) und (+1)
```

Tcl 8.6.14 and Tcl 9.0 read the emoji from a UTF-8 file correctly. unicode::readFile reads the file in binary mode and inserts the transcription before Tcl converts the bytes; it is meant for older 8.6 releases that turn such characters into U+FFFD when reading. A U+FFFD in the text becomes `(?)`, and the original character can then no longer be recognized.

3.6 Knowing What Was Missing

`sanitize` counts every character it replaces with "?" and remembers up to twelve different ones:

```
pdf4tcllib::unicode::substCount   ;# 8
pdf4tcllib::unicode::substReport  ;# count 8 chars {U+03B1 U+2211 ...}
pdf4tcllib::unicode::resetSubst
```

The first time, it also writes a warning to stderr:

```
pdf4tcllib::unicode: sanitize: at least one character is missing from
    the
font and was replaced with '?' (first: U+03B1). Ask
::pdf4tcllib::unicode::substReport for the list, or load a font that
    has
it -- fonts::init -cid 1 for Greek and mathematical symbols.
```

For a program that produces PDFs unattended, both counters belong at the end of every document:

```
set verloren [expr {[pdf4tcllib::unicode::substCount]
                    + [$pdf getSubstCount]}]
if {$verloren} {
    puts stderr "Warnung: $verloren Zeichen ersetzt\
        ([dict get [pdf4tcllib::unicode::substReport] chars])"
}
```

`substCount` counts across all documents of the process; call `resetSubst` before each new document. `$pdf getSubstCount` applies to the one pdf4tcl object. The case from the previous section (Helvetica after `fonts::init`) is reported by neither of them under Tcl 8.6.

3.7 Summary

- Output text with special characters through `unicode::safeText`, paragraphs through `text::writeParagraph`, both with fonts from the accessor procedures.

- For Greek, mathematics and German quotation marks use `fonts::init -cid 1`.
- `safeText` always replaces " " with "; for "..." use `writeParagraph` or `$pdf text`.
- At the end check `unicode::substCount` and `$pdf getSubstCount`.

Next chapter: Units and Page Context

4 Units and Page Context

4.1 Introduction

pdf4tcl calculates in points (1/72 inch), while forms and letter templates are dimensioned in millimeters. And every drawing operation needs the same numbers: where the type area begins on the left, how wide it is, where it ends at the bottom. page::context computes these numbers once and returns them as a dict. All building blocks of pdf4tcllib that place something on the page take this dict.

4.2 Converting Units

```
pdf4tcllib::units::mm   10      ;# 28.346...  Millimeter -> Punkt
pdf4tcllib::units::cm   2.5     ;# 70.866...
pdf4tcllib::units::inch 1       ;# 72.0
pdf4tcllib::units::to_mm 72     ;# 25.4       Punkt -> Millimeter
pdf4tcllib::units::to_cm 72     ;# 2.54
pdf4tcllib::units::to_inch 144  ;# 2.0
```

That is all the namespace units does. The results are floating-point numbers; rounding happens only when drawing.

4.2.1 Keep pdf4tcl in Points

pdf4tcl can calculate in millimeters itself (`pdf4tcl::new ... -unit mm`). Not with pdf4tcllib: `page::context` and all building blocks return and expect points. An object with `-unit mm` takes the point values as millimeters, and a footer at y = 813 ends up 813 mm down, far below the page. The PDF is then empty, without an error message.

The same goes for pdf4tcl's `-margin` option: it shifts the origin by the margin. `page::context` already includes the margin; both together give twice the margin. A test with 20 mm in both placed the text at x = 113 pt instead of 57 pt.

Therefore, for pdf4tcllib:

```
# ohne -unit, ohne -margin
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
```

4.3 page::context

```
pdf4tcllib::page::context paper ?-margin mm? ?-landscape 0|1? \
    ?-orient 0|1?
```

Argument	Default	Meaning
paper	–	a3, a4, a5, b5, letter, legal (upper or lower case)
-margin	20	margin in millimeters, the same on all four sides
-landscape	0	1: landscape, width and height swapped
-orient	1	1: y grows downward, 0: upward

Any other paper is rejected:

```
unknown paper size: din-lang (known: letter a3 legal a4 a5 b5)
```

The short form `context a4 20 0` is equivalent to `context a4 -margin 20 -orient 0`; the second word is therefore the direction, not landscape. The form with options is easier to read.

4.3.1 What the Dict Contains

```
# 040-seitenkontext.tcl -- Satzspiegel aus page::context, hoch und quer
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init

proc zeige {ctx} {
    foreach k {paper page_w page_h margin left right top bottom
               text_w text_h} {
        set v [dict get $ctx $k]
        if {[string is double -strict $v]} { set v [format %.2f $v] }
        puts [format "  %-7s %s" $k $v]
    }
}

puts "A4 hoch, 20 mm Rand, y nach unten (orient true):"
set ctx [pdf4tcllib::page::context a4 -margin 20]
zeige $ctx

puts "A4 quer, 15 mm Rand, y nach oben (orient false):"
set quer [pdf4tcllib::page::context a4 -margin 15 -landscape 1 \
    -orient false]
zeige $quer

# Den Satzspiegel einzeichnen: einmal je Richtung, jeweils mit
# passendem pdf4tcl-Objekt
foreach {name c} [list hoch $ctx quer $quer] {
    set pdf [pdf4tcl::new %AUTO% -paper a4 \
        -landscape [dict get $c landscape] \
        -orient [dict get $c orient]]
    $pdf startPage
```

```
    set l [dict get $c left]; set r [dict get $c right]
    set t [dict get $c top];  set b [dict get $c bottom]
    $pdf setStrokeColor 0.2 0.4 0.8
    $pdf rectangle $l [expr {min($t, $b)}] \
        [dict get $c text_w] [dict get $c text_h]
    $pdf setFont 10 [pdf4tcllib::fonts::fontSans]
    set y $t
    set mm [pdf4tcllib::units::to_mm [dict get $c text_w]]
    set zeilen [list "Satzspiegel $name" "[format %.0f $mm] mm breit"]
    foreach zeile $zeilen {
        pdf4tcllib::page::_advance $c y 14
        pdf4tcllib::unicode::safeText $pdf $zeile \
            -x [expr {$l + 6}] -y $y
    }
    pdf4tcllib::page::number $pdf $c 1
    $pdf endPage
    $pdf write -file 040-seitenkontext-$name.pdf
    $pdf destroy
}
```

```
A4 hoch, 20 mm Rand, y nach unten (orient true):
  paper   a4
  page_w  595.28
  page_h  841.89
  margin  56.69
  left    56.69
  right   538.59
  top     56.69
  bottom  785.20
  text_w  481.89
  text_h  728.50
A4 quer, 15 mm Rand, y nach oben (orient false):
  paper   a4
  page_w  841.89
  page_h  595.28
  margin  42.52
  left    42.52
  right   799.37
  top     552.76
  bottom  42.52
  text_w  756.85
  text_h  510.24
```

Key	Meaning	also as
page_w, page_h	page in points	PW, PH
margin	margin in points	margin_pt

Key	Meaning	also as
margin_mm	margin, as given	–
left, right	left and right edge of the type area	SX for left
top, bottom	upper and lower edge, in the chosen direction	SY for top
text_w, text_h	width and height of the type area	SW, SH
paper, landscape, orient	the inputs	–

The short names exist for older code. Other spellings (`pageW`, `textWidth`) do not exist; `dict get` then reports an error.

4.3.2 top and bottom Depend on the Direction

`top` is always the upper edge of the type area, `bottom` the lower one, but in the page's way of counting:

	-orient 1 (y downward)	-orient 0 (y upward)
top	small (margin)	large (height − margin)
bottom	large (height − margin)	small (margin)
next line	y larger	y smaller

The direction in the context and the direction of the pdf4tcl object must match. The simplest way is to take it from the context:

```
set pdf [pdf4tcl::new %AUTO% -paper a4 \
    -landscape [dict get $ctx landscape] \
    -orient [dict get $ctx orient]]
```

The building blocks in this book assume `-orient 1`, the default of both packages. Tables from `table::draw` reject `-orient 0` with an error (chapter Tables with Page Breaks).

4.3.3 Line by Line

page::_advance moves a y variable in the right direction, without the code having to know the direction:

```
set y [dict get $ctx top]
foreach zeile $zeilen {
    pdf4tcllib::page::_advance $ctx y 14   ;# erst weiter, dann setzen
    pdf4tcllib::unicode::safeText $pdf $zeile \
        -x [dict get $ctx left] -y $y
}
```

The underscore in the name is misleading: the procedure is documented and is used by the form building blocks themselves.

The line spacing for a font size comes from page::lineheight: font size × 1.4, rounded up (`page::lineheight 10` gives 14). A different factor goes in as the second argument.

4.3.4 The First Line

For `$pdf text`, y is the baseline. Text placed at `y = top` sticks out of the type area into the margin with its ascenders. The first line therefore starts one font size lower, at `top` plus the font size (in the `-orient 1` direction). That is exactly what the example above does with `_advance` before writing.

4.4 Header, Footer and Page Number

Three procedures write into the margin, above and below the type area:

```
# Mitte oben, Linie darunter
pdf4tcllib::page::header $pdf $ctx "Monatsbericht"
# links der Text, rechts "Seite 3"
pdf4tcllib::page::footer $pdf $ctx "Vertraulich" 3
# Mitte unten "- 3 / 10 -"
pdf4tcllib::page::number $pdf $ctx 3 10
```

`page::footer` writes the page number as "Seite 3". That is a default of the library, not a translation; for an English document:

```
# für alle Aufrufe
set ::pdf4tcllib::page::pageLabelFormat "Page %s"
# nur für diesen Aufruf
pdf4tcllib::page::footer $pdf $ctx "Confidential" 3 9 \
    -pagelabel "Page %s"
```

The font is the loaded TrueType font, Helvetica without TrueType; the size is the last numeric argument (default 10 for the header, 9 for footer and page number). All three go through `safeText` (chapter Safe Unicode Output).

A fourth procedure, `page::centerText $pdf $ctx text y ?size? ?font?`, centers a line, for example a title. Without a font argument it uses **Helvetica**, not the loaded TrueType font; Helvetica is not embedded, and the PDF is then no longer suitable for PDF/A. Therefore always pass the font:

```
pdf4tcllib::page::centerText $pdf $ctx "Jahresbericht" 120 20 \
    [pdf4tcllib::fonts::fontSansBold]
```

How many pages a document has is only known at the end. "Seite 3 von 10" (page 3 of 10) therefore needs a second pass; this is shown in the chapter Headers, Footers, Page Numbers.

4.5 Guide Lines During Development

Procedure	draws
page::grid $pdf $ctx ?abstand?	a gray grid over the whole page, with coordinates at the edge (default 50 pt)
page::debugGrid $pdf $ctx ?abstand?	the same, but only when the environment variable PDF4TCL_DEBUG is set

Procedure	draws
page::orientationLegend $pdf $ctx	one line in the right corner: paper, direction, position of the origin, unit

`debugGrid` can stay in the code: without `PDF4TCL_DEBUG=1` it draws nothing.

```
PDF4TCL_DEBUG=1 tclsh bericht.tcl
```

`page::grid` does not divide the page into columns. A grid for columns is built from `left` and `text_w`; text across several columns is set by the package `pdf4tclflow` (chapter Column Flow and Watermarks).

4.6 Summary

- pdf4tcllib calculates in points; margins are given in millimeters.
- Create pdf4tcl objects without `-unit` and without `-margin`.
- `page::context` returns `left`, `top`, `text_w` and the rest; the direction in the context and in the pdf4tcl object must be the same.
- `page::_advance` and `page::lineheight` for the next line; the first line one font size below `top`.
- `page::header`, `page::footer`, `page::number` for the margin; adjust "Seite" through `pageLabelFormat`.

Next chapter: Setting Text

5 Setting Text

5.1 Introduction

pdf4tcl puts a line where it is told to. If the line is too long, it runs over the margin. For everything beyond that, the namespace text has four tools: measure width, wrap, truncate, shrink. In addition there is `writeParagraph` for a whole paragraph in one go.

This chapter shows the four tools and builds from them a small procedure that sets text across any number of pages.

5.2 Measuring Width

```
pdf4tcllib::text::width text size font ?pdf?
```

With the pdf4tcl object as the last argument, `width` asks pdf4tcl for the real metrics of the font. Without the object it estimates by character class: narrow letters such as `i` and `l` count less, `m` and `W` more.

```
# 050-messen-und-umbrechen.tcl -- messen, umbrechen, kuerzen
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
```

```
set font [pdf4tcllib::fonts::fontSans]
set pdf  [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Breite: geschaetzt (ohne pdf) und gemessen (mit pdf)
set t "Lieferung Mo 17.08. Spedition Nord"
puts [format "geschaetzt %.1f pt, gemessen %.1f pt" \
    [pdf4tcllib::text::width $t 10 $font] \
    [pdf4tcllib::text::width $t 10 $font $pdf]]

# Umbrechen: am Leerzeichen und nach einem Bindestrich
set lang "Die Muster-Beispiel-Stadt liegt an der Bundesstrasse."
foreach z [pdf4tcllib::text::wrap $lang 90 10 $font 0 $pdf] {
    puts "  |$z|"
}

# Ein Wort, das allein zu breit ist: gekuerzt oder zerlegt
set name "pdf4tcllib::unicode::substReport"
set ohne [pdf4tcllib::text::wrap $name 80 10 $font 0 $pdf]
set mit  [pdf4tcllib::text::wrap $name 80 10 $font 0 $pdf 1]
puts "ohne hardBreak: $ohne"
puts "mit hardBreak:  $mit"

# Kuerzen mit "..." oder kleiner setzen
set k "Tour Sped1 Nord"
puts "truncate: [pdf4tcllib::text::truncate $k 60 10 $font $pdf]"
puts "fitSize:  [pdf4tcllib::text::fitSize $k 60 10 $font -pdf $pdf]"
puts "fits:     [pdf4tcllib::text::fits $k 60 7 $font $pdf]"
$pdf destroy
```

```
geschaetzt 156.0 pt, gemessen 159.7 pt
  |Die Muster-Beispiel-|
  |Stadt liegt an der|
  |Bundesstrasse.|
ohne hardBreak: pdf4tcllib::unic...
mit hardBreak:  pdf4tcllib::unicod e::substReport
truncate: Tour Sped1...
fitSize:  8
fits:     1
```

For this sentence the estimate is off by 2 %, for other texts with DejaVu by up to about 6 %. That is enough to wrap a line that would have fit, or to let one through that sticks out. That is why all calls in this book append the object `$pdf`. The estimate exists only for the case that no document exists yet.

`width` needs no `setFont` beforehand: font and size are part of the call.

5.3 Wrapping

```
pdf4tcllib::text::wrap line maxW size font \
    ?code? ?pdf? ?hardBreak? ?trenner?
```

`wrap` returns a list of lines, each at most `maxW` wide. Nothing is drawn. The order of the arguments is unusual: `pdf` comes sixth, after `code`.

Argument	Default	Meaning
code	0	1: every wrapped line except the last ends in `\`, like a continued Tcl line
pdf	–	object for exact widths
hardBreak	0	what happens to a word that is too wide on its own (see below)
trenner	{- /}	characters at which a break is also allowed inside a word

Lines are broken at spaces and after a hyphen or slash between two letters or digits. The separator stays at the end of the line ("Muster-Beispiel-"), so the reader sees that the word continues. A dash at the start of a word does not break: "-5 Euro" stays together. pdf4tcl breaks lines in `drawTextBox` by the same rules.

5.3.1 A Word That Does Not Fit

What happens to a word that on its own is wider than the line is decided by `hardBreak`:

- `0` (default): the word is **truncated** and ends in "...". The rest is gone.
- `1`: the word is split character by character; nothing is lost.

The default is meant for running text, in which such a word is rare. In table cells with paths, package names or item numbers it is wrong: `pdf4tcllib::unicode::substReport` becomes `pdf4tcllib::unic...`. Set `hardBreak 1` there.

5.4 Truncating or Shrinking

```
pdf4tcllib::text::truncate text maxW size font ?pdf?
pdf4tcllib::text::fitSize  text maxW size font ?-min n? ?-pdf obj?
pdf4tcllib::text::fits     text maxW size font ?pdf?
```

text::truncate cuts off and appends three dots. text::fitSize instead looks for the largest font size at which the text fits completely, in steps of 1 point downward to `-min` (default 6). If it does not fit even there, `-min` is returned and the caller decides.

Which is better depends on the content. "Tour Sped1 Nord" truncated is "Tour Sped1...": whether it was "Nord" or "Süd" (north or south) is no longer there. For names, routes, amounts and identifiers a smaller font is better than a cut-off end. A remark that is only a note anyway may be truncated.

With `fitSize` the object is called `-pdf`; with the others it is the last argument, without a name.

5.5 A Paragraph in One Go

```
set y [pdf4tcllib::text::writeParagraph pdf text x y width \
    ?size? ?align? ?tag?]
```

text::writeParagraph wraps and draws in one step, through pdf4tcl's `drawTextBox`. It returns the y position below the last line. `align` is `left`, `right`, `center` or `justify`.

Three things are worth knowing:

- The font is always the regular one from `fonts::init` (Helvetica without TrueType). Bold or italic is not possible with it.
- There is no page break. A paragraph that no longer fits at the bottom runs past the margin.
- The text does not go through `sanitize` (chapter Safe Unicode Output).

For a short paragraph at a known position `writeParagraph` is the right choice. Text of unknown length needs wrapping across pages.

5.6 Text Across Pages

The following procedure `absatz` (paragraph) wraps a paragraph with `wrap` and sets it line by line. If the next line no longer fits above `bottom`, `neueSeite` (new page) starts a new page with a footer. Font and size can be chosen freely, bold included.

```
# 050-text-ueber-seiten.tcl -- Fliesstext mit eigenem Seitenumbruch
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Der Zustand der Seite: wo die naechste Zeile steht
set y 0

proc neueSeite {} {
    global pdf ctx y
    if {[$pdf currentPage] > 0} {
        set n [$pdf currentPage]
        pdf4tcllib::page::footer $pdf $ctx "Lagerbericht" $n
        $pdf endPage
    }
    $pdf startPage
    set y [dict get $ctx top]
}

# Einen Absatz setzen: umbrechen, Zeile fuer Zeile, bei Bedarf eine
# neue Seite
proc absatz {text {size 11} {font ""}} {
    global pdf ctx y
```

```
    if {$font eq ""} { set font [pdf4tcllib::fonts::fontSans] }
    set lh [pdf4tcllib::page::lineheight $size]
    set breite [dict get $ctx text_w]
    foreach zeile [pdf4tcllib::text::wrap $text $breite \
            $size $font 0 $pdf 1] {
        if {$y + $lh > [dict get $ctx bottom]} { neueSeite }
        set y [expr {$y + $lh}]
        $pdf setFont $size $font
        pdf4tcllib::unicode::safeText $pdf $zeile \
            -x [dict get $ctx left] -y $y
    }
    set y [expr {$y + $lh / 2}]
}

neueSeite
absatz "Bestand am Monatsende" 16 [pdf4tcllib::fonts::fontSansBold]
for {set i 1} {$i <= 40} {incr i} {
    absatz "Regal $i: Der Bestand wurde gezählt und mit der Buchung\
        abgeglichen. Abweichungen unter zwei Stück sind ausgebucht,\
        größere gehen an die Lagerleitung zur Klärung."
}
pdf4tcllib::page::footer $pdf $ctx "Lagerbericht" [$pdf currentPage]
$pdf endPage
puts "Seiten: [$pdf pageCount]"
$pdf write -file 050-text-ueber-seiten.pdf
$pdf destroy
```

```
Seiten: 3
```

What the procedure does right:

- It checks **before** each line whether the line still fits, not after. The last line of a page then still lies inside the type area.
- It advances y **before** writing, one line height below `top`. That way the first line of a page does not stick into the margin (chapter Units and Page Context).
- It sets the font again before every line. `neueSeite` sets a different font in the footer, and without the new `setFont` the paragraph would continue on the new page in the footer's font.
- It uses `hardBreak 1`, so that no word is truncated.

The page number comes from `$pdf currentPage`; after the last `endPage`, `$pdf pageCount` tells how many pages there are.

What the procedure does not do: it does not prevent a single line at the top of a page or a heading at the bottom. For that, the height of a block must be known before setting it; this is shown in the chapter on `measure` and `layout`.

5.7 Summary

- Always call `text::width`, `text::wrap`, `text::truncate`, `text::fits` with the pdf4tcl object; otherwise they estimate.

- `wrap` truncates a word that is too wide; use `hardBreak 1` where nothing may be lost.
- `fitSize` instead of `truncate` for names, identifiers and amounts.
- `writeParagraph` for a short paragraph in the regular font; for longer text, a loop of your own with page breaks.

Next chapter: Headers, Footers, Page Numbers

6 Headers, Footers, Page Numbers

6.1 Introduction

Headers and footers appear on every page, and they are written at the moment a page begins or ends. pdf4tcllib provides the three building blocks for this (chapter Units and Page Context); when they are called is decided by your own code. This chapter fixes that place and answers the question every report eventually asks: "page 2 of how many?"

6.2 One Place for the Page Change

Everything that belongs on every page goes into one procedure that finishes the old page and begins the new one: in the previous chapter, `neueSeite`. Code that writes header and footer in several places forgets them in one.

What	When	Procedure
footer, page number	before the page ends	page::footer, page::number
header, watermark	after the new page begins	page::header, drawing::watermark
y back to the start	after the header	dict get $ctx top

The footer belongs at the end of a page, not at its beginning: only then is it certain that the page has content and that its number is correct.

The three building blocks write into the margin, not into the type area:

Building block	Position (y downward)
page::header	text at half the margin from the top, line 2 pt above top
page::footer	line 2 pt below bottom, text at half the margin from the bottom
page::number	like the footer text, in the middle

With a 20 mm margin the text of header and footer is 10 mm from the paper edge. With a smaller margin it moves closer; with a 10 mm margin it is 5 mm, and that can be too tight for a printer.

6.3 Page 2 of 3

How many pages a document has is only settled when it is finished. By then, however, the footer of the first page has long been written. The usual solution: set the document twice. The first pass goes into a document that is only counted and then discarded; the second knows the

number.

```
# 060-seite-x-von-y.tcl -- "Seite 2 von 3": zwei Durchlaeufe
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]

# Der ganze Bericht als Prozedur: sie bekommt das Dokument und die
# Gesamtzahl der Seiten ("" im ersten Durchlauf) und hat keine
# Nebenwirkungen -- sie laeuft zweimal.
proc bericht {pdf gesamt} {
    global ctx
    set font [pdf4tcllib::fonts::fontSans]
    set lh   [pdf4tcllib::page::lineheight 11]
    set seite 0
    set y 0
    set neu {
        if {$seite > 0} { rand $pdf $seite $gesamt; $pdf endPage }
        $pdf startPage
        incr seite
        if {$seite > 1} {
            pdf4tcllib::page::header $pdf $ctx "Lagerbericht Oktober"
        }
        set y [dict get $ctx top]
    }
    eval $neu
    for {set i 1} {$i <= 120} {incr i} {
        if {$y + $lh > [dict get $ctx bottom]} { eval $neu }
        set y [expr {$y + $lh}]
        $pdf setFont 11 $font
        pdf4tcllib::unicode::safeText $pdf "Position $i: geprüft" \
            -x [dict get $ctx left] -y $y
    }
    rand $pdf $seite $gesamt
    $pdf endPage
}

proc rand {pdf seite gesamt} {
    global ctx
    set label "Seite %s"
    if {$gesamt ne ""} { append label " von $gesamt" }
    pdf4tcllib::page::footer $pdf $ctx "Vertraulich" $seite 9 \
        -pagelabel $label
}
```

```
# Durchlauf 1: in ein Wegwerf-Dokument, nur um die Seiten zu zaehlen
set probe [pdf4tcl::new %AUTO% -paper a4 -orient true]
bericht $probe ""
set gesamt [$probe pageCount]
$probe destroy

# Durchlauf 2: das echte Dokument
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
bericht $pdf $gesamt
$pdf write -file 060-seite-x-von-y.pdf
$pdf destroy
puts "Seiten: $gesamt"
```

```
Seiten: 3
```

The footers read "Seite 1 von 3", "Seite 2 von 3", "Seite 3 von 3". The first page has no header, the following ones have "Lagerbericht Oktober".

What this requires:

- **The content is in a procedure** that receives the document as an argument. It runs twice and therefore must not do anything that turns out differently the second time: no appending to a file, no incrementing a counter outside, no reading from a channel that is empty afterwards. Read data that comes from a database once beforehand and pass it to the procedure.
- **The page number must not change the line breaks.** "Seite 9 von 9" and "Seite 9 von 10" differ in width, but both are in the footer, outside the type area. The content breaks the same way in both passes. Using the total in the text ("Dieser Bericht hat 3 Seiten", this report has 3 pages) can change the number of lines; then check the second pass against the first.
- **-pagelabel takes a format.** `page::footer` inserts the page number with `format`. "Seite %s von 3" gives "Seite 2 von 3". A percent sign in the text must therefore be written `%%`.

The throwaway document costs only computing time; it is never written. `$probe pageCount` counts the finished pages, which is why the first pass also ends with `endPage`.

The package `pdf4tcltoc` works on the same principle to put the table of contents with real page numbers in front of the text (chapter Table of Contents).

6.4 Page Number Without a Word

`page::number` writes `- 2 -` or `- 2 / 3 -` in the middle, without any language. That fits when there is no footer with text:

```
pdf4tcllib::page::number $pdf $ctx [$pdf currentPage] $gesamt
```

With `""` as the total, only the number appears. That is handy in the first pass, when the total is not known yet.

6.5 Summary

- Header, footer, watermark and the start of the page belong in one procedure for the page change.

- Footer before `endPage`, header after `startPage`.
- "Page X of Y": set the document twice, take the total from `$probe pageCount`; the content as a procedure without side effects.
- `-pagelabel "Seite %s von $gesamt"`; a `%` in the text as `%%`.

Next chapter: Column Flow and Watermarks

7 Column Flow and Watermarks

7.1 Introduction

A circular in two columns, minutes with "ENTWURF" (draft) across every page: both can be built with the loop from the chapter Setting Text, but the package pdf4tclflow takes the column arithmetic off your hands. It fills column after column and page after page until the text is used up. The watermark is part of the pdf4tcllib core.

7.2 Text in Columns

```
package require pdf4tclflow
set r [pdf4tcllib::flow::columns pdf ctx text ?option wert ...?]
```

The text is split into paragraphs at blank lines; line breaks and multiple spaces within a paragraph become a single space. Between two paragraphs one blank line remains, except at the top of a column.

Option	Default	Meaning
-columns	2	number of columns
-gap	18	space between the columns in points
-size	10	font size
-font	from fonts::init	font
-top, -bottom	from the context	where each column begins and ends
-firsty	-top	start of the first column, for example below a heading
-newpage	–	script for the page change
-tag	P	structure type for accessible PDFs, "" for none

The columns are of equal width and fill the type area; with two columns on A4 with a 20 mm margin that is 232 points (82 mm) each. flow::boxes returns the position of each column as `{x breite}` (x, width), for an image in column two or a line between the columns.

```
# 070-spalten.tcl -- zwei Spalten ueber mehrere Seiten, Wasserzeichen
package require pdf4tcl
package require pdf4tcllib
package require pdf4tclflow

```

```
pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Ein langer Text; Absaetze durch eine Leerzeile getrennt
set absatz "Die Inventur beginnt am ersten Werktag des Monats. Jedes\
    Regal wird gezählt, die Zahl mit der Buchung verglichen und die\
    Abweichung notiert. Kleine Abweichungen werden sofort ausgebucht,\
    größere gehen an die Lagerleitung."
set text [join [lrepeat 30 $absatz] "\n\n"]

# erste Grundlinie, im Satzspiegel
set oben [expr {[dict get $ctx top] + 10}]

$pdf startPage
pdf4tcllib::drawing::watermark $pdf $ctx "ENTWURF" -size 110
$pdf setFont 16 [pdf4tcllib::fonts::fontSansBold]
pdf4tcllib::unicode::safeText $pdf "Inventur im Lager Nord" \
    -x [dict get $ctx left] -y [expr {$oben + 6}]

set r [pdf4tcllib::flow::columns $pdf $ctx $text -columns 2 -size 10 \
    -top $oben -firsty [expr {$oben + 36}] -newpage {
        pdf4tcllib::page::number $pdf $ctx [$pdf currentPage]
        $pdf endPage
        $pdf startPage
        pdf4tcllib::drawing::watermark $pdf $ctx "ENTWURF" -size 110
        pdf4tcllib::page::header $pdf $ctx "Inventur im Lager Nord"
    }]
pdf4tcllib::page::number $pdf $ctx [$pdf currentPage]
$pdf endPage
puts "Ergebnis: $r"

# Ohne -newpage: was nicht passt, kommt in "rest" zurueck
$pdf startPage
set r2 [pdf4tcllib::flow::columns $pdf $ctx $text \
    -columns 3 -top $oben]
$pdf endPage
puts "ohne -newpage: [dict get $r2 lines] Zeilen gesetzt,\
    [llength [split [dict get $r2 rest] \n]] Zeilen in rest"

$pdf write -file 070-spalten.pdf
$pdf destroy
```

```
Ergebnis: column 1 y 472.6929133858268 pages 1 lines 150 rest {}
ohne -newpage: 136 Zeilen gesetzt, 116 Zeilen in rest
```

The result is a dict:

Key	Meaning
column	column in which the text ends (from 0)
y	y below the last line
pages	number of page changes
lines	lines set
rest	what was not set, empty if everything fit

7.2.1 The Page Change Belongs to the Caller

When the last column is full, `columns` runs the script from `-newpage` in the caller's scope. It must close the old page, open a new one, and write into it everything a new page needs: page number, header, watermark. Afterwards `columns` restores font and size itself.

Without -newpage, columns stops when the columns of the page are full, and returns the remainder in `rest`. In the example, 136 lines fit on the page in three columns; 116 lines, including the blank lines between the paragraphs, are in `rest`. Code that does not check `rest` loses the rest of the text without any message.

7.2.2 The First Baseline

`columns` sets the first line of each column with its baseline on `-top`. If not given, that is `top` from the context, and the ascenders stick into the margin. The example therefore gives `-top` 10 points lower.

`-firsty` applies only to the first column of the first page. The second column next to it starts at `-top` again. A heading across both columns therefore needs `-firsty` **and** enough space in the second column; in the example it stands only above the first.

7.2.3 What Does Not Work

- `-align` is among the defaults (`flow::configure`) but has no effect: every line is set left-aligned.
- Font, size and line spacing apply to the whole text. A subheading in the flow is only possible by splitting the text and calling `columns` several times, with `-firsty` from the `y` of the previous call.
- The text goes through `safeText`; typographic quotation marks become straight ones (chapter Safe Unicode Output).

More columns do not save pages. Narrow columns wrap more often and leave more space free at the end of the line. Columns are a matter of readability: a line of 60 to 80 characters is easier to read than one of 130.

`flow::configure` changes the defaults for all later calls in the program. A library that uses `columns` therefore passes its options with the call instead of changing its caller's defaults.

7.3 Watermarks

```
pdf4tcllib::drawing::watermark pdf ctx text ?-size pt? ?-angle grad? \
    ?-color {r g b}? ?-font name?
```

drawing::watermark writes a text in gray diagonally across the middle of the page. It returns the font size used.

Option	Default	Meaning
-size	0	0: large enough for the text to fill most of the diagonal
-angle	45	angle counterclockwise
-color	{0.86 0.86 0.86}	shade of gray
-font	from fonts::init	font

Draw it first. pdf4tcl has no transparency; the watermark is opaque light gray. Whatever comes after it lies on top. In the example it therefore comes right after `startPage`, in the `-newpage` script as well.

With `-size 0`, "ENTWURF" on A4 becomes about 180 points high and reaches from the left almost to the right paper edge. A fixed size of around 110 points leaves a margin and looks calmer.

The watermark is built for pages with `-orient 1`. With `-orient 0` it ends up outside the page.

`pdftotext` returns rotated text scrambled. Whether the watermark is placed correctly can only be seen in a viewer.

7.4 Summary

- `flow::columns` sets text in columns across pages; the caller provides the page change as a `-newpage` script.
- Without `-newpage` the remainder is in `rest`; always check it.
- `-top` one line below `top`, otherwise the first line sticks into the margin.
- `-align` has no effect; columns do not save pages.
- `drawing::watermark` first on every page, with a fixed size of around 110 points, only with `-orient 1`.

Next chapter: Tables with Page Breaks

8 Tables with Page Breaks

8.1 Introduction

In pdf4tcl a table is a series of rectangles, lines and texts whose positions you compute yourself. For one page that works; for a list of unknown length it no longer does: at some point the page ends, the header row must be repeated on the next one, and the total belongs at the end. table::draw handles this from simple data: a list of columns and a list of rows.

pdf4tcllib also has older table procedures (`table::render`, `table::simpleTable`). This book uses only `table::draw`; it is the procedure on which the export of `tablelist` widgets is built as well.

8.2 Columns and Rows

```
set y [pdf4tcllib::table::draw pdf x y spalten zeilen \
    ?option wert ...?]
```

Columns (`spalten`) are a list, one option list per column:

Option	Default	Meaning
-header	–	text of the header cell
-width	auto	width in points or auto (by the widest content)
-align	left	left, center, right
-font	reg	reg, bold, italic, bolditalic, mono
-notruncate	0	1: never truncate text in this column (see below)

Rows (`zeilen`) are a list of lists of plain text. Formatting is not in the data but in separate options that address row and column by number (from 0, not counting the header row).

`table::draw` returns the y position below the table.

8.3 A List Across Two Pages

```
# 080-tabelle.tcl -- eine Artikelliste ueber mehrere Seiten
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Daten: 70 Artikel, jede Zeile eine Liste von Texten
set data {}
for {set i 1} {$i <= 70} {incr i} {
    set preis [expr {($i * 37 % 500) + 0.5}]
    set text "Schraube M$i, verzinkt, Packung zu 100 Stück"
    lappend data [list [format "A-%04d" $i] $text \
        [expr {$i * 3 % 40}] [format %.2f $preis]]
}
set summe 0.0
foreach z $data { set summe [expr {$summe + [lindex $z 3]}] }

set cols {
    {-header "Nr."         -width auto}
    {-header "Bezeichnung" -width auto}
    {-header "Bestand"     -width 50 -align right}
    {-header "Preis €"     -width 60 -align right}
}
```

```
# Seitenwechsel: laeuft im globalen Bereich und muss die neue
# y-Position liefern
proc tabellenUmbruch {} {
    global pdf ctx
    pdf4tcllib::page::footer $pdf $ctx "Artikelliste" \
        [$pdf currentPage]
    $pdf endPage
    $pdf startPage
    pdf4tcllib::page::header $pdf $ctx "Artikelliste (Fortsetzung)"
    return [dict get $ctx top]
}

$pdf startPage
$pdf setFont 16 [pdf4tcllib::fonts::fontSansBold]
pdf4tcllib::unicode::safeText $pdf "Artikelliste" \
    -x [dict get $ctx left] -y [expr {[dict get $ctx top] + 16}]

set y [pdf4tcllib::table::draw $pdf [dict get $ctx left] \
    [expr {[dict get $ctx top] + 30}] $cols $data \
    -ctx $ctx -zebra 1 -fontsize 9 \
    -pagebreakcmd tabellenUmbruch \
    -cellstyles {4,2 {-fg {0.8 0 0} -font bold}} \
    -footer [list "" "Summe" "" [format %.2f $summe]]]

pdf4tcllib::page::footer $pdf $ctx "Artikelliste" [$pdf currentPage]
$pdf endPage
puts "Seiten: [$pdf pageCount], Tabelle endet bei y = [format %.1f $y]"
$pdf write -file 080-tabelle.pdf
$pdf destroy
```

```
Seiten: 2, Tabelle endet bei y = 542.7
```

The header row appears on both pages, the total at the end, and the row `A-0005` has its stock (`Bestand`) in red bold.

8.3.1 What the Page Break Needs

- **-ctx** turns page breaking on. Without a context, `table::draw` draws all rows on the current page, even beyond the margin.
- **-pagebreakcmd** is your own page change: footer of the old page, new page, header. The procedure must return the y position at which the table continues.

Two things differ from `-newpage` of `flow::columns`:

- The command runs in the **global** scope, not in the caller's scope. It does not see local variables of the calling procedure; that is why the example has `global pdf ctx`.
- It must **return** the new y position. If it returns nothing, `table::draw` takes `top` from the context.

Without `-pagebreakcmd`, `table::draw` breaks the page itself: it writes `- 1 -` at the bottom

right of the old page and begins a new one, without header and footer. The last page gets no number. For a report that is rarely right; `-pagebreakcmd` is the normal case.

8.4 Appearance

Option	Default	Meaning
-fontsize	9	font size
-rowheight	0	row height; 0 means 1.8 × font size
-pad	4	inner padding of the cells
-border	1	draw lines
-header	1	draw the header row
-headerbg, -headerfg	light blue, black	colors of the header row
-zebra	0	shade every second row; -zebracolor
-maxwidth	text_w from -ctx, otherwise 480	maximum width of the table
-footer	–	total row, one value per column, bold

`-width auto` measures the widest content of the column, header included. Fixed widths apply as given. If all widths together exceed `-maxwidth`, they are scaled down in the same proportion. A table never becomes wider than needed; one spanning the whole type area needs fixed widths that add up to `text_w`.

8.4.1 Individual Rows and Cells

```
-rowstyles  {3 {-bg {1 0.95 0.8} -font bold}}
-cellstyles {4,2 {-fg {0.8 0 0} -font bold} 7,1 {-align right}}
-rowindent  {1 14 2 14}
```

`-rowstyles` takes `-bg`, `-fg` and `-font`, `-cellstyles` in addition `-align` and `-notruncate`. The cell beats the row, the row beats the zebra pattern. `-rowindent` indents the first column of a row, for example for sub-items below a bold group row.

8.5 One Row, One Value: Truncation

A row in `table::draw` is **one** line of text high. Whatever is wider than the column is truncated and ends in "...", in the header row too. There is no wrapping inside a cell.

For remarks and article names that is acceptable. For values that someone types in from the printout (passwords, serial numbers, account numbers) it is dangerous: the sheet looks complete. For such columns use `-notruncate 1`, for individual cells the same in `-cellstyles`. Then the whole value is shown, if necessary beyond the column edge. The column must therefore be wide enough; `table::widths` tells in advance how wide (section How Wide Will the Table Be?).

```
Text                          Betrag
Kennwort: Xk9-7Rt-22pQ-...      1.00      ohne -notruncate
Kennwort: Xk9-7Rt-22pQ-88aZ     1.00      mit -notruncate
```

8.6 Merged Cells

Instead of a text, a cell may be a dict with `text` and `colspan` or `rowspan`:

```
set zeilen [list \
    [list [dict create text Nord rowspan 2] Q1 120] \
    [list Q2 90] \
    [list [dict create text Summe colspan 2] 210]]
```

```
R       Q     W
        Q1   120
Nord
        Q2    90
Summe        210
```

With `colspan` the cell spans several columns, and the row has correspondingly fewer elements. With `rowspan` it occupies its column in the following rows; there it is simply left out, as in HTML: Q2 is the first element of its row and still stands in the second column. The next chapter uses `colspan` for subtotals.

What `table::draw` does with them:

- No line runs through a merged cell, and a cell with `rowspan` has the background of the row in which it begins over its whole height. Its text is centered in the block.
- Rows joined by `rowspan` are not split across two pages; the whole block moves to the next page. If it is taller than a page, `table::draw` aborts with an error.
- In an accessible PDF the cell carries `/ColSpan` or `/RowSpan`.

Up to pdf4tcllib 0.6.6, `table::draw` did not know `rowspan` (the following row slipped one column to the left) and also drew the lines through a merged cell ("Summe No|rd").

8.7 How Wide Will the Table Be?

To know before drawing how wide a table will be, or whether it fits next to an image, ask table::widths:

```
set b [pdf4tcllib::table::widths $cols $data -fontsize 9 -pdf $pdf]
dict get $b natural       ;# je Spalte: so breit, dass nichts fehlt
dict get $b min           ;# je Spalte: das längste Wort
dict get $b naturalTotal  ;# Summe
```

And how many rows fit into the rest of the page, table::fitRows:

```
set platz [expr {[dict get $ctx bottom] - $y}]
set n [pdf4tcllib::table::fitRows $platz $data -fontsize 9]
```

With this a table can be split up by hand, for example to insert a remark after the first `n` rows.

8.8 Worth Knowing

- **Unknown options are an error.** `-zebar 1` instead of `-zebra 1` aborts with "unknown option -zebar" and the list of allowed options. Up to pdf4tcllib 0.6.6 the typo passed silently: no stripes, no message.
- **-orient 0 does not work.** `table::draw` reads the direction from the document and aborts with an error when y points upward.
- **The caller formats numbers.** `table::draw` writes the text as it is in the data; `format %.2f` belongs before it.
- **Text goes through sanitize.** The quotation marks "" become straight (chapter Safe Unicode Output).

8.9 Summary

- `table::draw` with columns and plain text rows; formatting through `-rowstyles` and `-cellstyles`.
- For several pages `-ctx` and `-pagebreakcmd`; the command runs globally and returns the new y position.
- Cells are single-line and get truncated; `-notruncate` for values that are typed in from the printout.
- Merged cells with `colspan` and `rowspan`; a `rowspan` block stays on one page.

Next chapter: Group Totals and Crosstabs

9 Group Totals and Crosstabs

9.1 Introduction

A list of bookings is rarely printed the way it comes out of the database. What is wanted is the total per region, per customer, per month, or an overview with regions as rows and quarters as columns. table::group and table::pivot compute this and return rows that `table::draw` draws directly. Nothing is drawn in the process; both work on the data only.

9.2 The Example

Nine sales figures, sorted by region. One field is empty: for West (`West`) in the second quarter there is no value.

```
# 090-gruppen.tcl -- Zwischensummen je Gruppe und eine Kreuztabelle
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Umsatz je Region, Quartal und Vertreter -- nach Region sortiert
```

```
set umsatz {
    {Nord Q1 Albers 1200}  {Nord Q2 Albers 1350}  {Nord Q2 Brandt 400}
    {Nord Q4 Brandt 980}   {Süd  Q1 Celik  2100}
    {Süd  Q3 Celik  1875.5}
    {West Q1 Dorn   640}   {West Q2 Dorn   ""}    {West Q3 Dorn   720}
}

# Zahlen fuer die Ausgabe formatieren; leere Felder bleiben leer
proc betrag {v} {
    if {[string is double -strict $v]} { return [format %.2f $v] }
    return $v
}

# 1. Gruppieren: Zwischensumme je Region, Gesamtsumme am Ende
set g [pdf4tcllib::table::group $umsatz -by 0 -sum {3} \
    -label "Summe %s" -totallabel "Gesamt" -blank 1]
puts "Zwischensummen in Zeile: [dict get $g subtotals],\
    Gesamt: [dict get $g total]"

set zeilen [lmap z [dict get $g rows] {
    lset z end [betrag [lindex $z end]]
    set z
}]
set stile {}
foreach i [dict get $g subtotals] {
    lappend stile $i {-font bold -bg {0.93 0.93 0.93}}
}
lappend stile [dict get $g total] {-font bold -bg {0.85 0.85 0.85}}

$pdf startPage
set x [dict get $ctx left]
set y [expr {[dict get $ctx top] + 10}]
set y [pdf4tcllib::table::draw $pdf $x $y {
    {-header Region   -width 80}
    {-header Quartal  -width 50}
    {-header Vertreter -width 80}
    {-header Umsatz   -width 70 -align right}
} $zeilen -rowstyles $stile]

# 2. Kreuztabelle: Regionen als Zeilen, Quartale als Spalten
set p [pdf4tcllib::table::pivot $umsatz -row 0 -col 1 -value 3 \
    -rowlabel Region -colorder {Q1 Q2 Q3 Q4}]
puts "Kopf: [dict get $p header]"
foreach z [dict get $p rows] { puts "  $z" }

set kopf [lmap h [dict get $p header] {
    set a [expr {$h eq "Region" ? "left" : "right"}]
```

```
    list -header $h -width 60 -align $a
}]
set zeilen [lmap z [dict get $p rows] { lmap v $z { betrag $v } }]
pdf4tcllib::table::draw $pdf $x [expr {$y + 30}] $kopf $zeilen \
    -rowstyles [list [dict get $p total] {-font bold}]

$pdf endPage
$pdf write -file 090-gruppen.pdf
$pdf destroy
```

```
Zwischensummen in Zeile: 4 7 11, Gesamt: 12
Kopf: Region Q1 Q2 Q3 Q4 Gesamt
  Nord 1200 1750 {} 980 3930
  Süd 2100 {} 1875.5 {} 3975.5
  West 640 {} 720 {} 1360
  Gesamt 3940 1750 2595.5 980 9265.5
```

The PDF shows the grouped list with highlighted total rows and below it the crosstab:

```
Region      Quartal  Vertreter     Umsatz
Nord        Q1       Albers       1200.00
            Q2       Albers       1350.00
            Q2       Brandt        400.00
            Q4       Brandt        980.00
Summe Nord                        3930.00
Süd         Q1       Celik        2100.00
...
Gesamt                            9265.50

Region        Q1       Q2       Q3      Q4   Gesamt
Nord     1200.00  1750.00           980.00  3930.00
Süd      2100.00           1875.50          3975.50
West      640.00            720.00          1360.00
Gesamt   3940.00  1750.00  2595.50  980.00  9265.50
```

9.3 table::group

```
set g [pdf4tcllib::table::group zeilen ?option wert ...?]
```

Option	Default	Meaning
-by	0	column to group by
-sum	–	list of the columns to be summed
-label	Summe %s	text of the subtotal; %s becomes the group value
-total	1	append a grand total (only with -sum)
-totallabel	Gesamt	text of the grand total

Option	Default	Meaning
-header	0	1: a row with the group value before each group
-blank	0	1: show the group value only in the first row of the group
-strict	0	1: raise an error for unsorted data

The result is a dict:

Key	Content
rows	the rows with subtotals inserted
subtotals	the numbers of the subtotal rows
groups	the numbers of the group header rows (with -header 1)
total	the number of the grand total row, otherwise -1

The numbers fit directly into `-rowstyles` of `table::draw`; that is how the totals in the example get bold text and a gray background.

9.3.1 The Data Must Be Sorted

`group` does not sort by itself; the order of the rows stays as it comes. If equal values are not next to each other, a group falls apart into several blocks, and each gets its own subtotal. `group` notices this, but raises an error only with `-strict 1`. Without it, the warning ends up in `$::pdf4tcl::warnings`, where only someone who looks will see it. Therefore sort beforehand:

```
set umsatz [lsort -index 0 $umsatz]
```

9.3.2 Empty Fields Are Not Zero

A value that is not a number (empty, "k. A." for "not available") is skipped when summing, not counted as 0. For a sum that makes no difference; for an average in the crosstab it does.

9.3.3 The Label Spans Several Columns

The subtotal begins with a merged cell (`colspan`) that reaches up to the first sum column. Since pdf4tcllib 0.6.7, `table::draw` no longer draws a line through it (chapter Tables with Page Breaks). With older versions the first column must be as wide as the longest label, in the example 80 points for "Summe West", otherwise the line runs through the text.

9.3.4 Formatting Totals

`group` and `pivot` calculate with `expr`. What comes out is a number in Tcl notation: `3930`, `3975.5`, for some amounts also `1234.5000000000002`. For output, a `format` belongs in front; in the example `betrag` does this for every cell that is a number. Neither package sets a decimal comma:

```
proc betrag {v} {
    if {![string is double -strict $v]} { return $v }
    string map {. ,} [format %.2f $v]
}
```

9.4 table::pivot

```
set p [pdf4tcllib::table::pivot zeilen ?option wert ...?]
```

Option	Default	Meaning
-row	0	column whose values become the rows
-col	1	column whose values become the columns
-value	2	column with the numbers
-agg	sum	sum, count, avg, min, max
-empty	""	what goes into a field without a value
-colorder	sorted	order of the columns
-rowlabel	""	header of the first column
-total	1	append a total column and a total row
-totallabel	Gesamt	their label

The result contains `header` (the header row), `rows`, `cols` (the column values without header and total) and `total` (number of the total row). The header row is a list of texts; for `table::draw` it has to be turned into a column description, in the example with `lmap`.

Every crosstab makes three decisions, and `pivot` leaves them to the caller:

- **Which columns.** They result from the data. A quarter without a single value is missing; to see all four, pass `-colorder {Q1 Q2 Q3 Q4}`. Without `-colorder` the columns are sorted alphabetically, and month names then become "Apr Aug Dez ...".
- **What goes into an empty cell.** The default is nothing, not 0. "No sales" and "no data" are different statements; for 0, set `-empty 0`.
- **What happens with several values per field.** Nord has two sales in Q2 (Albers and Brandt); `-agg sum` adds them up to 1750.

With `-agg avg` the total row is the average over all values of the column, not the sum of the averages.

9.5 Summary

- `table::group` inserts subtotals and tells in which rows they are; the numbers go to `-rowstyles`.
- Sort by the group column beforehand, or use `-strict 1`.
- `table::pivot` for crosstabs; `-colorder` for the columns, `-empty` for empty fields.
- Format numbers before drawing, the decimal comma included.

Next chapter: Charts

10 Charts

10.1 Introduction

Many reports include a picture of the numbers: revenue per month as bars, two years as lines, shares as a pie. The package pdf4tclchart draws these three forms directly with pdf4tcl, without Tk and without creating an image. It is not a library for scientific charts: no logarithmic axes, no error bars, no second y-axis.

10.2 Data and Box

Each of the three procedures takes a box and the data:

```
package require pdf4tclchart
set y [pdf4tcllib::chart::bar  pdf x y breite hoehe daten ?opt ...?]
set y [pdf4tcllib::chart::line pdf x y breite hoehe daten ?opt ...?]
set y [pdf4tcllib::chart::pie  pdf x y breite hoehe daten ?opt ...?]
```

x y is the top left corner (y pointing down), width and height in points. The return value is the y position below the chart, so charts stack one below the other like paragraphs.

The data are pairs of label and number, flat or as a list of pairs:

```
{Jan 120 Feb 145 Mär 98}
{{Jan 120} {Feb 145} {Mär 98}}
```

A value that is not a number raises an error that names the label. Empty values are therefore not allowed here, unlike in `table::pivot`.

10.3 Three Charts on One Page

```
# 100-diagramme.tcl -- Balken, Linien und Torte auf einer Seite
package require pdf4tcl
package require pdf4tcllib
package require pdf4tclchart

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage

set x [dict get $ctx left]
set w [dict get $ctx text_w]
set y [expr {[dict get $ctx top] + 10}]

# Balken: Paare aus Beschriftung und Wert
set y [pdf4tcllib::chart::bar $pdf $x $y $w 180 \
    {Jan 120 Feb 145 Mär 98 Apr 160 Mai -20 Jun 132} \
    -title "Ergebnis je Monat (Tsd. €)" -values 1 -format %.0f]

# Linien: mehrere Reihen, je {Name {Paare}}
set reihen {
```

```
    {2025 {Q1 100 Q2 130 Q3 120 Q4 175}}
    {2026 {Q1 140 Q2 120 Q3 165 Q4 190}}
}
set oben [expr {$y + 20}]
set y [pdf4tcllib::chart::line $pdf $x $oben [expr {$w - 80}] 170 \
    $reihen -title "Umsatz je Quartal"]
# Die Linien haben keine eigene Legende; die Reihen selbst beschriften
pdf4tcllib::chart::legend $pdf \
    [expr {$x + $w - 70}] [expr {$oben + 30}] \
    {{2025 0} {2026 0}} [pdf4tcllib::chart::configure]

# Torte mit Legende und Anteilen
set y [pdf4tcllib::chart::pie $pdf $x [expr {$y + 20}] $w 170 \
    {Wartung 42 Neugeräte 31 Ersatzteile 18 Schulung 9} \
    -title "Umsatz nach Bereich" -legend 1]

puts "Skala für 137: [pdf4tcllib::chart::niceScale 0 137]"
puts "unten bei y = [format %.1f $y]"
$pdf endPage
$pdf write -file 100-diagramme.pdf
$pdf destroy
```

```
Skala für 137: 0.0 150.0 50.0
unten bei y = 626.7
```

10.3.1 Bars

One bar per pair, each in a different color. Negative values sit below the zero line, and the scale then extends into the negative range (in the example, Mai (May) with −20). `-values 1` writes the value above each bar, in the format given by `-format`.

10.3.2 Lines

One series or several. Several series are a list of `{Name {Paare}}` (name and pairs); the x-axis labels come from the first series, and the others are drawn at the same positions. Each data point gets a dot.

`line` does not draw the series names (`2025`, `2026`), and it has no legend. Without a legend, the reader cannot tell which line is which year. The example therefore makes the chart narrower and draws a legend next to it with chart::legend, in the same colors:

```
pdf4tcllib::chart::legend $pdf $x $y {{2025 0} {2026 0}} \
    [pdf4tcllib::chart::configure]
```

The fourth argument is the pairs, the fifth the options (for size and colors); `chart::configure` without arguments returns the defaults.

10.3.3 Pie

The slices run clockwise from twelve o'clock; `pie` computes the shares from the total. With `-legend 1`, a legend with percentages appears on the right, and the circle becomes smaller accordingly. Negative values or a total of 0 are an error.

10.4 Options

Option	Default	Meaning
-title	–	heading above the chart
-titlesize, -labelsize, -valuesize	11, 8, 8	font sizes
-values	0	write the values on the bars
-format	%g	format of every number, including on the axis
-colors	six colors	list of colors, used in order
-color	–	one color for everything
-grid	4	number of grid lines; 0 for none
-min, -max	from the data	fix the scale
-legend	0	pie only
-alt	-title	alternate text in the accessible PDF

The six default colors differ in lightness as well, not only in hue. A chart that is printed or copied in gray thus stays readable. Custom colors are best chosen the same way.

10.4.1 The Scale

The axis does not end at the largest value but at a round number: chart::niceScale turns 137 into the scale 0 to 150 in steps of 50. A grid line at 137 helps nobody who wants to read off the value. For two charts meant to be compared, give both the same scale with `-min` and `-max`; otherwise a small value in one looks as large as a large value in the other.

10.4.2 Numbers with a Decimal Comma

`-format` is passed to Tcl's `format`, which knows no decimal comma. `%.0f` and `%d` are therefore the safe choice for German reports; decimal places appear with a point.

10.5 Accessibility

In an accessible PDF (Part 5), each chart is a `Figure` element with the alternate text from `-alt` or the title. Bars, lines and labels are the content of this element, and the alternate text stands for them. A screen reader therefore says "Abbildung: Ergebnis je Monat" ("Figure: result per month") and not the individual numbers. If the numbers matter, they belong in a table as well.

10.6 Summary

- `chart::bar`, `chart::line`, `chart::pie` with a box and pairs; each returns the y position below the chart.
- Several lines need a legend of their own via `chart::legend`.

- Fix the scale with `-min`/`-max` when charts are compared.
- `--format` without a decimal comma.

Next chapter: Table of Contents

11 Table of Contents

11.1 Introduction

A table of contents stands at the front, yet it lists page numbers that are only known once everything after it has been set. And it shifts those page numbers itself, by as many pages as it is long. It cannot be done without two passes. The package pdf4tcltoc makes both passes and also writes a bookmark for each heading.

11.2 The Content as a Script

```
package require pdf4tcltoc
set r [pdf4tcllib::toc::document pdf papier skript ?option wert ...?]
```

The content is not drawn directly but passed as a script, because toc::document has to run it twice:

1. into a throwaway document; each heading records the page it lands on;
2. into the real document, after the table of contents has been written.

The script sees two variables: `pdf` (in the first pass, the throwaway document) and `ctx`. The script starts and ends its pages itself. It sets headings with toc::heading; only what is set this way goes into the table of contents.

```
# 110-inhaltsverzeichnis.tcl -- Verzeichnis mit echten Seitenzahlen
package require pdf4tcl
package require pdf4tcllib
package require pdf4tcltoc

pdf4tcllib::fonts::init -cid 1

set kapitel {
    {1 "Einleitung"} {2 "Zweck des Berichts"} {2 "Datenquellen"}
    {1 "Bestand"}    {2 "Lager Nord"} {2 "Lager Süd"}
    {1 "Ausblick"}
}
set absatz "Der Bestand wurde am Monatsende gezählt und mit der\
    Buchung abgeglichen. Abweichungen unter zwei Stück sind\
    ausgebucht."

# Der Inhalt als Skript: es sieht die Variablen pdf und ctx,
# beginnt seine Seiten selbst und laeuft zweimal -- also keine
# Nebenwirkungen.
set inhalt {
```

```
    $pdf startPage
    set y [dict get $ctx top]
    foreach k $kapitel {
        lassign $k ebene titel
        if {$ebene == 1 && $y > [dict get $ctx top]} {
            $pdf endPage; $pdf startPage; set y [dict get $ctx top]
        }
        pdf4tcllib::toc::heading $pdf $ctx y $ebene $titel
        for {set i 0} {$i < 12} {incr i} {
            if {$y > [dict get $ctx bottom] - 40} {
                $pdf endPage; $pdf startPage; set y [dict get $ctx top]
            }
            set y [pdf4tcllib::text::writeParagraph $pdf $absatz \
                [dict get $ctx left] $y [dict get $ctx text_w] 11]
        }
    }
    $pdf endPage
}

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
set r [pdf4tcllib::toc::document $pdf a4 $inhalt -title "Inhalt"]
puts "Verzeichnis: [dict get $r tocPages] Seite(n),\
    Inhalt: [dict get $r contentPages] Seiten"
foreach e [dict get $r entries] { puts "  $e" }
$pdf write -file 110-inhaltsverzeichnis.pdf
$pdf destroy
```

```
Verzeichnis: 1 Seite(n), Inhalt: 5 Seiten
  1 Einleitung 2
  2 {Zweck des Berichts} 2
  2 Datenquellen 3
  1 Bestand 4
  2 {Lager Nord} 4
  2 {Lager Süd} 5
  1 Ausblick 6
```

The table of contents itself on page 1:

```
Inhalt
Einleitung ..................................... 2
    Zweck des Berichts ......................... 2
    Datenquellen ............................... 3
Bestand ........................................ 4
    Lager Nord ................................. 4
    Lager Süd .................................. 5
Ausblick ....................................... 6
```

The page numbers in the result are already shifted by the length of the table of contents; they match the finished PDF.

11.2.1 What the Script Must Not Do

Because it runs twice, the script must have **no side effects**: no appending to a file, no incrementing a counter outside the script, no reading from a channel that the first pass has already drained. Data is loaded beforehand, like `kapitel` and `absatz` in the example.

11.2.2 Which Context Applies

`toc::document` builds its own context from the paper with `page::context papier`, that is, with a 20 mm margin and y pointing down. A `ctx` that the caller built beforehand is **not** used but overwritten: after the call, `ctx` holds the context of `toc::document`, and `pdf` holds the real document again. A different margin requires the manual route (section A Cover Page in Front).

11.3 Headings

```
pdf4tcllib::toc::heading pdf ctx yVar ebene text ?option wert ...?
```

The level (1 to 6) determines three things: the font size (18 minus 2 per level, at least 10), the indentation in the table of contents and, in the accessible PDF, the element `H1` to `H6`. `yVar` is the name of the y variable, as with the form procedures; afterwards it points below the heading.

Option	Meaning
-size	font size instead of the level's default
-gap	space below the heading (default 4)
-tag	structure element instead of H<ebene>

Two things `heading` does not do:

- **Bold.** The heading goes through `text::writeParagraph` and is set in the regular font, only larger. A bold heading has to be drawn separately without calling `heading`, but then it is missing from the table of contents.
- **Breaking.** `heading` does not check whether there is still room below the heading. The example therefore starts every level-1 chapter on a new page and checks the bottom margin before each paragraph.

11.4 Options of toc::document

Option	Default	Meaning
-title	Contents	heading of the table of contents
-titlesize	16	its size
-size	11	size of the entries
-leader	.	leader character up to the page number; "" for none
-indent	14	indentation per level in points
-gap	4	space below each heading

Option	Default	Meaning
-bookmarks	1	one bookmark per heading

The default for `-title` is English; a German document needs `-title "Inhalt"`.

`-gap` and `-bookmarks` apply to the headings of this call; afterwards the package configuration is back to what it was. For all following documents in the same process:

```
pdf4tcllib::toc::configure -bookmarks 0
```

Up to pdf4tcltoc 0.1 the two options had no effect on `toc::document`: `heading` only read the configuration, and `-bookmarks 0` still wrote bookmarks. On the manual route (below), `heading` is called without `document`; there `toc::configure` still applies.

11.5 A Cover Page in Front

`toc::document` writes the table of contents first into the document. A cover page in front of it is not possible this way. The building blocks that `document` consists of can, however, be called one by one: toc::collect, `toc::pageCount` and `toc::render`.

Procedure	does
toc::collect skript ?versatz?	runs the script and returns the entries
toc::pageCount ctx einträge ?option ...?	how many pages the table of contents needs
toc::render pdf ctx einträge ?option ...?	writes the table of contents, on pages of its own

With these, any order can be built:

```
# 110-mit-deckblatt.tcl -- Deckblatt vor dem Inhaltsverzeichnis
package require pdf4tcl
package require pdf4tcllib
package require pdf4tcltoc

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]

set inhalt {
    foreach titel {Einleitung Bestand Ausblick} {
        $pdf startPage
        set y [dict get $ctx top]
        pdf4tcllib::toc::heading $pdf $ctx y 1 $titel
        $pdf endPage
    }
}

# 1. Probelauf in ein Wegwerf-Dokument: wo stehen die Ueberschriften?
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
set eintraege [pdf4tcllib::toc::collect $inhalt]
```

```
$pdf destroy

# 2. Seitenzahlen verschieben: um das Deckblatt und das Verzeichnis
set vorne 1
set n [pdf4tcllib::toc::pageCount $ctx $eintraege -title "Inhalt"]
set eintraege [lmap e $eintraege {
    lset e 2 [expr {[lindex $e 2] + $vorne + $n}]
}]

# 3. Das echte Dokument: Deckblatt, Verzeichnis, Inhalt
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
pdf4tcllib::page::centerText $pdf $ctx "Bestandsbericht" 300 28 \
    [pdf4tcllib::fonts::fontSansBold]
$pdf endPage
pdf4tcllib::toc::render $pdf $ctx $eintraege -title "Inhalt"
eval $inhalt

puts "Einträge: $eintraege"
puts "Seiten: [$pdf pageCount]"
$pdf write -file 110-mit-deckblatt.pdf
$pdf destroy
```

```
Einträge: {1 Einleitung 3} {1 Bestand 4} {1 Ausblick 5}
Seiten: 5
```

Here the script runs in the caller's own scope and therefore sees the caller's own `ctx` with its own margin. `collect` sets no bookmarks; they are only created in the second pass with `eval`. The shift is computed by hand: pages before the table of contents plus the length of the table of contents.

`pageCount` and `render` must receive the same options; otherwise the number of table-of-contents pages is wrong. `toc::document` checks this and raises an error; the manual route checks nothing.

11.6 Summary

- `toc::document` with the content as a script without side effects; it runs twice.
- Headings with `toc::heading`; they are set in the regular font and do not break pages themselves.
- The context comes from the paper, not from the caller.
- Turn off bookmarks with `-bookmarks 0` on the call or `toc::configure -bookmarks 0` for all documents.
- Cover page in front: `collect`, `pageCount`, `render` by hand.

Next chapter: Form Layout

12 Form Layout

12.1 Introduction

In pdf4tcl, a fillable form consists of fields that are placed at a position with `addForm`. Labels, spacing, sections and the arithmetic of where the next field goes are left to the caller. The namespace form in pdf4tcllib takes over this layout: a label on the left, the field on the right, the next one a line further down.

How fields themselves work (types, values, calculation, JavaScript) is explained in the book *pdf4tcl*. This chapter shows the structure and the places where a form from pdf4tcllib looks different than expected.

12.2 The Building Blocks

The procedures form::section, form::labelField, form::row, `form::separator`, form::orderTable and form::sumLine all take `pdf`, `ctx` and the **name** of the y variable and move it further down:

Procedure	draws
form::section pdf ctx yVar titel	a gray bar with a bold title
form::labelField pdf ctx yVar beschriftung typ ?option ...?	label and field on one line
form::row pdf ctx yVar felder	several pairs side by side
form::separator pdf ctx yVar	a thin line
form::orderTable pdf ctx yVar köpfe breiten ?zeilen? ?option ...?	an item table
form::sumLine pdf ctx yVar breiten text wert ?option ...?	the total line below it

The types are those of `addForm`: `text`, `password`, `checkbox`, `combobox`, `listbox`, `radiobutton`, `pushbutton`, `signature`. The x position comes from `SX` in the context, the width from `SW`.

12.3 An Order Form

```
# 120-formular.tcl -- ein ausfuellbares Bestellformular
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
# Formularfelder brauchen eine Schrift mit cp1252: dieselbe
# TrueType-Datei ein zweites Mal angemeldet, fuer form:: eingestellt
pdf4tcl::createFont _Pdf4tcl_Base_Regular FormSans     cp1252
pdf4tcl::createFont _Pdf4tcl_Base_Bold    FormSansBold cp1252
pdf4tcllib::form::configure -fontFamily FormSans \
    -fontFamilyBold FormSansBold
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
```

```
$pdf startPage
set y [expr {[dict get $ctx top] + 10}]
# Rahmen und Hintergrund gibt jedes Feld selbst an
set rahmen {-bordercolor {0.6 0.6 0.6} -borderwidth 1
            -bgcolor {0.97 0.97 0.97}}

pdf4tcllib::form::section    $pdf $ctx y "Kunde"
pdf4tcllib::form::labelField $pdf $ctx y "Name:" text -id f_name \
    -init "Müller GmbH" {*}$rahmen
pdf4tcllib::form::labelField $pdf $ctx y "E-Mail:" text -id f_mail \
    {*}$rahmen
# row nimmt dieselben Angaben als Schluessel ohne Bindestrich
set r [dict create bordercolor {0.6 0.6 0.6} borderwidth 1 \
    bgcolor {0.97 0.97 0.97} labelw 40 type text]
pdf4tcllib::form::row $pdf $ctx y [list \
    [dict merge $r {label "PLZ:" width 120 id f_plz}] \
    [dict merge $r {label "Ort:" width 260 id f_ort}]]
pdf4tcllib::form::labelField $pdf $ctx y "Versand:" combobox \
    -id f_versand -options {Paket Spedition "Abholung Lager Süd"} \
    -init Paket -fieldw 150 {*}$rahmen
# Das Kaestchen bekommt seinen Rahmen als Grafik (siehe Text)
set labelW [dict get [pdf4tcllib::form::configure] labelW]
set xk [expr {[dict get $ctx SX] + $labelW + 4}]
$pdf setStrokeColor 0.6 0.6 0.6
$pdf rectangle $xk $y 12 12
$pdf setStrokeColor 0 0 0
pdf4tcllib::form::labelField $pdf $ctx y "Eilig:" checkbox \
    -id f_eilig -fieldw 12 -fieldh 12
pdf4tcllib::form::separator  $pdf $ctx y

pdf4tcllib::form::section    $pdf $ctx y "Bestellung"
set breiten {40 230 60 90}
set zahl {number decimals 2 sep de}
pdf4tcllib::form::orderTable $pdf $ctx y \
    {Pos Artikel Menge Betrag} $breiten \
    {{1 "Schraube M8" 100 "12.50"}} -emptyRows 3 -cellForm f_pos \
    -cellOpts [list 3 [list -align right -format $zahl]]
# Die Summe rechnet der Betrachter: aus den Feldern der Spalte Betrag
set betraege [lmap r {0 1 2 3} { string cat f_pos_ $r _3 }]
pdf4tcllib::form::sumLine $pdf $ctx y $breiten "Summe:" "" \
    -id f_summe -init "12.50" -calculate [list sum $betraege] \
    -format $zahl
pdf4tcllib::form::labelField $pdf $ctx y "Bemerkung:" text -id f_bem \
    -multiline 1 -fieldh 40 {*}$rahmen

$pdf endPage
$pdf write -file 120-formular.pdf
```

```
$pdf destroy
puts "y am Ende: [format %.1f $y]"
```

```
y am Ende: 388.7
```

The PDF contains 24 fields: Name, E-Mail, PLZ (postal code), Ort (city), Versand (shipping), Eilig (urgent), sixteen cells of the item table (`f_pos_0_0` to `f_pos_3_3`), the total and the Bemerkung (remark).

12.3.1 labelField

Everything `labelField` does not know itself is passed to `addForm`: `-id`, `-init`, `-options`, `-multiline`, the border colors. Three options belong to `labelField`:

Option	Default	Meaning
-labelw	90	width of the label
-fieldw	rest of the line	width of the field
-fieldh	16	height of the field

`-fieldw` applies to every type. A checkbox without `-fieldw` is therefore as wide as the whole line; that is why the example has `-fieldw 12 -fieldh 12`.

12.3.2 row

`row` takes a list of dicts with the keys `label`, `type`, `width` (label and field together), `id`, `init`, `options`, `labelw`, `fieldh` and the appearance keys `bordercolor`, `borderwidth`, `bgcolor`, `align`, `calculate`, `format`. Without `labelw`, the label is as wide as its text.

12.3.3 orderTable and sumLine

`orderTable` draws a header row (white on dark blue), the rows from `zeilen` and `-emptyRows` empty rows in addition. Without `-cellForm` the cells are printed text; with `-cellForm präfix` they are text fields with the ID `präfix_zeile_spalte` (prefix, row, column). `-cellOpts {spalte {optionen}}` gives the fields of one column further options for `addForm`, in the example right-aligned and numeric.

`sumLine` sets the text right-aligned in front of the last column and the value into the last one. With `-id` the value becomes a field, and with `-calculate` the viewer does the arithmetic:

```
-calculate {sum {f_pos_0_3 f_pos_1_3 f_pos_2_3 f_pos_3_3}}
-format    {number decimals 2 sep de}
```

Instead of `sum`, `product`, `average`, `min` and `max` also work. `sep de` shows the number with a decimal comma. The calculation only happens in viewers with form JavaScript, such as Acrobat Reader; in Evince or Okular the value from `-init` stays.

12.4 Fonts in Forms

A form field has two fonts: the one in which the page shows the value (the appearance), and the one in which the viewer redraws the value when someone types. The viewer must be able to encode the second one itself. With the fonts from `fonts::init` this goes wrong:

- **With -cid 1**, a drop-down list shows boxes instead of letters (□□□N□H□ instead of `Paket`) as

soon as the viewer redraws it. `addForm` cannot handle CID fonts.
- **Without -cid**, pdf4tcllib encodes the font with a character table of its own. The appearance is correct, but the stored value uses the same table: when the form data is exported, `Müller` becomes `MÜller`, and `ß` becomes `¿`.

The remedy is the same TrueType file, registered a second time, this time with the encoding `cp1252`, and `form` configured to use this font:

```
pdf4tcl::createFont _Pdf4tcl_Base_Regular FormSans     cp1252
pdf4tcl::createFont _Pdf4tcl_Base_Bold    FormSansBold cp1252
pdf4tcllib::form::configure -fontFamily FormSans \
    -fontFamilyBold FormSansBold
```

`_Pdf4tcl_Base_Regular` and `_Pdf4tcl_Base_Bold` are the names under which `fonts::init` loads the files. The rest of the text on the page stays in the CID font. With `cp1252`, German umlauts, `ß` and `€` work; characters outside it do not belong in a form field.

Incidentally, without `form::configure` the header row of `orderTable` and the text of `sumLine` always use **Helvetica-Bold**, even after `fonts::init`. This font is not embedded; a form for PDF/A or PDF/UA fails with it.

The cells of `orderTable` appear bold: with `-cellForm` the fields take over the font of the header row. There is no option against it.

12.5 Border and Background

Every field created by `labelField`, `row`, `orderTable -cellForm` and `sumLine -id` gets its background and border from `form::configure`: `fieldBg`, `fieldBorder` and `fieldBorderW` become `-bgcolor`, `-bordercolor` and `-borderwidth`. If the call gives one of these options itself, its value applies. In the example every field gets a slightly darker border:

```
set rahmen {-bordercolor {0.6 0.6 0.6} -borderwidth 1
            -bgcolor {0.97 0.97 0.97}}
```

`fieldBg {}` leaves out the background, `fieldBorderW 0` the border. Up to pdf4tcllib 0.6.6 the settings did not reach any field; without an explicit value a field had no border, and on screen there was no way to tell where to click.

For a checkbox this is not enough. As soon as one field of the document calculates or formats (`-calculate`, `-format`, `-js`), the PDF asks the viewer to redraw all fields (`/NeedAppearances`). For text fields the viewer then takes border and background from the settings above. For a checkbox, however, pdf4tcl writes the border only into its own appearance, which then no longer applies, and the box is invisible. The example therefore draws its border as a rectangle on the page, at the same position as the field.

12.6 Settings

`form::configure` without arguments returns all settings; with pairs it sets them. Unknown names are an error.

Name	Default	Meaning
fontFamily, fontFamilyBold	Helvetica, Helvetica-Bold	fonts (see above)
fontSize, fontSizeLabel, fontSizeSection	9, 9, 10	sizes
fieldH	16	field height and row height of the table
labelW, labelGap	90, 4	width of the label, gap to the field
rowGap, sectionGap	6, 10	space after a row and a section
sectionBg, lineColor	gray	section bar, separator line
fieldBg, fieldBorder	light gray, gray	background and border of the fields
fieldBorderW	1	border width of the fields; 0 = none

The names are stored without a hyphen but set with one: `form::configure -labelW 110`.

12.7 Worth Knowing

- **No page break.** The procedures move y forward but do not start a new page. A form that spans more than one page has to be split by hand.
- **The label sits at the bottom.** It sits on the baseline of the field; with a tall field (`-fieldh 40`), "Bemerkung:" therefore stands next to the lower edge.
- **Accessibility.** Label and field lie together in a `Form` element, with the label as alternate text; in the item table it is the column header.

12.8 Summary

- `section`, `labelField`, `row`, `orderTable`, `sumLine` build a form line by line; they move y forward.
- Register a `cp1252` font for the fields and set it with `form::configure`; otherwise wrong values or boxes appear.
- Give the border on each field explicitly; a checkbox in a calculating form needs a drawn border.
- `sumLine -calculate` calculates in the viewer, not in Tcl.

Next chapter: Label Sheets

13 Label Sheets

13.1 Introduction

Address labels, shelf tags, shipping stickers: a sheet with 24 labels is a page with 24 rectangles

whose position has to be accurate to a tenth of a millimeter. The package pdf4tcllabels knows the dimensions of common sheets, distributes the records over the positions and starts new sheets. What goes on a label is written once by the caller, as a script.

13.2 The Sheets

Name	Grid	per sheet	Label (mm)	Margin l/t (mm)
3427	2 × 2	4	105.0 × 148.0	0 / 0
3474	3 × 8	24	70.0 × 37.0	0 / 0
3475	3 × 7	21	70.0 × 42.3	0 / 0.45
3483	3 × 5	15	70.0 × 50.8	0 / 21.5
4737	3 × 9	27	63.5 × 29.6	7.25 / 13.0

All on A4. 3427 is an A6 shipping label; 4737 is the only sheet with gaps between the labels.

labels::sheet returns the dimensions of a sheet as a dict, in points for calculation and in millimeters for comparison with the package:

```
set geo [pdf4tcllib::labels::sheet 3474]
dict get $geo cols            ;# 3
dict get $geo w               ;# 198.425... (Punkt)
dict get $geo wmm             ;# 70.0
```

A sheet of one's own is added with `labels::define`, in millimeters:

```
pdf4tcllib::labels::define regal {
    w 48.5 h 25.4 cols 4 rows 10 left 8.0 top 21.5
    pitchx 50.0 pitchy 25.4 desc "Regalschilder aus dem Schrank"
}
```

`pitchx` and `pitchy` are the distance from label to label, not the gap. On sheets without gaps they equal the label size.

13.3 Addresses on Two Sheets

Thirty addresses, and the sheet has already been used: the first two labels are missing.

```
# 130-etiketten.tcl -- Adressetiketten auf Bogen 3474 (3 x 8)
package require pdf4tcl
package require pdf4tcllib
package require pdf4tcllabels

pdf4tcllib::fonts::init -cid 1

set adressen {}
set lang "Gebrüder Hoffmann-Schwarzenbach Metallbau GmbH & Co. KG"
for {set i 1} {$i <= 30} {incr i} {
```

```
    lappend adressen [dict create \
        name    [expr {$i == 5 ? $lang : "Firma Beispiel $i"}] \
        strasse "Industriestraße $i" \
        ort     "[expr {10000 + $i}] Musterstadt"]
}

set geo [pdf4tcllib::labels::sheet 3474]
puts "Bogen 3474: [dict get $geo cols] x [dict get $geo rows],\
    [dict get $geo wmm] x [dict get $geo hmm] mm"

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
set bogen [pdf4tcllib::labels::render $pdf 3474 $adressen {x y w h a} {
    set font [pdf4tcllib::fonts::fontSans]
    set fett [pdf4tcllib::fonts::fontSansBold]
    set breit [expr {$w - 16}]
    set name [dict get $a name]
    # Name: kleiner setzen, bis 8 pt; reicht das nicht, zwei Zeilen
    set g [pdf4tcllib::labels::fitSize $pdf $fett $name $breit 11 8]
    $pdf setFont $g $fett
    if {[$pdf getStringWidth $name] <= $breit} {
        set zeilen [list $name]
    } else {
        set zeilen [pdf4tcllib::labels::wrap $pdf $name $breit]
        set zeilen [lrange $zeilen 0 1]
    }
    set ty [expr {$y + 26 - 5 * ([llength $zeilen] - 1)}]
    foreach z $zeilen {
        $pdf text $z -x [expr {$x + 8}] -y $ty
        set ty [expr {$ty + $g + 1}]
    }
    $pdf setFont 10 $font
    set tx [expr {$x + 8}]
    $pdf text [dict get $a strasse] -x $tx -y [expr {$y + 46}]
    $pdf text [dict get $a ort]     -x $tx -y [expr {$y + 60}]
} -start 2 -clip 1 -frame layer]
puts "Bögen: $bogen"
$pdf write -file 130-etiketten.pdf
$pdf destroy
```

```
Bogen 3474: 3 x 8, 70.0 x 37.0 mm
Bögen: 2
```

On the first sheet, positions 1 and 2 stay empty, addresses 1 to 22 fill the rest, and the remaining eight are on the second sheet.

13.3.1 render

```
set boegen [pdf4tcllib::labels::render pdf bogen datensätze \
    {x y w h satz} skript ?option wert ...?]
```

labels::render opens and closes the pages itself; a `startPage` before it is wrong. For each record, the script runs in the caller's scope with the five variables from the list: top left corner, width and height in points, and the record as it appears in the list. The return value is the number of sheets.

Option	Default	Meaning
-start N	0	leave the first N positions empty
-only {i j ...}	–	use exactly these positions, for reprints
-offsetx, -offsety	0.0	printer offset in millimeters
-clip	0	1: draw nothing beyond the edge of the label
-frame	0	1: print outlines permanently, layer: outlines as a layer that can be switched off
-tag	Sect	structure element per label in the accessible PDF

Unknown options are an error.

`-start` applies to the first sheet only. How many labels are already missing is known only to whoever holds the sheet; a program that asks for it saves half a sheet per print run.

13.3.2 Outlines for Checking

`-frame 1` draws the outline of each label into the page, below the content. That is for a test print on paper, not for the labels themselves. `-frame layer` puts the outlines into a layer "Label grid", above the content; the viewer can switch it off, and it is not printed. This way the screen shows whether everything lies within the label, and the same file is printed. `-frame layer` requires pdf4tcl 0.9.4.70 or newer and does not work with PDF/A.

13.4 Text That Is Too Long

A label is narrow: at 11 points, 70 mm hold about 35 characters. A long company name would otherwise run over the edge onto the neighboring label. The package has three helpers, labels::fitSize, `labels::wrap` and `labels::ellipsize`, all with real width measurement:

Procedure	does
labels::fitSize pdf schrift text maxB größe ?min?	shrinks in steps of 0.5 pt, not below min (default 6)
labels::wrap pdf text maxB	wraps into lines at spaces, using the current font
labels::ellipsize pdf text maxB ?ende?	shortens and appends …

The example works in two stages: first shrink down to 8 points; if the name still does not fit, use two lines. Label 5 ("Gebrüder Hoffmann-Schwarzenbach Metallbau GmbH & Co. KG") thus appears in two lines at 8 points.

`fitSize` stops at `min`, even if the text still does not fit. The result therefore has to be measured again, as the example does with `getStringWidth`. `wrap` measures with the font that is currently set; call `setFont` first.

`ellipsize` is meant for remarks and references. A shortened street is not an address.

`text::fitSize` from the chapter Setting Text does the same as `labels::fitSize`, but with a different argument order and in whole points. Do not mix the two up.

13.4.1 -clip as a Safeguard

`-clip 1` cuts off everything that reaches beyond the edge of the label. That does not replace fitting, because the rest of the name is then simply missing. But it prevents a forgotten case from writing on the neighboring label.

13.5 Calibrating the Printer

Hardly any printer places output accurate to the millimeter. An offset of one or two millimeters is common, and on a sheet without margins such as 3474 the first line then slides half off the label.

```
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
pdf4tcllib::labels::calibration $pdf 3474
$pdf write -file kalibrierung.pdf
```

`calibration` writes a sheet with a millimeter scale along the edges and the outlines of all labels. Print it, lay it on a real sheet, hold both against the light, read off the offset. From then on, the two numbers go to `render` as `-offsetx` and `-offsety`, once per printer.

Print at **actual size**. "Fit to page" shrinks by a few percent: invisible in the first row, several millimeters in the last.

13.6 Summary

- `labels::render` with a sheet name, records and a script per label; it creates the pages itself.
- `-start` for partly used sheets, `-only` for reprints.
- Fit long names with `fitSize` and `wrap`, `-clip 1` as a safeguard.
- Calibrate the printer once with `calibration` and pass the offset.

Next chapter: Images

14 Images

14.1 Introduction

pdf4tcl places an image into a rectangle with `putImage`. If width and height are given, the image is stretched to exactly that area; if the aspect ratio does not match, it is distorted, and nothing points this out. pdf4tcllib has two procedures that avoid this: image::place fits an image into a frame without distorting it, and image::gallery places many images with captions in a grid across several pages.

Both work with images that pdf4tcl has loaded with `addImage` (PNG, JPEG). Tk is not needed for this.

14.2 Loading

```
set id [$pdf addImage datei.png]
lassign [$pdf getImageSize $id] breite hoehe     ;# in Pixel
```

`addImage` returns an identifier. The image is then stored once in the file, even if it appears on many pages.

A peculiarity of pdf4tcl: **a JPEG that is loaded before the first startPage starts a page by itself.** A PNG does not. Calling `startPage` afterwards leaves an empty first page. That is why the example starts the page first and then loads the images.

14.3 An Image in a Frame

```
lassign [pdf4tcllib::image::place pdf id x y breite hoehe \
    ?option wert ...?] bx by bb bh
```

Option	Default	Meaning
-fit	contain	contain, cover or stretch
-anchor	center	nw n ne w center e sw s se
-dpi	–	warn if the resolution is below this
-alt	–	alternate text; the image becomes a Figure element
-clip	1	with cover, cut off the overhang

- **contain**: The whole image is visible; part of the frame stays empty.
- **cover**: The frame is filled completely; whatever sticks out is cut off.
- **stretch**: like `putImage` with width and height, i.e. distorted.

With `contain`, `-anchor` says where the image sits in the frame; with `cover`, it says which part is kept. A portrait with the head at the top wants `n`.

The return value is the area the image actually occupies: with `contain` smaller than the frame, with `cover` the frame itself. The next line continues below it.

With `cover`, the image is only hidden, not cropped: the whole image is stored in the file. To use a small part of a large photo, crop it beforehand.

14.4 The Example

Page 1 places the same portrait image (400 × 600 pixels) into three equal frames of 150 × 120 points, with an image that is too small below them. Pages 2 and 3 are a gallery with five images, four per page.

```
# 140-bilder.tcl -- Bilder einpassen und als Galerie setzen
package require pdf4tcl
package require pdf4tcllib

```

```
pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Bilder einmal laden; addImage gibt eine Kennung zurueck.
# Erst die Seite beginnen: ein JPEG vor startPage beginnt selbst eine.
$pdf startPage
set dir [file join [file dirname [info script]] bilder]
foreach name {hochformat.png querformat.jpg quadrat.png
              panorama.jpg klein.png} {
    set bild($name) [$pdf addImage [file join $dir $name]]
}
puts "hochformat.png: [$pdf getImageSize $bild(hochformat.png)] Pixel"

# Seite 1: dasselbe Hochformat in drei gleich grossen Rahmen
set x [dict get $ctx left]
set y [expr {[dict get $ctx top] + 20}]
$pdf setFont 9 [pdf4tcllib::fonts::fontSans]
foreach fit {contain cover stretch} {
    set r [pdf4tcllib::image::place $pdf $bild(hochformat.png) \
        $x $y 150 120 \
        -fit $fit -anchor n -alt "Beispielbild ($fit)"]
    puts [format "%-8s -> %s" $fit [lmap v $r {format %.0f $v}]]
    $pdf setStrokeColor 0.8 0 0
    $pdf rectangle $x $y 150 120              ;# der Rahmen
    $pdf setStrokeColor 0 0 0
    $pdf text "-fit $fit" -x $x -y [expr {$y + 134}]
    set x [expr {$x + 170}]
}

# Zu wenig Aufloesung: 80 x 60 Pixel auf 160 x 120 Punkt sind 36 dpi
pdf4tcllib::image::place $pdf $bild(klein.png) [dict get $ctx left] \
    [expr {$y + 160}] 160 120 -dpi 150
set w [lindex $::pdf4tcl::warnings end]
puts "Warnung: [string map {" (" "\n  ("} $w]"
$pdf endPage

# Seite 2 und 3: Galerie mit Unterschriften, 2 x 2 je Seite
proc kopf {} {
    global pdf ctx
    pdf4tcllib::page::header $pdf $ctx "Bildergalerie"
}
set liste [list \
    [list $bild(querformat.jpg) "Querformat 900 x 500"] \
    [list $bild(hochformat.png) "Hochformat 400 x 600"] \
    [list $bild(quadrat.png)    "Quadrat"] \
    [list $bild(panorama.jpg) \
```

```
        "Panorama 1200 x 300, eine sehr lange Unterschrift"] \
    [list $bild(querformat.jpg) "noch einmal quer"]]
$pdf startPage
kopf
set seiten [pdf4tcllib::image::gallery $pdf $liste \
    [dict get $ctx left] [expr {[dict get $ctx top] + 10}] \
    [dict get $ctx text_w] 500 \
    -cols 2 -rows 2 -gap 12 -anchor s -pagecmd kopf]
$pdf endPage
puts "Galerie: $seiten Seiten"

$pdf write -file 140-bilder.pdf
$pdf destroy
```

```
hochformat.png: 400 600 Pixel
contain  -> 92 77 80 120
cover    -> 227 77 150 120
stretch  -> 397 77 150 120
Warnung: image::place: image11 at 36 dpi
  (80x60 pixels on 160.0x120.0 points), below 150
Galerie: 2 Seiten
```

With `contain`, the image is 80 points wide and sits in the middle of the frame; 35 points stay free on the left and on the right. With `cover`, it fills the frame; with `-anchor n` the upper part is kept, and the label "hoch" ("portrait") at the bottom edge is lost. With `stretch`, the circle is an ellipse.

14.4.1 Resolution

80 × 60 pixels on 160 × 120 points are 36 dpi; printed, that looks jagged. `-dpi 150` then writes a warning to `$::pdf4tcl::warnings`. It does not go to the screen; to see it, print the list at the end. pdf4tcl itself already warns below 150 dpi; with `-dpi` the caller's own threshold applies, and the warning comes only once.

14.5 Gallery

```
set seiten [pdf4tcllib::image::gallery pdf bilder x y breite hoehe \
    ?option wert ...?]
```

Each entry in `bilder` is `{id "Unterschrift"}` or just `id`. The box `x y breite hoehe` is divided into `-cols` × `-rows` cells; when a page is full, `gallery` starts the next one and continues at the same position there. The return value is the number of pages.

Option	Default	Meaning
-cols, -rows	2, 3	grid per page
-gap	8	spacing between the cells
-caption	unten	unten or keine

Option	Default	Meaning
-captionH	14	height of the band for the caption
-fontsize, -font	8, Sans	font of the caption
-fit, -anchor, -dpi	as for place	per image
-pagecmd	–	runs after each new startPage

`-caption` expects the German words (`unten` "below", `keine` "none"). Any other value, `below` included, silently switches the captions off.

The caption sits at the bottom edge of the cell, the image with `-anchor center` in the middle above it. For a landscape image in a tall cell, this leaves a gap in between. With `-anchor s`, each image sits directly on its caption, as in the example.

A caption wider than the cell is set smaller (down to 5 points), not shortened. The panorama caption in the example shows this.

`-pagecmd` runs in the caller's scope. The caller does the first `startPage` and its header line, and the last `endPage` as well.

If the box is too small for the grid, `gallery` aborts with `the frame is too small for 2x3 cells` (numbers depending on the grid). Up to pdf4tcllib 0.6.6, a bug in the message produced `invalid command name "2"` instead.

14.6 Images from Tk

For an image that exists as a Tk `photo` (from a canvas, from a camera), there are `image::insert` and `image::insertAt`. They need Tk, scale to a width and start a new page if needed, but take twelve arguments. If the image also exists as a file, `addImage` with `place` is simpler.

14.7 Summary

- Load images with `addImage`, only after the first `startPage`.
- `image::place` with `-fit contain` or `cover`, never distorted; `-anchor` chooses the position or the visible part.
- `-dpi` warns of too low a resolution, in `$::pdf4tcl::warnings`.
- `image::gallery` for grids across several pages; `-anchor s` places the images on their captions.

Next chapter: Arrows and Callouts

15 Arrows and Callouts

15.1 Introduction

A flow diagram, a labeled plan, a note next to a table: these need boxes, arrows between them and now and then a callout that points at something. pdf4tcl has lines, rectangles and polygons; the arithmetic of which direction an arrowhead points in and where the line in front of it has to stop is done by the namespace drawing in pdf4tcllib.

15.2 The Procedures

Four procedures: drawing::arrow, drawing::arrowHead, drawing::callout and drawing::roundedRect.

Procedure	draws
drawing::arrow pdf x1 y1 x2 y2 ?option ...?	a line with a head
drawing::arrowHead pdf x y winkel ?option ...?	only the head, at a point
drawing::callout pdf x y b h zielX zielY text ?option ...?	a box with a pointer and text
drawing::roundedRect pdf x y b h r ?rand? ?füllen?	a rectangle with rounded corners

Color and line width are the ones currently set; none of the procedures changes them (except `callout`, see below).

15.3 A Flow Diagram

Three boxes in a row, a fourth one below, straight and bent arrows and two callouts.

```
# 150-pfeile.tcl -- ein Ablaufschema mit Pfeilen und einer Sprechblase
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage

# Ein Kasten mit runden Ecken und Text in der Mitte
proc kasten {x y w h text} {
    global pdf
    $pdf setFillColor 0.88 0.92 0.97
    pdf4tcllib::drawing::roundedRect $pdf $x $y $w $h 6 1 1
    $pdf setFillColor 0 0 0
    $pdf setFont 11 [pdf4tcllib::fonts::fontSansBold]
    $pdf text $text -align center \
        -x [expr {$x + $w / 2.0}] -y [expr {$y + $h / 2.0 + 4}]
}

set top [expr {[dict get $ctx top] + 120}]
set links [dict get $ctx left]
set b 110; set h 40; set abst 50
foreach name {Auftrag Lager Versand} {
    kasten $links $top $b $h $name
```

```
    set rechts($name) [expr {$links + $b}]
    set mitte($name)  [expr {$links + $b / 2.0}]
    set links [expr {$links + $b + $abst}]
}
set unten [expr {$top + 110}]
kasten [expr {$mitte(Lager) - $b / 2.0}] $unten $b $h "Rücksendung"

# Gerade Pfeile: von Kante zu Kante, die Spitze endet genau am Kasten
set my [expr {$top + $h / 2.0}]
$pdf setLineWidth 1.2
foreach k {Auftrag Lager} {
    pdf4tcllib::drawing::arrow $pdf \
        $rechts($k) $my [expr {$rechts($k) + $abst}] $my
}

# Geknickt, Versand -> Ruecksendung: Linie selbst ziehen, die
# Spitze mit arrowHead; sie zeigt nach links (Winkel pi)
set yr [expr {$unten + $h / 2.0}]
set xr [expr {$mitte(Lager) + $b / 2.0}]          ;# rechte Kante
$pdf setStrokeColor 0.7 0 0
$pdf setFillColor 0.7 0 0
$pdf line $mitte(Versand) [expr {$top + $h}] $mitte(Versand) $yr
$pdf line $mitte(Versand) $yr [expr {$xr + 8}] $yr
pdf4tcllib::drawing::arrowHead $pdf $xr $yr [expr {acos(-1)}]
# Beidseitig, nur umrissen: Lager <-> Ruecksendung
pdf4tcllib::drawing::arrow $pdf $mitte(Lager) [expr {$top + $h}] \
    $mitte(Lager) $unten -at both -filled 0
$pdf setStrokeColor 0 0 0
$pdf setFillColor 0 0 0

# Sprechblase ueber dem Lager, Zeiger auf die obere Kante
set n [pdf4tcllib::drawing::callout $pdf [expr {$mitte(Lager) + 20}] \
    [expr {$top - 95}] 170 50 $mitte(Lager) $top \
    "Der Bestand wird jeden Morgen um 6 Uhr mit der Buchhaltung\
     abgeglichen." \
    -fill {1 0.97 0.8} -radius 6]
puts "Sprechblase: $n Zeilen"

# Zu viel Text fuer den Kasten: wird kleiner, nicht gekuerzt
set n [pdf4tcllib::drawing::callout $pdf \
    [dict get $ctx left] [expr {$top - 95}] 120 40 \
    $mitte(Auftrag) $top \
    "Aufträge vor 14 Uhr gehen am selben Tag raus, alle anderen am\
     nächsten Werktag." \
    -fill {0.93 0.93 0.93}]
puts "Kleine Blase: $n Zeilen"
```

```
$pdf endPage
$pdf write -file 150-pfeile.pdf
$pdf destroy
```

```
Sprechblase: 2 Zeilen
Kleine Blase: 3 Zeilen
```

15.3.1 Straight Arrows

`arrow` draws the line from `x1 y1` to `x2 y2` and puts the head at the end. The head **ends** exactly at the target point, and the line stops one head length before it. If the line ran all the way to the end, it would stick through the filled head and leave a stroke in its middle. For arrows between boxes, this means: calculate from edge to edge, not from center to center.

Option	Default	Meaning
-at	end	end, start or both
-length	8	length of the head
-width	6	width of the head
-filled	1	0: head only outlined

Unknown options and a wrong value for `-at` are errors.

15.3.2 Bent Arrows

`arrow` draws only straight lines. For a connection around a corner, draw the lines yourself and put the head at the end with `arrowHead`. The angle is in radians; 0 points to the right, π to the left. With y pointing down, π/2 points **down**.

Here too, the last line has to stop before the head, in the example 8 points (the default for `-length`) before the box. For a slanted last segment, `atan2(dy, dx)` gives the angle.

15.4 Callouts

`callout` draws a box with rounded corners, a triangular pointer to the point `zielX zielY` and the text inside, wrapped to the width. Which side the pointer sits on follows from the position of the target: if it lies above the box, at the top; below, at the bottom; otherwise left or right. The return value is the number of text lines.

Option	Default	Meaning
-fill	–	fill color {r g b}; without it only the outline
-stroke	1	draw the outline
-radius	4	corner radius
-tail	12	width of the pointer at the box
-pad	5	inner padding
-fontsize, -font	9, Sans	font

Option	Default	Meaning
-alt	–	alternate text; the whole becomes a Figure element

If the text does not fit into the height, it is set smaller, in steps of 1 point down to 5 points. Nothing is cut: the small callout on the left holds the whole sentence, in three lines at 7 points instead of 9. Whether that is still readable is for the caller to decide; the number of lines in the result is a clue, the font size is not returned.

`callout` sets the fill color to black for the text and leaves it that way.

Pointer and box are two shapes, each with its own outline. A line therefore remains where the pointer joins the box; the callout is not open there as in a comic strip. To avoid this, use `-stroke 0` and a stronger fill color.

15.5 Rounded Corners

```
# Radius 6, mit Umriss, gefüllt
pdf4tcllib::drawing::roundedRect $pdf $x $y $b $h 6 1 1
```

Each corner is made of eight straight segments, not of curves. With small radii this is invisible; with a radius of 30 points under strong magnification it shows.

According to the documentation, the option `-clip 1` sets a clipping path with rounded corners, for example for a photo with rounded corners. But it clips to the **square-cornered** rectangle. pdf4tcllib offers no way to get rounded corners around an image.

15.6 Accessibility

In the accessible PDF, arrows, heads, boxes and pointers are decoration (artifacts); a screen reader skips them. The text of a callout is content and appears as a paragraph in the structure tree. A diagram as a whole needs an alternate text; otherwise all that remains of it is a sequence of words without arrows. More on this in Part 5.

15.7 Summary

- `drawing::arrow` from edge to edge; the line ends before the head.
- Bent arrows: draw the lines yourself, the head with `arrowHead`, angle in radians.
- `drawing::callout` chooses the side of the pointer itself and shrinks text that is too long.
- `roundedRect -clip` clips with square corners.

Next chapter: Formulas

16 Formulas

16.1 Introduction

A test report with a tolerance formula, a data sheet with a characteristic curve, an invoice with exponents: formulas also turn up in documents that are not textbooks. pdf4tcl writes text on a baseline; superscripts, fractions and roots have to be assembled by hand. The namespace math in pdf4tcllib does this for two notations: a small subset of LaTeX and an older eqn-like notation

from the Tcler's Wiki.

This is not formula typesetting as in TeX. For multi-line equations, matrices or brackets that grow with their content, an external program (for example KaTeX to SVG, then as an image) is the better route.

16.2 The Font

Greek letters, ±, ∑, ∞ and √ are not in any of the standard fonts. Without a TrueType font in CID mode they become?, without an error message:

```
pdf4tcllib::fonts::init -cid 1
```

16.3 The LaTeX Subset

```
set x [pdf4tcllib::math::renderLatex pdf x y formel \
    ?-size n? ?-font name?]
lassign [pdf4tcllib::math::measureLatex pdf formel ?-size n?] \
    breite oben unten
```

math::renderLatex sets the formula with its baseline at y and returns the x position after it. math::measureLatex draws nothing and returns the width, the height above and the depth below the baseline.

Input	Result
x^2, a_i, x_{i,j}^{2}	superscript and subscript, {} for more than one character
\frac{a}{b}	fraction with a bar, nested as well
\sqrt{x+1}	root with a bar above
\sum_{i=1}^{n}, \int_0^1, \prod	large operators with limits
\alpha … \omega, \Gamma … \Omega	Greek letters
\pm \cdot \le \ge \ne \infty …	symbols
\, \;	a small space

Not included: `\text`, `\mathrm`, `\left`/`\right`, `\quad`, environments, matrices. An unknown command is set as text, with its backslash, and reported on stderr:

```
pdf4tcllib::math: unknown symbol '\quad' rendered as literal text
```

16.3.1 Spaces Do Not Count

As in LaTeX, spaces in the input are skipped. Unlike LaTeX, however, `renderLatex` adds no space around = and + either: `a = b` becomes a=b, set tightly together. Where room is needed, `\,` goes in:

```
{E \, = \, mc^2}
```

The third displayed formula in the example shows the difference; it is set without `\,`.

16.3.2 The Minus Sign

`-b` sets a hyphen, not a minus. It is shorter and sits lower. For a real minus, write the character U+2212 into the formula yourself, for example with `[format %c 0x2212]`.

16.4 Inline and Displayed

```
# 160-formeln.tcl -- Formeln im Text und abgesetzt
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1      ;# Griechisch und Operatoren
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
set font [pdf4tcllib::fonts::fontSans]
set x0 [dict get $ctx left]
set y  [expr {[dict get $ctx top] + 20}]

# 1. Eine Formel mitten im Satz: Text, Formel, Text
$pdf setFont 11 $font
$pdf text "Die Energie ist " -x $x0 -y $y
set x [expr {$x0 + [$pdf getStringWidth "Die Energie ist "]}]
set x [pdf4tcllib::math::renderLatex $pdf $x $y {E \, = \, mc^2} \
    -size 11]
$pdf setFont 11 $font       ;# renderLatex verstellt die Schrift
$pdf text ", die Masse also " -x $x -y $y
set x [expr {$x + [$pdf getStringWidth ", die Masse also "]}]
set x [pdf4tcllib::math::renderLatex $pdf $x $y \
    {m \, = \, \frac{E}{c^2}} -size 11]
$pdf setFont 11 $font
$pdf text "." -x $x -y $y

# 2. Abgesetzte Formeln, mittig, mit Abstand nach ihrer Hoehe.
#    \, ist ein kleiner Abstand; Leerzeichen zaehlen nicht.
set w [dict get $ctx text_w]
set y [expr {$y + 30}]
set nr 0
foreach f {
    {x \, = \, \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}}
    {\sum_{i=1}^{n} i \, = \, \frac{n(n+1)}{2}}
    {\alpha + \beta + \gamma = \pi}
} {
    lassign [pdf4tcllib::math::measureLatex $pdf $f -size 14] \
        fb oben unten
    set y [expr {$y + $oben}]
    set xm [expr {$x0 + ($w - $fb) / 2.0}]
```

```
    pdf4tcllib::math::renderLatex $pdf $xm $y $f -size 14
    puts [format "Formel %d: Breite %5.1f  oben %4.1f  unten %4.1f" \
        [incr nr] $fb $oben $unten]
    set y [expr {$y + $unten + 16}]
}

# 3. Die eqn-Schreibweise: Zeichen durch Leerzeichen getrennt
set y [expr {$y + 10}]
pdf4tcllib::math::renderFormula $pdf $x0 $y \
    "H _ 2 O ~ + ~ CO _ 2" -size 12
set y [expr {$y + 24}]
pdf4tcllib::math::renderFormula $pdf $x0 $y \
    "SUM from i=0 to infty ~ a _ i x ^ i" -size 12

$pdf endPage
$pdf write -file 160-formeln.pdf
$pdf destroy
```

```
Formel 1: Breite  95.1  oben 23.4  unten 12.9
Formel 2: Breite  79.6  oben 17.1  unten 14.7
Formel 3: Breite  63.0  oben 10.1  unten  2.8
```

16.4.1 Inline

Text and formula alternate, and each piece starts where the previous one ended: for text via `getStringWidth`, for a formula via the return value of `renderLatex`.

After `renderLatex`, the **font is changed**: the size of the last superscript character stays set, in the example 7.7 instead of 11 points. That is why each formula is followed by a new `setFont` .

A fraction inside a sentence reaches above and below the line. `m = E/c²` at 11 points extends 14 points above and 14 points below the baseline. With the usual line spacing of 1.4, it reaches into the next line; for fractions in running text, `a/b` on one line is often the better choice.

`text::writeParagraph` knows nothing about formulas. A paragraph with formulas is set piece by piece, as in the example, and does not wrap by itself.

16.4.2 Displayed

For a formula on a line of its own, `measureLatex` measures first. The width gives the center, oben (above) says how far below the previous line the baseline must lie, `unten` (below) where the next line may start. This way the example calculates the spacing of each formula from its own height.

16.5 The eqn Notation

```
set x [pdf4tcllib::math::renderFormula pdf x y formel \
    ?-size n? ?-font name?]
```

math::renderFormula is the older notation, a port of Arjen Markus' "MathFormula" from the

Tcler's Wiki. Each element is separated by spaces:

Token	Meaning
^ x	x as superscript
_ x	x as subscript
~	a space that is kept
SUM, INT, PROD	large operators
from a, to b	limits after an operator
alpha, infty, pm ...	symbols, without a backslash

`from` and `to` are **always** limits; an arrow is called `rightarrow`. This notation has no fractions or roots with a bar. The limits above `SUM` are small and sit very close above the symbol (the ∞ in the example).

Afterwards, `renderFormula` resets the font to the base size, in the regular font, even if bold was set before.

For new documents, `renderLatex` is the better choice: it can do more, and more readers know the notation. The pdf4tcllib documentation says in the section on `renderFormula` that fractions and roots are not supported; that applies to `renderFormula` only.

16.6 Summary

- Without `fonts::init -cid 1`, formula symbols become?.
- `renderLatex` for superscript, subscript, fraction, root, sum; `measureLatex` measures beforehand.
- Spaces do not count, spacing with `\,`; minus is a hyphen.
- Set the font again after `renderLatex`.
- Unknown commands are set as text.

Next chapter: Treeview

17 Treeview

17.1 Introduction

Many Tk programs show their data in a `ttk::treeview`: warehouses with items, projects with tasks, folders with files. The request "that as a PDF" then means: exactly what is in the window. The package pdf4tcltreeview reads out the widget and draws it, either as a table with an indented tree column or as an outline.

It is a thin layer over `table::draw` (chapter Tables with Page Breaks). It needs Tk, because it reads a widget.

17.2 Table or Outline

A treeview is a table and a tree at the same time. In the PDF, one has to choose, and that is why

there are two procedures instead of one option, treeview::render and treeview::outline:

Procedure	draws	in the accessible PDF
treeview::render	a table, tree column indented	Table, TR, TD
treeview::outline	only the tree column, as a list	L, LI, LBody

Both return the y position after the last entry.

17.3 A Stock List

```
# 170-treeview.tcl -- ein ttk::treeview als Tabelle und als Gliederung
package require Tk
package require pdf4tcl
package require pdf4tcllib
package require pdf4tcltreeview

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]

# Das Widget: Baumspalte #0 plus zwei Wertespalten
ttk::treeview .tv -columns {menge wert} -show {tree headings}
.tv heading #0    -text "Lager / Artikel"
.tv heading menge -text "Menge"
.tv heading wert  -text "Wert €"
.tv column #0    -width 260
.tv column menge -width 80 -anchor e
.tv column wert  -width 100 -anchor e

foreach {lager offen artikel} {
    "Lager Nord" 1 {{"Schraube M6" 600 29.40} {"Schraube M8" 400 31.20}
                    {"Mutter M8" 900 18.00}}
    "Lager Süd"  1 {{"Dübel 8 mm" 1200 42.00}
                    {"Winkel 40 mm" 150 67.50}}
    "Außenlager" 0 {{"Palette leer" 34 0.00}}
} {
    set summe 0.0
    foreach a $artikel { set summe [expr {$summe + [lindex $a 2]}] }
    set k [.tv insert {} end -text $lager -open $offen \
        -values [list "" [format %.2f $summe]]]
    foreach a $artikel {
        lassign $a name menge wert
        .tv insert $k end -text $name \
            -values [list $menge [format %.2f $wert]]
    }
}
pack .tv
```

```
update idletasks

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]

# Seite 1: wie auf dem Bildschirm -- das Aussenlager ist zugeklappt
$pdf startPage
set x [dict get $ctx left]
set y [pdf4tcllib::treeview::render $pdf .tv $x [dict get $ctx top] \
    -maxwidth 300 -ctx $ctx]
puts "Tabelle wie angezeigt endet bei y = [format %.1f $y]"
# Dasselbe mit allen Zweigen
set y [pdf4tcllib::treeview::render $pdf .tv $x [expr {$y + 30}] \
    -maxwidth 300 -ctx $ctx -all 1]
puts "Tabelle mit -all 1 endet bei y = [format %.1f $y]"
$pdf endPage

# Seite 2: nur die Baumspalte als eingerueckte Liste, mit einem
# Lesezeichen je Knoten
$pdf startPage
set y [pdf4tcllib::treeview::outline $pdf .tv $x [dict get $ctx top] \
    -all 1]
puts "Gliederung endet bei y = [format %.1f $y]"
$pdf endPage

$pdf write -file 170-treeview.pdf
$pdf destroy
exit                        ;# sonst wartet Tk auf Ereignisse
```

```
Tabelle wie angezeigt endet bei y = 200.7
Tabelle mit -all 1 endet bei y = 390.7
Gliederung endet bei y = 182.7
```

Page 1 as text:

```
Lager / Artikel   Menge   Wert €
Lager Nord                 78.60
  Schraube M6       600    29.40
  Schraube M8       400    31.20
  Mutter M8         900    18.00
Lager Süd                 109.50
  Dübel 8 mm       1200    42.00
  Winkel 40 mm      150    67.50
Außenlager                  0.00
```

The second table additionally has "Palette leer" (empty pallet) below.

17.3.1 The Script Needs an exit

With `package require Tk`, `tclsh` waits for events after the last command instead of terminating. The `exit` at the end is therefore required. The window does not have to be visible,

but Tk needs a display. On a server without a screen, the script runs with `xvfb-run tclsh 170-treeview.tcl`.

The package documentation recommends the `update idletasks` before the export. In the example it changes nothing, because all columns have a fixed width; for columns whose width Tk only computes when they are displayed, it is the safe way.

17.4 render

```
set y [pdf4tcllib::treeview::render pdf tv x y ?option wert ...?]
```

Option	Default	Meaning
-maxwidth	480	width of the table
-fontsize	9	font size
-all	0	1: collapsed branches as well
-tree	1	0: do not indent the tree column
-indentW	12	indentation per level
-zebra, -border	1, 1	stripes, lines
-ctx	–	page context; switches on page breaking
-font, -boldfont	–	fonts instead of those from fonts::init
-summary	–	summary of the table (accessibility)

What is collapsed stays collapsed. The PDF shows what is visible in the window; in the example, the first table lacks the contents of the "Außenlager" (outdoor storage). `-all 1` takes everything.

The column widths come from the widget in pixels and are scaled to `-maxwidth`; the proportions are kept. The alignment comes from the column's `-anchor`: `e` becomes right-aligned.

What `render` does not read: colors and fonts of individual entries (`-tags` with `tag configure`), images. The nodes "Lager Nord" etc. appear in the same font as the items. `render` does not pass `-rowstyles` through to `table::draw`; to get bold group rows, read out the widget yourself and call `table::draw` directly.

17.4.1 Page Breaks

With `-ctx`, the table breaks at the bottom margin, but with the built-in page break of `table::draw`: `- 1 -` at the bottom right, no header or footer on the following page. `render` has no `-pagebreakcmd`; passing it gives an error with the list of known options.

17.5 outline

```
set y [pdf4tcllib::treeview::outline pdf tv x y ?option wert ...?]
```

Option	Default	Meaning
-fontsize	10	font size
-indentW	14	indentation per level
-lineH	0	line spacing; 0 means 1.4 × size
-all	0	collapsed branches as well
-bookmarks	1	one bookmark per node
-font	–	font

A node without text takes the first value.

17.5.1 Bookmarks

`-bookmarks 1` creates a bookmark for each node, at the depth in the tree: "Schraube M6" sits below "Lager Nord". In the example this is the default. Up to pdf4tcltreeview 0.1, none were created: `outline` called `bookmarkAdd` in a form that pdf4tcl does not understand and silently caught the error. With 0.1, create them yourself:

```
proc lesezeichen {tv knoten ebene} {
    global pdf
    foreach k [$tv children $knoten] {
        $pdf bookmarkAdd -title [$tv item $k -text] -level $ebene
        lesezeichen $tv $k [expr {$ebene + 1}]
    }
}
lesezeichen .tv {} 0
```

The bookmarks point to the top of the page on which they are created; `bookmarkAdd` knows no position within the page.

17.6 Both in One Document

Table and outline describe the same data. A screen reader reads both aloud if both are present. If both are needed (overview at the front, table at the back), put them on separate pages and say in a heading what follows.

17.7 Summary

- `treeview::render` for the table, `treeview::outline` for the outline; both need Tk.
- The script ends with `exit`; without a screen, use `xvfb-run`.
- Collapsed branches are missing, except with `-all 1`.
- `outline` creates one bookmark per node (since 0.1.1).
- Page breaking only the built-in way, without a header.

Next chapter: Accessible PDFs

18 Accessible PDFs

18.1 Introduction

A screen reader does not see a page but a tree: heading, paragraph, table with rows and cells, figure with a description. If this structure tree is not in the PDF, the program reads the page in the order in which the texts lie in the content stream, page number, header and table lines included. Public authorities and many companies therefore require accessible PDFs according to PDF/UA (ISO 14289).

pdf4tcl writes the structure tree when it is switched on. pdf4tcllib makes sure that its building blocks fill it correctly: a table becomes `Table` with `TR` and `TD`, a header becomes decoration. Whatever is drawn by hand has to be classified by hand. This chapter shows both with a report.

18.2 What a Document Needs for PDF/UA

Requirement	in pdf4tcl
structure tree	$pdf tagged 1
language of the document	tagged 1 -lang de-DE
title, shown in the window instead of the file name	metadata -title ..., viewerPreferences -displaydoctitle 1
embedded fonts	fonts::init; none of the 14 standard fonts
all content in an element or marked as decoration	pdf4tcllib building blocks, own drawing with tag::
alternate text for figures	-alt
identification as PDF/UA	tagged 1 -ua 1

`-ua 1` only writes the claim into the metadata that the document conforms to PDF/UA-1. pdf4tcl cannot check this. A document that makes the claim and does not live up to it is worse than one that claims nothing; that is why `-ua` is not the default.

`-displaydoctitle` is not the default either, and `-ua 1` does not set it. PDF/UA requires it, though: without it, the viewer shows the file name instead of the title.

18.3 A Report with a Structure Tree

```
# 180-barrierefrei.tcl -- ein Bericht mit Strukturbaum fuer PDF/UA
package require pdf4tcl
package require pdf4tcllib
package require pdf4tclchart

pdf4tcllib::fonts::init -cid 1           ;# eingebettete Schrift
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
```

```
# Strukturbaum an, Sprache, Anspruch auf PDF/UA-1, Titel
$pdf tagged 1 -lang de-DE -ua 1
$pdf metadata -title "Lagerbericht Oktober" -author "Lager Nord"
$pdf viewerPreferences -displaydoctitle 1      ;# Titel statt Dateiname
set ::pdf4tcl::warnings {}

$pdf startPage
set x [dict get $ctx left]
set w [dict get $ctx text_w]
pdf4tcllib::page::header $pdf $ctx "Lagerbericht Oktober"

# Ueberschrift: writeParagraph mit dem Typ H1 statt des Vorgabe-P
set y [pdf4tcllib::text::writeParagraph $pdf "Bestand am Monatsende" \
    $x [expr {[dict get $ctx top] + 20}] $w 16 left H1]
set y [pdf4tcllib::text::writeParagraph $pdf "Der Bestand wurde am\
    Monatsende gezählt. Die Tabelle nennt die Abweichungen je Regal." \
    $x [expr {$y + 6}] $w 11]

# Eine Tabelle: Table, TR, TH, TD von selbst
set y [pdf4tcllib::table::draw $pdf $x [expr {$y + 10}] {
    {-header Regal -width 80}
    {-header Soll -width 60 -align right}
    {-header Ist -width 60 -align right}
} {{A-01 120 118} {A-02 80 80} {B-01 45 47}} \
    -summary "Soll und Ist je Regal"]

# Ein Diagramm: ein Element Figure mit Ersatztext
set y [pdf4tcllib::chart::bar $pdf $x [expr {$y + 20}] 300 150 \
    {A-01 -2 A-02 0 B-01 2} -title "Abweichung je Regal" \
    -alt "Balkendiagramm: A-01 minus 2, A-02 null, B-01 plus 2"]

# Selbst gezeichnet: Text als P, die Linie als Schmuck
pdf4tcllib::tag::begin $pdf P
$pdf setFont 9 [pdf4tcllib::fonts::fontSans]
$pdf text "Gezählt von: M. Albers" -x $x -y [expr {$y + 20}]
pdf4tcllib::tag::end $pdf
pdf4tcllib::tag::artifact $pdf -type Layout
$pdf line $x [expr {$y + 26}] [expr {$x + $w}] [expr {$y + 26}]
pdf4tcllib::tag::artifactEnd $pdf

pdf4tcllib::page::footer $pdf $ctx "Intern" 1
puts "ohne Element: [$pdf getUntaggedCount]"
$pdf endPage
$pdf write -file 180-barrierefrei.pdf
$pdf destroy
foreach m $::pdf4tcl::warnings { puts "Warnung: $m" }
```

```
ohne Element: 0
```

Up to pdf4tclchart 0.1.1, a warning appeared below this: "the Figure element is empty". Back then the chart was marked entirely as an artifact, and the `Figure` had no content. Since 0.1.2, the drawing is its content, and the warning is gone.

The structure tree of the result, read out with a PDF tool:

```
Document
  H1          [Inhalt]
  P           [Inhalt]
  Table       Summary="Soll und Ist je Regal"
    TR
      TH  TH  TH        Scope=Column
    TR
      TD  TD  TD
    TR  ...
  Figure      Alt="Balkendiagramm: A-01 minus 2, A-02 null,
                   B-01 plus 2"  [Inhalt]
  P           [Inhalt]
```

Header, footer and the line below "Gezählt von" ("counted by") are not in the tree. They are marked as an artifact: decoration that a screen reader skips.

The table lines, on the other hand, hang as content on the `Table` element. The pdf4tcllib documentation calls them artifacts; in the content stream they appear with a content number below `Table`. A screen reader does not read lines aloud, and `getUntaggedCount` does not count them. Whether a checker objects to content directly below `Table` has not been tested here.

18.4 What the Building Blocks Do Themselves

Building block	in the structure tree
text::writeParagraph	P, or the type in the last argument
table::draw	Table, TR, TH with Scope=Column, TD; -summary
chart::bar, line, pie	Figure with alternate text from -alt or -title; bars and axes are its content
image::place, gallery	Figure with -alt; caption as Caption
toc::heading, toc::render	H1 to H6; TOC and TOCI
form::labelField, row	Form with label and field
labels::render	one Sect per label
drawing::callout	text as P, box and pointer as artifacts

Building block	in the structure tree
page::header, footer, number, drawing::watermark	artifact (pagination)

If `tagged` is off, the building blocks do none of this. The same code then produces an ordinary PDF.

18.4.1 Headings

`writeParagraph` sets `P` if nothing else is given. For a heading, the type goes in as the eighth argument, with size and alignment before it:

```
pdf4tcllib::text::writeParagraph $pdf "Bestand" $x $y $w 16 left H1
```

`writeParagraph` always sets the regular font; a `setFont` with bold before it has no effect. The heading in the example is therefore only larger, not bold. For a bold heading with a structure element, draw it yourself (section Classifying Your Own Drawing).

The levels must not skip: after `H1` comes `H2`, not `H3` straight away. pdf4tcl does not check this.

18.4.2 Charts

A chart becomes a `Figure` with alternate text; bars, axes and labels are its content. A screen reader speaks the alternate text, not the individual strokes. The title is usually too little as alternate text: "Abweichung je Regal" ("deviation per shelf") does not say what the deviation looks like. In the example, the values are in the alternate text.

A reader gets nothing from the chart beyond the alternate text. If the numbers matter, they belong in a table as well.

18.4.3 Tables

`-summary` describes the table in one sentence before the cells come. The header row becomes `TH` with `Scope=Column`; this way a reader knows for each cell which column it belongs to.

18.5 Classifying Your Own Drawing

Whatever is drawn directly with pdf4tcl belongs to nothing until it is classified. For this there are four procedures in the namespace tag:

Procedure	does
tag::begin pdf typ ?-alt text?	open an element (P, H2, Figure …)
tag::end pdf	close it
tag::artifact pdf ?-type Layout|Pagination?	start decoration
tag::artifactEnd pdf	end decoration

A bold heading, set by hand:

```
pdf4tcllib::tag::begin $pdf H2
$pdf setFont 13 [pdf4tcllib::fonts::fontSansBold]
$pdf text "Abweichungen" -x $x -y $y
pdf4tcllib::tag::end $pdf
```

All four do nothing when `tagged` is off. They can therefore stay in code that also produces ordinary PDFs.

`Layout` is decoration on the page (lines, areas, frames); `Pagination` belongs to the page design (header, page number, watermark).

18.5.1 getUntaggedCount

`$pdf getUntaggedCount` counts the drawing commands that are neither in an element nor in an artifact. Zero is the goal. Without the four `tag::` lines in the example, it would be 2: the text "Gezählt von" and the line. pdf4tcl then also writes a warning to `$::pdf4tcl::warnings` when saving.

Only what is drawn is counted. A color or font outside an element does no harm.

18.6 Checking

Zero in `getUntaggedCount` only means that everything is classified. Whether the classification is right, the number does not say. Three levels:

1. **getUntaggedCount and $::pdf4tcl::warnings** in the program, on every run. The example prints both.
2. **A checker** for the standard. veraPDF checks PDF/UA-1:

   `qpdf --check` only checks whether the file is cleanly built, not the standard.

   ```
   verapdf -f ua1 bericht.pdf
   ```

3. **A person with a screen reader** (NVDA, JAWS, Orca). A document can meet every rule and still be read aloud in an order that makes no sense.

18.7 Summary

- `tagged 1 -lang de-DE`, title with `metadata -title` and `viewerPreferences -displaydoctitle 1`, embedded font.
- The building blocks fill the structure tree themselves; headings need the type as an argument.
- Own drawing with `tag::begin/end` or as an artifact.
- `getUntaggedCount` must be 0; after that, veraPDF and a person.
- `-ua 1` is a claim, not a check.

Next chapter: Saving the Graphics State

19 Saving the Graphics State

19.1 Introduction

Every building block sets font and colors the way it needs them and leaves them that way afterwards. Anyone who set 20 point bold red before a table goes on writing in 9 point regular black afterwards, without noticing. In short scripts this shows at once; in a report built from many procedures, finding the cause takes a long time.

state::with holds the state across a block and restores it afterwards.

19.2 What Gets Lost

```
# 190-zustand.tcl -- Schrift und Farbe ueber einen Baustein retten
package require pdf4tcl
package require pdf4tcllib

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
set x [dict get $ctx left]
set fett [pdf4tcllib::fonts::fontSansBold]
set cols {{-header Regal} {-header Bestand -align right}}
set rows {{A-01 118} {A-02 80}}

# pdf4tcl hat keine Abfrage fuer die Schrift; die Breite eines
# Probetextes zeigt, welche Groesse gerade eingestellt ist
proc probe {} { format %.1f [$::pdf getStringWidth "Probe"] }

# 1. Ohne Schutz: danach ist die Schrift der Tabelle eingestellt
$pdf setFont 20 $fett
$pdf setFillColor 0.8 0 0
puts "vorher:          [probe]"
set y [pdf4tcllib::table::draw $pdf $x 80 $cols $rows]
puts "nach table:      [probe]"
$pdf text "Nachher (ohne Schutz)" -x $x -y [expr {$y + 30}]

# 2. Mit state::with: Schrift und Farbe bleiben erhalten
$pdf setFont 20 $fett
$pdf setFillColor 0.8 0 0
set y [pdf4tcllib::state::with $pdf [list 20 $fett] {0.8 0 0} {
    pdf4tcllib::table::draw $pdf $x [expr {$y + 60}] $cols $rows
}]
puts "nach state::with: [probe]"
$pdf text "Nachher (mit state::with)" -x $x -y [expr {$y + 30}]

$pdf endPage
$pdf write -file 190-zustand.pdf
$pdf destroy
```

```
vorher:          59.5
nach table:      22.9
nach state::with: 59.5
```

pdf4tcl cannot tell which font is currently set; there is `setFont`, but no `getFont`. The example therefore measures the width of the word "Probe" (sample): 59.5 points in 20 point bold, 22.9 points in the font that `table::draw` last wrote with. In the PDF the first "Nachher" (after) line is small and black, the second one large and red.

19.3 state::with

```
set ergebnis [pdf4tcllib::state::with pdf {größe schrift} {r g b} \
    skript]
```

The script runs in the caller's scope. Afterwards font and fill color are the given ones again, even if the script throws an error. The return value is whatever the script returns; in the example, the y of the table.

Because pdf4tcl does not hand out the state, it has to be **given explicitly**. `state::with` restores what it is told, not what was set before. Passing `{20 Helvetica}` while 11 point Sans was actually set gives 20 point Helvetica afterwards.

Whatever is left out (an empty `{}` instead of font or color) is not touched. `state` queries line spacing and kerning from pdf4tcl itself and restores them as well.

19.3.1 By Hand

```
set z [pdf4tcllib::state::save $pdf 20 $fett {0.8 0 0}]
pdf4tcllib::table::draw $pdf ...
pdf4tcllib::state::restore $pdf $z
```

`save` also takes the stroke color as a fifth argument. It returns a dict that `restore` puts back in place. An error between the two calls leaves the state changed; `with` is therefore usually the better choice.

19.4 Why Not gsave and grestore

PDF has its own pair for saving the state, `q` and `Q`, in pdf4tcl `gsave` and `grestore`. Within a page it is the right tool, for example to lift a clipping path again. Across a page break it does not work: each page has its own content stream, and a `gsave` on page 1 with the `grestore` on page 2 gives two unbalanced streams. Neither pdf4tcl nor qpdf reports this. A table with `-ctx`, however, breaks pages in exactly this way.

Within a page, `grestore` also restores the font that pdf4tcl keeps for measuring: after `gsave`, `setFont 9` and `grestore`, "Probe" measures 59.5 points again.

19.5 Summary

- Building blocks leave their font and color behind.
- `state::with pdf {größe schrift} {r g b} skript` restores them afterwards, even on an error.
- Give the state explicitly; pdf4tcl cannot supply it.
- `gsave`/`grestore` only within a page.

Next chapter: Measuring and Stacking

20 Measuring and Stacking

20.1 Introduction

Every report contains the same loop: draw a block, take the returned `y`, add a gap, draw the next block. And before a table comes the question: does it still fit on this page, or does it belong entirely on the next one? The building blocks of pdf4tcllib do not answer that; they draw and say afterwards where they stopped.

Two namespaces help: measure measures a block before it is drawn, and layout::stack places blocks one below the other and starts a new page when a block no longer fits completely.

20.2 Measuring by Drawing

```
set m [pdf4tcllib::measure::block varName skript ?option wert ...?]
```

measure::block really draws the block, but into a throwaway document that is never written. What is measured is therefore exactly what comes out later. Inside the script, the throwaway document is held in the variable whose name is passed (`mp` in the example). The result is a dict:

Key	Content
height	height of the block, from its return value
pages	pages it ran across
overflow	1 if it ran off the bottom of the sheet
value	what the script returned
sheet	height of the sheet

Options: `-paper` (default `a4`), `-orient`, `-landscape`, `-margin`, `-y`. The height is the distance between `-y` (default 0) and the return value; the block should therefore start at y = 0 or at the `y` given with `-y`. If the script does not return a number, `height` is empty.

Fonts loaded by `fonts::init` are known to the throwaway document as well; they apply to all documents of a process.

20.2.1 pages and overflow

The page count only goes up if the block breaks pages itself, for example `table::draw` with `-ctx`. Without `-ctx`, `table::draw` draws past the edge of the page, and `pages` stays 1. Measure the block the same way it will be drawn later.

`writeParagraph` never breaks pages. A long paragraph runs off the bottom of the sheet, `pages` stays 1, and only `overflow` says that something is wrong. In the example the long paragraph is 896 points high, the sheet only 842.

measure::fits `platz varName skript` sums this up: 1 if the block is at most `platz` high, stays on one page and does not overflow. measure::height returns only the height.

20.3 The Example

```
# 200-messen-und-stapeln.tcl -- Hoehe vorab messen, Bloecke stapeln
package require pdf4tcl
package require pdf4tcllib
package require pdf4tclchart

pdf4tcllib::fonts::init -cid 1
set ctx [pdf4tcllib::page::context a4 -margin 20]
set x [dict get $ctx left]
set w [dict get $ctx text_w]

set text "Der Bestand wurde am Monatsende gezählt und mit der Buchung\
    abgeglichen. Abweichungen unter zwei Stück sind ausgebucht."
set cols {
    {-header Regal -width 120}
    {-header Soll  -width 60 -align right}
    {-header Ist   -width 60 -align right}
}
set rows {}
for {set i 1} {$i <= 30} {incr i} {
    set soll [expr {$i * 7}]
    lappend rows [list "Regal $i" $soll [expr {$soll - $i % 3}]]
}

# 1. Messen: wie hoch wird die Tabelle, passt der Absatz?
set m [pdf4tcllib::measure::block mp {
    pdf4tcllib::table::draw $mp 0 0 $cols $rows -maxwidth $w
}]
puts "Tabelle: Höhe [format %.1f [dict get $m height]],\
    Seiten [dict get $m pages], Überlauf [dict get $m overflow]"
set lang [string repeat "$text " 40]
set m [pdf4tcllib::measure::block mp {
    pdf4tcllib::text::writeParagraph $mp $lang 0 0 $w 11
}]
puts "Langer Absatz: Höhe [format %.1f [dict get $m height]],\
    Seiten [dict get $m pages], Überlauf [dict get $m overflow]"
puts "passt in 200 pt: [pdf4tcllib::measure::fits 200 mp {
    pdf4tcllib::text::writeParagraph $mp $text 0 0 $w 11
}]"

# 2. Stapeln: Bloecke untereinander, Tabelle und Diagramm am Stueck
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
proc kopf {} {
    global pdf ctx
    pdf4tcllib::page::header $pdf $ctx "Lagerbericht"
}
$pdf startPage
kopf
```

```
# Die Linie der Kopfzeile liegt 2 pt unter top, im Satzspiegel. stack
# beginnt jede Seite bei SY (= top); also SY tiefer, SH kuerzer.
set oben [expr {[dict get $ctx top] + 12}]
set sh   [expr {[dict get $ctx SH] - 12}]
set sctx [dict replace $ctx SY $oben SH $sh]

set absatz  {pdf4tcllib::text::writeParagraph $bpdf $text \
                 $bx $by $bw 11}
set balken  {pdf4tcllib::chart::bar $bpdf $bx $by $bw 200 \
                 {Nord 120 Süd 95 West 60} -title "Bestand je Lager"}
set tabelle {pdf4tcllib::table::draw $bpdf $bx $by $cols $rows \
                 -maxwidth $bw}
set ende    {pdf4tcllib::text::writeParagraph $bpdf "Ende." \
                 $bx $by $bw}
set blocks [list \
    [list script $absatz] \
    [list script $balken keep 1] \
    [list script $absatz] \
    [list script $tabelle keep 1] \
    {gap 10} \
    [list script $ende]]
set r [pdf4tcllib::layout::stack $pdf $x $oben $w $blocks \
    -ctx $sctx -gap 14 -font [list 11 [pdf4tcllib::fonts::fontSans]] \
    -pagecmd kopf]
puts "Seitenwechsel: [dict get $r pages], Ende bei y =\
    [format %.1f [dict get $r y]]"
foreach b [dict get $r boxes] { puts "  [lmap v $b {format %.0f $v}]" }
$pdf endPage
$pdf write -file 200-messen-und-stapeln.pdf
$pdf destroy
```

```
Tabelle: Höhe 502.2, Seiten 1, Überlauf 0
Langer Absatz: Höhe 896.0, Seiten 1, Überlauf 1
passt in 200 pt: 1
Seitenwechsel: 1, Ende bei y = 625.9
  57 69 482 32
  57 115 482 200
  57 329 482 32
  57 69 482 502
  57 595 482 17
```

Page 1 holds paragraph, chart and paragraph. The table (502 points) no longer fits into the type area from y = 375; with `keep 1` it moves entirely to page 2, with a header. After it comes "Ende." (end).

20.4 layout::stack

```
set r [pdf4tcllib::layout::stack pdf x y breite blöcke \
    ?option wert ...?]
```

Each entry in `blöcke` is one of two kinds:

Entry	Meaning
script skript ?keep 1?	a block
gap n	n points of space before the next block

Four variables are available in the script: `$bpdf`, `$bx`, `$by`, `$bw`, that is the document, the top left corner and the width. Like every building block, the script returns the `y` below it (or an area `{x y b h}`).

In the script, $bpdf, not $pdf. While measuring, `$bpdf` is the throwaway document. A script that uses `$pdf` draws into the real document while measuring, and the block appears twice.

The script runs in the caller's scope and sees its variables (`$text`, `$cols`, `$rows`). That is why the scripts in the example are in curly braces: the variables are substituted only when the script runs. A script spanning several lines needs `\` at the end of the line, like any other Tcl command.

Option	Meaning
-ctx	page context; without it there is never a page break
-gap	space after each block
-font {größe schrift}, -fill {r g b}	set before each block, restored afterwards
-pagecmd	runs after each page break, in the caller's scope
-paper	paper used for measuring

The result is a dict with `y` (below the last block), `pages` (number of page breaks that `stack` made itself) and `boxes`, `{x y b h}` per block.

20.4.1 What keep Does and What It Does Not

Only blocks with `keep 1` are measured. If such a block no longer fits above the bottom margin, `stack` ends the page, starts a new one, calls `-pagecmd` and places the block at the top.

Blocks **without** `keep` are neither measured nor checked. A paragraph without `keep` at the end of the page runs past the bottom margin, even with `-ctx`. That is why `keep 1` pays off for short blocks as well, when it is not known where they will land.

A block taller than a whole page does not fit on the new page either. `stack` places it there anyway; the block then has to break pages itself (a table with `-ctx`). The pages it starts in doing so are not counted in `pages`, and `boxes` does not say which page a block is on.

20.4.2 The Header and the Top Margin

`stack` starts each new page at `SY` from the context, which is `top`. `page::header`, however, draws its rule 2 points **below** `top`, that is inside the type area. A paragraph starting exactly at `top` gets struck through by the rule. The example therefore gives `stack` a context in which `SY` is 12 points lower and `SH` correspondingly shorter:

```
set oben [expr {[dict get $ctx top] + 12}]
set sh   [expr {[dict get $ctx SH] - 12}]
set sctx [dict replace $ctx SY $oben SH $sh]
```

SY and SH are the keys that `stack` reads; the other building blocks use `top` and `bottom`. The modified copy therefore applies only to `stack`.

20.5 Summary

- `measure::block` draws as a trial and returns height, pages and overflow; `overflow` sees what `pages` does not.
- `layout::stack` places blocks one below the other; in the scripts use `$bpdf`, `$bx`, `$by`, `$bw`.
- Only `keep 1` blocks are measured and moved to the next page when needed.
- For room below the header, adjust SY and SH in the context for `stack`.

Next chapter: Case Study: Warehouse Report

21 Case Study: Warehouse Report

21.1 Introduction

A report on one month of stock movements in a warehouse, from the raw data to the finished PDF: subtotals and a crosstab, a chart, a table across several pages with headers and footers, a table of contents with real page numbers, and a structure tree. Each part comes from an earlier chapter; here they come together in one script, and the seams show what matters.

Part	Chapter
table::group, table::pivot	Group Totals and Crosstabs
chart::bar	Charts
table::draw with -ctx and -pagebreakcmd	Tables with Page Breaks
page::header, page::footer	Headers, Footers, Page Numbers
toc::document, toc::heading	Table of Contents
tagged, metadata	Accessible PDFs

21.2 The Script

```
# 210-fallstudie.tcl -- Lagerbericht: Tabellen, Diagramm, Verzeichnis
package require pdf4tcl
package require pdf4tcllib
package require pdf4tclchart
package require pdf4tcltoc

pdf4tcllib::fonts::init -cid 1
```

```
# --- 1. Daten: vor dem Inhalt laden, er laeuft zweimal
# Bewegungen {Lager Gruppe Artikel Menge Wert}, nach Lager sortiert
set bewegungen {}
set nr 0
foreach lager {Nord Süd West} {
    incr nr
    for {set i 1} {$i <= 22} {incr i} {
        set gruppe [lindex {Schrauben Dübel Werkzeug} [expr {$i % 3}]]
        set menge  [expr {($i * 7 + $nr * 11) % 40 - 8}]
        set wert   [format %.2f [expr {$menge * (1.5 + $i % 5)}]]
        lappend bewegungen [list $lager $gruppe \
            [format "%s-%03d" [string index $lager 0] $i] $menge $wert]
    }
}
set g [pdf4tcllib::table::group $bewegungen -by 0 -sum {3 4} \
    -label "Summe %s" -totallabel "Gesamt" -blank 1]
set p [pdf4tcllib::table::pivot $bewegungen -row 0 -col 1 -value 4 \
    -rowlabel Lager -colorder {Schrauben Dübel Werkzeug}]

# Wert je Lager fuer das Diagramm: die Zeile "Gesamt" der Kreuztabelle
# hat die Spaltensummen, die letzte Spalte jeder Zeile die Zeilensumme
set balken {}
foreach z [lrange [dict get $p rows] 0 end-1] {
    lappend balken [lindex $z 0] [lindex $z end]
}

set stile {}
foreach i [dict get $g subtotals] { lappend stile $i {-font bold} }
lappend stile [dict get $g total] {-font bold -bg {0.85 0.85 0.85}}

proc betrag {v} {
    if {[string is double -strict $v]} { return [format %.2f $v] }
    return $v
}

# --- 2. Seitenwechsel: Fuss auf die alte, Kopf auf die neue Seite
# Laeuft im globalen Bereich; dort ist pdf im ersten Durchlauf das
# Wegwerf-Dokument, im zweiten das echte.
proc seitenwechsel {} {
    global pdf ctx
    pdf4tcllib::page::footer $pdf $ctx "Lagerbericht Oktober" \
        [$pdf currentPage]
    $pdf endPage
    $pdf startPage
    pdf4tcllib::page::header $pdf $ctx "Lagerbericht Oktober"
    return [expr {[dict get $ctx top] + 10}]
```

```
}

# --- 3. Der Inhalt als Skript
set inhalt {
    set x [dict get $ctx left]
    set w [dict get $ctx text_w]

    $pdf startPage
    set y [dict get $ctx top]
    pdf4tcllib::toc::heading $pdf $ctx y 1 "Überblick"
    set y [pdf4tcllib::text::writeParagraph $pdf "Bewegungen der drei\
        Lager im Oktober, bewertet zum Einstandspreis. Negative Mengen\
        sind Abgänge." $x $y $w 11]
    set y [pdf4tcllib::chart::bar $pdf $x [expr {$y + 10}] $w 160 \
        $balken -title "Wert je Lager (€)" -values 1 -format %.0f \
        -alt "Balkendiagramm: Wert der Bewegungen je Lager"]

    set y [expr {$y + 16}]
    pdf4tcllib::toc::heading $pdf $ctx y 2 "Lager und Warengruppe"
    set kopf [lmap h [dict get $p header] {
        set a right
        if {$h eq "Lager"} { set a left }
        list -header $h -width 90 -align $a
    }]
    set y [pdf4tcllib::table::draw $pdf $x $y $kopf \
        [lmap z [dict get $p rows] { lmap v $z { betrag $v } }] \
        -rowstyles [list [dict get $p total] {-font bold}] \
        -summary "Wert je Lager und Warengruppe"]
    pdf4tcllib::page::footer $pdf $ctx "Lagerbericht Oktober" \
        [$pdf currentPage]
    $pdf endPage

    $pdf startPage
    pdf4tcllib::page::header $pdf $ctx "Lagerbericht Oktober"
    set y [expr {[dict get $ctx top] + 10}]
    pdf4tcllib::toc::heading $pdf $ctx y 1 "Bewegungen"
    set y [pdf4tcllib::table::draw $pdf $x $y {
        {-header Lager   -width 50}
        {-header Gruppe  -width 80}
        {-header Artikel -width 60}
        {-header Menge   -width 50 -align right}
        {-header Wert    -width 70 -align right}
    } [lmap z [dict get $g rows] {
        lset z end [betrag [lindex $z end]]
    }] \
        -ctx $ctx -zebra 1 -rowstyles $stile \
        -pagebreakcmd seitenwechsel -summary "Bewegungen je Lager"]
```

```
    pdf4tcllib::page::footer $pdf $ctx "Lagerbericht Oktober" \
        [$pdf currentPage]
    $pdf endPage
}

# --- 4. Dokument: Strukturbaum, Verzeichnis, zwei Durchlaeufe
set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf tagged 1 -lang de-DE
$pdf metadata -title "Lagerbericht Oktober" -author "Lagerverwaltung"
$pdf viewerPreferences -displaydoctitle 1
set ::pdf4tcl::warnings {}
set r [pdf4tcllib::toc::document $pdf a4 $inhalt -title "Inhalt"]

puts "Verzeichnis: [dict get $r tocPages] Seite(n),\
    Inhalt: [dict get $r contentPages] Seiten"
foreach e [dict get $r entries] { puts "  $e" }
puts "ohne Element: [$pdf getUntaggedCount]"
$pdf write -file 210-fallstudie.pdf
$pdf destroy
foreach m $::pdf4tcl::warnings { puts "Warnung: $m" }
```

```
Verzeichnis: 1 Seite(n), Inhalt: 3 Seiten
  1 Überblick 2
  2 {Lager und Warengruppe} 2
  1 Bewegungen 3
ohne Element: 0
```

The PDF has four pages:

Page	Content
1	table of contents "Inhalt" (contents) with three entries
2	"Überblick" (overview): paragraph, bar chart, crosstab
3	"Bewegungen" (movements): header, table up to "S-018"
4	header, continuation of the table with the column header row repeated, "Summe West" (subtotal) and "Gesamt" (grand total)

The script consists of four parts, numbered in the comments.

21.3 1. The Data First

`toc::document` runs the content twice (Table of Contents). Everything that may happen only once therefore comes before it: creating, grouping and reshaping the data. In the report of a real program, the database query would go here; a second read from the same channel would find it empty.

One list yields three views:

- `table::group` inserts a subtotal per warehouse and the grand total at the end, for the long

table. The rows must be sorted by warehouse; `group` does not sort.
- `table::pivot` turns it into the crosstab of warehouse by product group. `-colorder` fixes the columns; otherwise they would be in alphabetical order.
- The chart needs pairs of name and value. The crosstab already supplies them: the last column of each row is the row total, and the last row ("Gesamt") is left out.

The totals from `group` and `pivot` are raw Tcl numbers. `betrag` formats them only when drawing, so that the data itself stays numeric.

21.4 2. The Page Break

`-pagebreakcmd` runs in the global scope (Tables with Page Breaks). The procedure `seitenwechsel` therefore gets `pdf` and `ctx` with `global`. This works here because `toc::document` sets the variable `pdf` in the caller's scope: in the first pass it holds the throwaway document, in the second the real one. A procedure that kept the document in a variable of its own instead would write into the wrong one in the first pass.

The page number in the footer is `[$pdf currentPage]`. In the first pass the table of contents page is missing, so the numbers there are one too small. That does no harm: the footer does not shift anything, and only the second pass is printed. There the table of contents is counted, and the footer says "Seite 2" where the table of contents says "Überblick ... 2".

21.5 3. The Content

The script starts and ends its pages itself. The first content page has no header, only the footer; from the second page on the script sets both, and on the continuation pages of the table `seitenwechsel` does it.

Two gaps are set by hand: 10 points before the chart and 16 points after it. `chart::bar` returns the bottom edge of its box, and `toc::heading` puts the heading exactly there. Without the gap, "Lager und Warengruppe" (warehouse and product group) sticks to the axis labels.

The row "Summe Nord" starts with a cell merged across three columns. Since pdf4tcllib 0.6.7, `table::draw` no longer draws a vertical rule through it; before that, the row read "Summe No|rd".

21.6 4. The Document

Three lines make the report accessible:

- `tagged 1 -lang de-DE` turns on the structure tree and names the language;
- `metadata -title` gives the title;
- `viewerPreferences -displaydoctitle 1` makes the viewer show the title instead of the file name.

The building blocks do the rest: `toc::heading` sets `H1` and `H2`, `text::writeParagraph` a `P`, `table::draw` the tables with header cells, `chart::bar` a `Figure` with alternate text, and headers and footers are artifacts. `getUntaggedCount` reports 0. The throwaway document of the first pass has no structure tree; there the building blocks do none of this.

`::pdf4tcl::warnings` stays empty. With pdf4tclchart 0.1.1 this still said "the Figure element is empty": the chart was marked entirely as an artifact, and the `Figure` had no content.

21.7 What Is Deliberately Missing

- **"Page X of Y".** `toc::document` gives the total only afterwards (`contentPages` plus `tocPages`), when the footers have already been drawn. The manual route from Table of Contents knows it earlier: after the trial run with `toc::collect`, the throwaway document has `pageCount` pages, plus the pages from `toc::pageCount`.
- **A cover page.** `toc::document` writes the table of contents as the first page. A cover page before it is only possible by hand, with `collect`, `pageCount` and `render`.
- **Multi-line cells.** `table::draw` truncates texts that are too long; for remarks, `table::render` would be the choice.
- **Numbers with a decimal comma.** `format %.2f` writes a period. For a German report, a `string map {.,}` belongs in `betrag`; in the chart this is not possible, `-format` knows no decimal comma.

21.8 Summary

- Load and reshape the data before `toc::document`; the content runs twice.
- `group` for the long table, `pivot` for the overview, the chart values from the crosstab.
- `-pagebreakcmd` with `global pdf`: the throwaway document in the first pass, the real one in the second.
- Gaps around charts by hand; `chart::bar` ends at its bottom edge.
- `tagged`, `metadata -title`, `displaydoctitle`; the building blocks do the rest.

Next chapter: Pitfalls

22 Pitfalls

Short list, by topic. The reasons are in the chapter named.

22.1 Setup

- `package require pdf4tcllib` does not load pdf4tcl; write both lines (Setup).
- Tcl 8.6 reads scripts in the system encoding. Without a UTF-8 locale, umlauts in the source turn into "Ã¼" in the PDF.
- Without a matching TrueType file everything carries on with Helvetica; only stderr says so. Check `fonts::hasTtf`.
- The default family `DejaVuSansCondensed` is in `fonts-dejavu-extra` on Debian, not in `fonts-dejavu-core`.
- File names follow the pattern `<familie>.ttf`, `<familie>-Bold.ttf`; Liberation fonts do not match.
- CID mode is fixed by the first `fonts::init`. A later `fonts::init -cid 1` (even with `-force 1`) reloads nothing, but `fonts::isCidMode` then reports 1.

22.2 Text and Characters

- `unicode::sanitize` follows `fonts::init`, not `setFont`. Helvetica after `fonts::init` loses →, ✓ and box-drawing lines without a message (Unicode).

- `safeText` always turns "..." into "...", even in CID mode; so do the headers and footers from `page::`.
- `text::writeParagraph` does not call `sanitize`. Missing characters are then counted by pdf4tcl (`$pdf getSubstCount`), not by pdf4tcllib.
- `unicode::substCount` counts across the whole process; call `resetSubst` before each document.
- 256-character mode has no German quotation marks.

22.3 Setting Text

- `text::width`, `wrap`, `truncate`, `fits` without `$pdf` only estimate the width (Setting Text).
- `text::wrap` shortens a word that is too wide on its own to "..."; `hardBreak 1` splits it instead.
- In `text::wrap`, `pdf` is the sixth argument, after `code`; in `fitSize` it is called `-pdf`.
- `text::writeParagraph` always sets the regular font and does not break across pages.
- After a footer or header, its font is set; call `setFont` again before the next text.

22.4 Pages and Columns

- "Page X of Y" needs two passes; the content must not have side effects (Page Numbers).
- `-pagelabel` is a `format` string: write `%` as `%%`.
- `flow::columns` without `-newpage` returns the remainder in `rest` instead of breaking (Column Flow).
- `flow::columns` puts the first baseline at `-top`; without it the first line reaches into the margin.
- `-firsty` applies only to the first column; `-align` has no effect.
- Draw `drawing::watermark` before everything else; with `-orient 0` it lands outside the page.

22.5 Tables and Charts

- `table::draw` without `-ctx` does not break; without `-pagebreakcmd` there is no header or footer on the continuation pages (Tables).
- `-pagebreakcmd` runs in the global scope and must return the new y position.
- Cells are single-line and truncated with "...", the header row included; `-notruncate 1` for values that will be typed in by hand.
- A block of rows joined by `rowspan` is not split; if it is taller than a page, `table::draw` aborts with an error.
- `table::group` does not sort; unsorted data yields several subtotals per group (Groups).
- Totals from `group` and `pivot` are raw Tcl numbers; format them before drawing.
- `table::pivot` sorts the columns alphabetically; use `-colorder` for months and quarters.
- `chart::line` does not show the series names; draw the legend yourself (Charts).
- The `-format` of the charts knows no decimal comma.

22.6 Documents

- The script for `toc::document` runs twice; no side effects. `toc::document` builds its own context and overwrites `ctx` (Table of Contents).
- `toc::heading` does not set bold and does not wrap.
- `-gap` and `-bookmarks` of `toc::document` apply only to that call; on the manual route `toc::configure` counts.

- A cover page before the table of contents is only possible via `collect`, `pageCount`, `render`.
- Form fields with the fonts from `fonts::init`: boxes in choice lists in CID mode, wrongly stored umlauts otherwise. Register a `cp1252` font and set it with `form::configure` (Form Layout).
- Without `form::configure`, `orderTable` and `sumLine` always use Helvetica-Bold.
- A checkbox in a form with `-calculate` or `-format` is invisible in Poppler viewers; draw the frame.
- `labelField ... checkbox` without `-fieldw` is as wide as the row.
- `labels::render` starts the pages itself; no `startPage` before it (Label Sheets).
- `labels::fitSize` stops at `min` even if the text does not fit; measure again. Arguments differ from `text::fitSize`.
- Print at actual size, not "fit to page".
- A JPEG loaded before the first `startPage` starts a page (Images).
- `image::gallery -caption` knows only `unten` and `keine`; other values silently turn the captions off.
- `roundedRect -clip 1` clips with square corners (Arrows and Callouts).
- Without `fonts::init -cid 1`, formula symbols turn into? (Formulas).
- `renderLatex` puts no space around `=`; insert `\,`. Set the font again afterwards.
- A script with Tk needs `exit` at the end and a display (Treeview).
- A CID font set with `setFont` but never used ends up in the file in full: about 330 KB for DejaVu Sans Bold.

22.7 Accessibility and Structure

- `tagged 1 -ua 1` does not set `DisplayDocTitle`; PDF/UA requires it: `viewerPreferences -displaydoctitle 1` (Accessible PDFs).
- `writeParagraph` sets `P` when no type is given; for headings, pass `H1` etc. as the eighth argument. It always sets the regular font.
- To a screen reader, a chart is only its alternate text. Give important figures as a table as well. (Up to pdf4tclchart 0.1.1 the `Figure` stayed without content, and pdf4tcl warned.)
- Put anything drawn by hand inside `tag::begin`/`end` or mark it as an artifact; otherwise `getUntaggedCount` is not 0.
- Building blocks leave their font and color behind; `state::with` (Graphics State).
- `state::with` restores what is passed, not what was set.
- `gsave`/`grestore` across a page break yields unbalanced streams, without a message.
- `measure::block`: `pages` stays 1 if the block does not break by itself; read `overflow` (Measuring and Stacking).
- In scripts for `layout::stack`, `$bpdf` instead of `$pdf`.
- `layout::stack` checks only blocks with `keep 1`.
- `page::header` draws its rule 2 pt below `top`; content starting at `top` gets struck through.

22.8 Page

- pdf4tcl object without `-unit mm`: pdf4tcllib calculates in points, otherwise text lands outside the page (Page Context).
- The pdf4tcl option `-margin` together with `page::context` yields double the margin.
- `-orient` in the context and in the pdf4tcl object must match.
- `page::context a4 20 0`: the second word is the orientation, not landscape.

- y is the baseline: text at `y = top` reaches into the margin.
- `page::footer` writes "Seite N", even in an English document; set `pageLabelFormat`.
- `page::grid` is a helper grid for development, not a column grid.
- `page::centerText` without a font argument sets Helvetica, even after `fonts::init`; pass the font.

Next chapter: API per Module

23 API per Module

Quick reference for pdf4tcllib 0.6.7 and the add-on packages. The call forms are read from the procedures themselves (`info args`); arguments in?…? have a default. Many argument names are German, for example `datei` (file), `breite` (width), `zeilen` (rows), `spalten` (columns), `wert` (value), `farbe` (color). The options are a selection. Complete and in English: `docs/en/reference/` in the pdf4tcllib source tree.

All names live in the namespace `pdf4tcllib::`, which is left out here: `fonts::init` written out in full is `pdf4tcllib::fonts::init`.

23.1 Packages

Package	Version	Namespace	Chapter
pdf4tcllib	0.6.7	fonts unicode text math page tag state measure layout table drawing image units form	2 to 20
pdf4tclchart	0.1.2	chart	10
pdf4tclflow	0.1	flow	7
pdf4tcltoc	0.1.1	toc	11
pdf4tcllabels	0.3	labels	13
pdf4tclforms	0.2.3	forms	–
pdf4tclcanvas	0.1	canvas	–
pdf4tcltable	0.3.2	tablelist	–
pdf4tcltext	0.3	textwidget	–
pdf4tcltreeview	0.1.1	treeview	17

"–": no chapter of its own in the book, only the calls here.

```
package require pdf4tcl
package require pdf4tcllib
puts [pdf4tcllib::version]          ;# 0.6.7
```

23.2 Fonts: fonts

```
fonts::init ?-cid 0|1? ?-fontdir dir? ?-family name? ?-force 0|1?
fonts::ready
fonts::hasTtf
fonts::hasTtfItalic
fonts::isCidMode
fonts::fontSans
fonts::fontSansBold
fonts::fontSansItalic
fonts::fontSansBoldItalic
fonts::fontMono
fonts::setFont pdf size ?family? ?style?
fonts::isMonospace fontName
fonts::widthFactor fontName
fonts::subset
fonts::inSubset codepoint
```

`init` once per process; CID mode applies from the first call on. `style` for `setFont`: `Bold`, `Italic`, `BoldItalic`.

23.3 Characters: unicode

```
unicode::sanitize text ?-mono 0|1? ?-tabwidth n?
unicode::safeText pdf text ?-x x -y y -align a …?
unicode::readFile datei
unicode::preprocessBytes bytes
unicode::substCount
unicode::substReport                ;# dict: count, chars
unicode::resetSubst
```

23.4 Setting Text: text

```
text::width text size font ?pdf?
text::wrap text maxW size font ?code? ?pdf? ?hardBreak? ?trenner?
text::truncate text maxW size font ?pdf?
text::fits text maxW size font ?pdf?
text::fitSize text maxW wunschgroesse font ?-min n? ?-pdf pdf?
text::writeParagraph pdf text x y breite ?size? ?align? ?tag?
text::expandTabs zeile ?tabbreite?
text::detectFont zeile
text::superscript pdf text x y size font
text::subscript pdf text x y size font
text::mathSymbol name
text::mathSymbolNames
```

`writeParagraph` returns the y below the paragraph. `trenner` (separators) for `wrap`: default `{- /}`.

23.5 Formulas: math

```
math::renderFormula pdf x y formel ?-size n? ?-font f?
math::analyseFormula formel
math::renderLatex pdf x y latex ?-size n? ?-font f?
math::measureLatex pdf latex ?-size n?
              ?-font f?   ;# {breite oben unten}
math::pdfBackend pdf
math::canvasBackend canvas ?-family f? ?-fill farbe? ?-tags t?
math::renderLatexOn backend x y latex ?-size n? ?-font f?
math::measureLatexOn backend latex ?-size n? ?-font f?
```

23.6 Page: page, units

```
page::context papier ?-margin mm? ?-orient 0|1? ?-landscape 0|1?
page::lineheight size ?faktor?
page::header pdf ctx text ?size?
page::footer pdf ctx text seite ?size? ?-pagelabel format?
page::number pdf ctx seite ?gesamt? ?size?
page::centerText pdf ctx text y ?size? ?font?
page::grid pdf ctx spalten spalte
page::debugGrid pdf ctx ?abstand?
page::orientationLegend pdf ctx

units::mm wert      units::to_mm pt
units::cm wert      units::to_cm pt
units::inch wert    units::to_inch pt
```

Keys of the context: `page_w page_h left top bottom text_w text_h margin`, plus the short forms `PW PH SX SY SW SH`. Default for `pageLabelFormat`: `Seite %s`.

23.7 Structure Tree: tag

```
tag::begin pdf typ ?-alt t? ?-scope s? ?-colspan n? ?-rowspan n?
              ?-summary t?
tag::end pdf
tag::artifact pdf ?-type Layout|Pagination|Page?
tag::artifactEnd pdf
tag::isActive pdf
tag::depth pdf
tag::forget pdf
```

Without `$pdf tagged 1`, none of them does anything.

23.8 State, Measuring, Stacking: state, measure, layout

```
state::save pdf ?size? ?font? ?fill? ?stroke?
state::restore pdf zustand
state::with pdf {size font} {r g b} skript

measure::block varName skript ?-paper p? ?-orient 0|1? ?-margin mm?
              ?-landscape 0|1? ?-y y?
measure::height varName skript ?option …?
measure::fits platz varName skript ?option …?

layout::stack pdf x y breite bloecke ?-ctx ctx? ?-gap n?
              ?-font {size name}? ?-fill {r g b}? ?-pagecmd cmd?
              ?-paper p?
```

`measure::block` returns a dict: `height pages value overflow sheet`. Blocks for `layout::stack`: `{script {…} ?keep 1?}` or `{gap n}`; in the script `$bpdf $bx $by $bw`.

23.9 Tables: table

```
table::draw pdf x y spalten zeilen ?option …?
table::simpleTable pdf x y breiten zeilen ?-zebra 0|1? ?-pad n?
              ?-header_bg {r g b}? ?-row_height n? ?-font_size n?
table::render pdf daten x0 yVar maxW yTop yBot pageNoVar
              pageW pageH margin size lineH ?debug? ?pageBreakCmd?
table::group zeilen ?-by n? ?-sum {…}? ?-label fmt? ?-total 0|1?
              ?-totallabel t? ?-header 0|1? ?-blank 0|1? ?-strict 0|1?
table::pivot zeilen ?-row n? ?-col n? ?-value n?
              ?-agg sum|count|avg|min|max? ?-empty t? ?-total 0|1?
              ?-rowlabel t? ?-totallabel t? ?-colorder {…}?
table::widths spalten zeilen ?-pdf pdf? ?-fontsize n? ?-pad n?
table::breakPoints zeilen ?-fontsize n? ?-rowheight n? ?-header 0|1?
              ?-groups {…}?
table::fitRows platz zeilen ?option …?
```

Options of `draw`: `-ctx -maxwidth -header -headerbg -headerfg -zebra -zebracolor -zebrastart -fontsize -pad -border -rowheight -cellstyles -rowstyles -rowindent -footer -footerbg -footerbold -yvar -pagevar -orient -pagebreakcmd -fontreg -fontbold -summary`. Since 0.6.7 any other option is an error.

Per column: `-header -width (number or auto) -align -font -notruncate`. A cell may be a dict `{text … colspan n rowspan n}`.

23.10 Drawing: drawing

```
drawing::frame pdf x y w h ?linienbreite?
drawing::separator pdf x y w ?farbe? ?linienbreite?
drawing::roundedRect pdf x y w h r ?stroke? ?fill? ?-clip 1?
drawing::polygon pdf cx cy radius ecken ?stroke? ?fill?
drawing::star pdf cx cy radius ?zacken? ?verhaeltnis? ?stroke? ?fill?
drawing::gradient_v pdf x y w h farbe1 farbe2 ?schritte?
drawing::gradient_h pdf x y w h farbe1 farbe2 ?schritte?
drawing::interpolate farbe1 farbe2 t
drawing::arrow pdf x1 y1 x2 y2 ?-at end|start|both? ?-length n?
              ?-width n?
              ?-filled 0|1?
drawing::arrowHead pdf x y winkel ?-length n? ?-width n? ?-filled 0|1?
drawing::callout pdf x y w h zielX zielY text
              ?-radius -fill -stroke -tail
              -pad -fontsize -font -alt?
drawing::watermark pdf ctx text ?-angle grad? ?-size n?
              ?-color {r g b}?
              ?-layer 1|id?
drawing::textRotated pdf text x y winkel size ?font?
drawing::textScaled pdf text x y sx sy size ?font?
drawing::textSkewed pdf text x y skewX skewY size ?font?
```

`arrowHead` takes the angle in radians.

23.11 Images: image

```
image::place pdf id x y w h ?-fit contain|cover|stretch? ?-anchor a?
              ?-dpi n? ?-alt t?
image::gallery pdf bilder x y w h ?-cols n? ?-rows n? ?-gap n?
              ?-caption unten|keine? ?-captionH n? ?-fontsize n?
              ?-font f?
              ?-fit …? ?-anchor …? ?-dpi n? ?-pagecmd cmd?
image::insert pdf tkBild x yVar maxW yTop yBot pageNoVar
              pageW pageH margin size ?debug?
image::insertAt pdf tkBild xPos yVar maxW yTop yBot pageNoVar
              pageW pageH margin size ?debug?
```

`place` returns the occupied area `{x y w h}`, `gallery` the number of pages. `insert` and `insertAt` need Tk.

23.12 Form Layout: form

```
form::configure ?-option wert …?
form::fieldHeight
form::rowHeight
form::section pdf ctx yVar titel
form::labelField pdf ctx yVar label typ ?-labelw n? ?-fieldw n?
```

```
              ?-fieldh n?
              ?addForm-Optionen …?
form::row pdf ctx yVar felder
form::separator pdf ctx yVar ?abstand?
form::orderTable pdf ctx yVar koepfe breiten ?daten? ?-emptyRows n?
              ?-rowh n? ?-headerBg {r g b}? ?-cellForm präfix?
              ?-cellOpts {…}?
form::sumLine pdf ctx yVar breiten label wert ?-id id?
              ?-calculate {op {felder}}?
              ?-init wert?
```

All of them advance `yVar`. A field in `form::row`: a dict with `label type width` and optionally `id init options labelw fieldh readonly multiline`. Background and border of the fields come from `fieldBg`, `fieldBorder` and `fieldBorderW` of `form::configure` when the call does not give its own.

23.13 Charts: chart (pdf4tclchart)

```
chart::bar pdf x y w h daten ?option …?
chart::line pdf x y w h reihen ?option …?
chart::pie pdf x y w h daten ?option …?
chart::legend pdf x y paare optionen
chart::niceScale min max ?teilungen?        ;# {min max schritt}
chart::configure ?-option wert …?
```

Options: `-title -titlesize -labelsize -valuesize -values -format -color -colors -grid -gridcolor -axiscolor -min -max -legend -alt`. Data: `{name wert name wert …}` or `{{name wert} …}`; series for `line`: `{{name {paare}} …}`. Returns the y below the chart.

23.14 Column Flow: flow (pdf4tclflow)

```
flow::columns pdf ctx text ?-columns n? ?-gap n? ?-size n? ?-font f?
              ?-tag typ? ?-top y? ?-bottom y? ?-firstx x? ?-firsty y?
              ?-newpage skript?
flow::boxes ctx ?option …?
flow::measure pdf text breite size font
flow::configure ?-option wert …?
```

Without `-newpage`, `columns` returns the remainder instead of breaking; `-newpage` is a script and runs in the caller's scope.

23.15 Table of Contents: toc (pdf4tcltoc)

```
toc::document pdf papier skript ?-title t? ?-titlesize n? ?-size n?
              ?-leader z? ?-indent n? ?-gap n? ?-bookmarks 0|1?
toc::heading pdf ctx yVar ebene text ?-size n? ?-gap n? ?-tag typ?
toc::collect skript ?versatz?
toc::pageCount ctx eintraege ?option …?
toc::render pdf ctx eintraege ?option …?
toc::entries
toc::configure ?-bookmarks 0|1? ?…?
```

`document` returns a dict: `entries tocPages contentPages`. An entry is `{ebene titel seite}` (level, title, page).

23.16 Labels: labels (pdf4tcllabels)

```
labels::sheets
labels::sheet name
labels::define name spec
labels::place geometrie index               ;# {x y w h}
labels::render pdf name datensaetze {x y w h satz} skript
              ?-start n? ?-only {…}? ?-offsetx mm? ?-offsety mm?
              ?-frame 0|1|layer? ?-tag typ?
labels::fitSize pdf font text maxW size ?min?
labels::wrap pdf text maxW
labels::ellipsize pdf text maxW ?ende?
labels::calibration pdf name
```

`render` starts the pages itself. The fourth argument names the five variables the script receives per label.

23.17 Fillable Forms: forms (pdf4tclforms)

```
forms::renderSchema pdf ctx spec ?-yvar name? ?-pagebreak 0|1?
forms::renderSection pdf ctx yVar abschnitt ?pagebreak?
forms::field pdf ctx yVar feld ?pagebreak?
forms::checkboxLine pdf ctx yVar feld ?pagebreak?
forms::radioGroup pdf ctx yVar feld ?pagebreak?
forms::buttonBar pdf ctx yVar feld ?pagebreak?
forms::signatureLine pdf ctx yVar feld ?pagebreak?
forms::entryTable pdf ctx yVar tabelle ?pagebreak?
forms::template name
              ?…?          ;# callnote inventory checklist order
forms::read datei ?-schema s? ?-strict 0|1? ?-raw 0|1?
forms::write ein aus werte ?-schema s? ?-strict 0|1?
```

23.18 Tk Widgets: canvas, tablelist, textwidget, treeview

```
canvas::place pdf canvas x y w h ?-fit contain|stretch? ?-anchor a?
              ?-alt t?
              ?-caption t? ?-captionH n? ?-fontsize n? ?-font f?
              ?-bbox {…}? ?-fontmap {…}? ?-textscale f? ?-border 0|1?
canvas::fitPage pdf canvas ctx ?option …?

tablelist::render pdf tbl x y ?-maxwidth -fontsize -rowheight -zebra
              -border -tree -indentW -formatted -headerbg -headerfg
              -footer -footerbg -footerbold -ctx -yvar -font
              -boldfont?
tablelist::renderRange pdf tbl x y ?-firstrow n? ?-lastrow n? ?…?
tablelist::report                    ;# was beim letzten render fehlte

textwidget::render pdf textwidget x y ?-maxwidth n? ?-fontsize n?
              ?-fontfamily f? ?-linespacing n? ?-skipelided 0|1?
              ?-skipinternal 0|1? ?-ctx ctx? ?-yvar name?

treeview::render pdf tv x y ?-maxwidth -fontsize -rowheight -tree -all
              -zebra -border -indentW -ctx -yvar -pagevar -font
              -boldfont
              -summary?
treeview::outline pdf tv x y
              ?-fontsize -indentW -lineH -all -bookmarks
              -font -yvar?
```

All four need Tk and a display.

23.19 Miscellaneous

```
pdf4tcllib::version
pdf4tcllib::validate_pdf datei     ;# dict: valid size error
```

Next chapter: Glossary

24 Glossary

256-character mode (256-Zeichen-Modus) — default of `fonts::init`: ASCII, Latin-1 and 32 selected characters in a simple encoding.

Accessor procedure (Zugriffsprozedur) — `fonts::fontSans` and related procedures; they return the name under which `fonts::init` registered a font.

Alternate text (Ersatztext) — description of a figure for readers who cannot see it; `-alt`.

Appearance (Erscheinung) — what a form field shows on the page as long as nobody clicks into it. With `/NeedAppearances` the viewer rebuilds it.

Artifact (Artefakt) — content without meaning for the text (rules, header, page number); a screen reader skips it. `tag::artifact`.

Baseline (Grundlinie) — the line the letters stand on; `y` in `$pdf text` refers to it.

Bookmark (Lesezeichen) — entry in the outline at the side of the viewer; `toc::heading` creates one per heading.

CID mode (CID-Modus) — font with Identity-H encoding; every character of the TrueType file is available. Turn it on with `fonts::init -cid 1`.

Frame (image) (Rahmen) — the area into which `image::place` fits an image; `contain` leaves space free, `cover` crops.

Page context (Seitenkontext) — the dict from `page::context` with all measurements of the page.

Point (Punkt) — unit of measurement of PDF and pdf4tcl, 1/72 inch, about 0.353 mm.

Structure tree (Strukturbaum) — the outline of an accessible PDF in headings, paragraphs, tables and figures; `$pdf tagged 1`.

Type area (Satzspiegel) — the area inside the margins where the content goes; `left`, `top`, `text_w`, `text_h` in the page context.

Next chapter: Error Messages

25 Error Messages

The messages of pdf4tcllib 0.6.7 and the add-on packages, by module, with the usual cause. The texts are shown exactly as Tcl prints them; **…** stands for the inserted value.

There are three kinds:

- **Errors** abort the call (`catch` catches them).
- **Warnings** end up in `::pdf4tcl::warnings`, a list that nobody empties automatically. Run `set ::pdf4tcl::warnings {}` before the document and read the list afterwards.
- **Notes on stderr** are written by pdf4tcllib itself, usually only once per process.

25.1 For All Building Blocks

25.1.1 …: unknown option -… (allowed: …) / (known: …)

A typo, or an option that does not exist in this version. The message names the allowed ones. Since 0.6.7 this also applies to `table::draw`; before that, any option passed there silently.

25.1.2 …: expected an even number of option/value words

25.1.3 …: expected option/value pairs

An option is missing its value, often because a list is not braced: `-colors $a $b` instead of `-colors [list $a $b]`.

25.2 Page and Fonts

25.2.1 unknown paper size: … (known: …)

`page::context` does not know the paper. Upper and lower case do not matter; the list in the message names the known names.

25.2.2 pdf4tcllib::page::footer: unknown option …

In `footer` the size is the fifth argument, the options come after it: `page::footer $pdf $ctx "Text" $seite 9 -pagelabel "Seite %s"`.

25.2.3 pdf4tcllib::fonts: No TTF fonts found, falling back to Helvetica

Note on stderr, followed by the folders searched. From here on the 256-character mode with Helvetica applies; umlauts work, →, ✓ and typographic quotation marks do not. On Debian, `fonts-dejavu-extra` is usually missing (Setup and Fonts).

25.2.4 pdf4tcllib::fonts: TTF error: … / Fallback on Helvetica

The font file was found but could not be loaded: damaged, not a TrueType file, or locked by another program.

25.2.5 pdf4tcllib::unicode: sanitize: at least one character is missing from the font and was replaced with '?' (first: U+…). …

Note on stderr, once per process: characters were missing from the font and became?. `unicode::substReport` says which. For Greek and formula symbols, use `fonts::init -cid 1` (Safe Unicode Output).

25.2.6 pdf4tcllib::math: unknown symbol '\…' rendered as literal text

`math::renderLatex` does not know the command and sets it as text. `text::mathSymbolNames` lists the known names.

25.3 Text, Measuring, Stacking

25.3.1 text::fitSize: unknown option …

`fitSize` takes `-min` and `-pdf`; here `pdf` is not at a fixed position as in `text::wrap`.

25.3.2 layout::stack: an entry needs a script (or must be {gap })

A block in the list has neither `script` nor the form `{gap n}`. Common cause: the blocks were passed as one script instead of a list of dicts.

25.3.3 measure::block: unknown option …

Allowed: `-paper -orient -margin -landscape -y`.

25.4 Tables

25.4.1 table::draw: -orient 0 is not supported. …

The document was created with `-orient 0` (y pointing up). `table::draw` only draws with y pointing down. Create the pdf4tcl object with `-orient 1`, the default (Units and Page Context).

25.4.2 table::draw: rows … to … are joined by rowspan and need … pt, more than a page holds (… pt)

A block of rows joined by `rowspan` is never split. Here it does not fit on any page. Reduce `rowspan` or split the group into several blocks.

25.4.3 table::group: -by … is outside the row (0…..)

The group column does not exist; counting starts at 0.

25.4.4 table::group: the rows are not sorted by column …; a group falls into several blocks and each gets its own subtotal

Warning in `::pdf4tcl::warnings`, an error with `-strict 1`. Sort the data by the group column beforehand, for example with `lsort -index 0`. `group` deliberately does not sort.

25.4.5 table::pivot: -agg must be sum, count, avg, min or max, not "…"

25.4.6 orderTable: column widths total … pt, the text width is only … pt -- the table would be drawn past the edge of the page

`form::orderTable` does not fit into the type area. Reduce the widths or use a smaller margin in the context.

25.4.7 orderTable: … header(s) but only … column width(s)

One width per header.

25.4.8 tablelist::render: … could not be reproduced (…). …

Warning from `pdf4tcltable`: images, embedded windows or wrapped cells of the widget do not make it into the PDF. `tablelist::report` says what exactly.

25.5 Charts

25.5.1 chart::…: no data

25.5.2 chart::…: data must be label/value pairs or a list of {label value}, got … element(s)

An odd-length list: `{Jan 120 Feb}`. Empty values must be written as a pair, e.g. `{Feb 0}`.

25.5.3 chart::bar: value for "…" is not a number: "…"

Applies equally to `line` and `pie`. An empty field is not zero (Charts).

25.5.4 chart::bar: box too small -- … x … pt leaves no room for axis and labels

The box is smaller than axis and labels together. Increase the height or make `-labelsize` smaller.

25.5.5 chart::pie: negative value for "…" -- a slice of a whole cannot be negative

25.5.6 chart::pie: the values add up to 0

25.6 Column Flow and Table of Contents

25.6.1 flow::…: -columns must be at least 1

25.6.2 flow::boxes: … columns with a gap of … pt leave no width in … pt

Too many columns or too large a `-gap` for the width.

25.6.3 toc::heading: level must be 1..6, got "…"

25.6.4 toc::document: the contents was calculated as … page(s) and written as … -- every page number in it would be off by …

`pageCount` and `render` ran with different options, or `toc::configure` was changed in between. The table of contents would be numbered wrongly, so `document` aborts (Table of Contents).

25.7 Images and Drawing

25.7.1 image::place: -fit must be contain, cover or stretch, not "…"

25.7.2 image::place: width and height must be positive

25.7.3 image::place: image … has no size

The id does not come from `$pdf addImage`, or the image is empty.

25.7.4 image::place: … at … dpi (…x… pixels on …x… points), below …

Warning with `-dpi`: the image resolution is too coarse for the area.

25.7.5 image::gallery: -cols and -rows must be at least 1

25.7.6 image::gallery: the frame is too small for …x… cells

The frame is not large enough for the grid, usually because of `-gap` or `-captionH`. Up to 0.6.6 the result was `invalid command name "2"` instead: building the message ran the column count as a command.

25.7.7 pdf4tcllib::image needs Tk: …

`image::insert` and `insertAt` work with Tk images. For files without Tk: `$pdf addImage` and `image::place`.

25.7.8 drawing::arrow: -at must be end, start or both, not "…"

25.7.9 drawing::callout: width and height must be positive

25.7.10 drawing::watermark: no text

25.7.11 canvas::place: no such widget: …

25.7.12 canvas::place: no room left for the drawing after the caption

`-captionH` is as large as the box.

25.7.13 canvas::fitPage: the context leaves no height

25.8 Labels

25.8.1 unknown label sheet "..."; known: ...

The sheet is not registered. Define custom sheets with `labels::define` (Label Sheets).

25.8.2 label format ...: missing "..."

`labels::define` without one of the required keys.

25.8.3 label format ...: paper "roll" means one label per page, so cols and rows must both be 1

25.8.4 argSpec must name five variables: {x y w h record}

The fourth argument of `labels::render`.

25.8.5 -start must be between 0 and ... for sheet ...

25.8.6 -only: "..." is not a position on sheet ... (0.....)

25.8.7 -only names ... position(s) for ... record(s)

One position per record.

25.8.8 -frame: 0, 1 or layer, not "..."

25.8.9 -frame layer needs pdf4tcl 0.9.4.70 or newer: ... ignores -print 0, the grid would print

25.8.10 -frame layer under -pdfa ...: PDF/A forbids /AS, so -print 0 has no effect and the grid would print; use -frame 1 on a separate proof sheet

The helper grid sits on a layer that is not meant to be printed. Older pdf4tcl versions and PDF/A cannot do that; the grid would end up on the label.

25.9 Fillable Forms

25.9.1 forms::read: cannot read "..." / forms::write: cannot read "..."

The file is missing or is not a PDF with a form.

25.9.2 forms::write: ... has no field(s): ...

Warning, an error with `-strict 1`: these names do not exist in the form. Often the full name with a dot is missing (`person.city`, not `city`).

25.9.3 forms::write: nothing to fill in

The dict with the values is empty, or none of the names occurs in the form.

25.9.4 pdf4tclforms::renderSchema: spec needs 'sections' key

25.9.5 pdf4tclforms::renderSchema: 'sections' must be a dict (name -> section), not a list of sections

25.9.6 pdf4tclforms: field entry must be a dict, got: ...

25.9.7 pdf4tclforms::template: unknown template '...' (callnote inventory checklist order)

25.10 Tk Widgets

25.10.1 treeview::render: no such widget: … / treeview::outline: no such widget: …

The path is wrong, or the widget has already been destroyed.

25.11 From pdf4tcl

Two warnings from pdf4tcl itself that are common with pdf4tcllib:

25.11.1 tagged: the … element is empty -- no content, no child element and no annotation. …

A structure element without content. Up to pdf4tclchart 0.1.1 every chart triggered it (`Figure`); since 0.1.2 it no longer does. If it remains, there is a custom `tag::begin`/`tag::end` pair with nothing drawn in between.

25.11.2 … must contain at least one …

Error when closing an element, for example `Table` without `TR`. Occurs with hand-built tables, not with `table::draw` (Accessible PDFs).

Back to the beginning

26 License


```
Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Attribution-ShareAlike 4.0 International
(CC BY-SA 4.0)
The author's code examples: MIT License (below)

---

Text

This book is licensed under the Creative Commons license
"Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0).

You may copy and redistribute the material in any medium or format and
remix, transform and build upon it, for any purpose, even commercially,
under the following terms:

* Attribution: You must give appropriate credit, provide a link to the
  license, and indicate if changes were made.
```

```
* ShareAlike: If you remix, transform, or build upon the material, you
  must distribute your contributions under the same license as the
  original.

The full license text is binding:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

---

Code examples

The author's code examples (in the code blocks of the book and as
scripts next to it) are additionally released under the MIT License.
Code examples and quoted text taken from other projects (Tcl/Tk,
pdf4tcl and others) keep the license of their original; the source is
named where they appear.

MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Index

A

artifact 83

C

chart::legend 49
chart::niceScale 50

D

drawing 68
drawing::arrow 69
drawing::arrowHead 69
drawing::callout 69
drawing::roundedRect 69
drawing::watermark 38

F

flow::boxes 35
form 56
form::labelField 56
form::orderTable 56
form::row 56
form::section 56
form::sumLine 56

I

image::gallery 64
image::place 64

L

labels::fitSize 63
labels::render 63
labels::sheet 61
layout::stack 88

M

math 72
math::measureLatex 73
math::renderFormula 75
math::renderLatex 73
measure 88
measure::block 88
measure::fits 88
measure::height 88

P

page::_advance 25
page::context 22
page::lineheight 25

PDF/UA 81
pdf4tcl 9
pdf4tclchart 48
pdf4tclflow 35
pdf4tcllabels 61
pdf4tcllib 9
pdf4tcltoc 51
pdf4tcltreeview 76

S

state::with 85
structure tree 81

T

table::draw 38
table::fitRows 42
table::group 43
table::pivot 43
table::widths 42
tag 84
text 27
text::fitSize 29
text::truncate 29
text::writeParagraph 30
toc::collect 54
toc::document 51
toc::heading 51
treeview::outline 77
treeview::render 77

U

unicode::readFile 21
unicode::sanitize 17
units 22

V

veraPDF 85